

Федеральное государственное бюджетное учреждение науки «Институт проблем передачи информации им. А.А. Харкевича Российской академии наук



Институт проблем
передачи информации
им. А. А. Харкевича
Российской академии
наук

На правах рукописи

Чеканов Михаил Олегович

**Алгоритмы комплексирования разнородных данных при
построении модели окружающей сцены беспилотного
транспортного средства**

Специальность 2.3.8 —

«Информатика и информационные процессы»

Диссертация на соискание учёной степени

кандидата технических наук

Научный руководитель:
кандидат физико-математических наук
Николаев Илья Петрович

Москва — 2025

Оглавление

	Стр.
Введение	5
Глава 1. Анализ методов построения моделей окружающего пространства БТС на основе комплексирования данных разной природы	
1.1 Современное состояние беспилотных транспортных средств . . .	9
1.2 Модели окружающего пространства БТС	11
1.3 Обзор используемых в колёсной робототехнике датчиков и их ограничения	15
1.4 Методы комплексирования данных	18
1.5 Метрики оценки качества моделей окружающего пространства .	19
1.6 Заключение к главе 1	23
Глава 2. Алгоритмы проецирования визуальных данных на плоскость движения БТС	
2.1 Алгоритм проекции визуальных детекций на плоскость движения БТС на основе модели L-Shape	24
2.1.1 Этап 1. Проекция 2D-детекции изображения на вид сверху	24
2.1.2 Этап 2. Вписывание проекции объекта на виде сверху в ориентированную ограничивающую рамку.	27
2.2 Исследование точности алгоритма проекции визуальных детекций на основе модели L-Shape	30
2.2.1 Набор данных	30
2.2.2 Сравниваемые алгоритмы	31
2.2.3 Описание эксперимента	34
2.2.4 Результаты экспериментов.	34
2.3 Алгоритм проекции визуальных детекций на плоскость движения БТС на основе комплексирования данных с данными лазерных дальномеров	36
2.3.1 Разделение облака точек на точки земли и препятствий .	39

2.4	Алгоритм проекции сегментации проезжей части на изображении на плоскость движения ТС	43
2.4.1	Вычисление функции $z(x, y)$	49
2.4.2	Модификация алгоритма для обнаружения препятствий на проезжей части	53
2.5	Заключение к главе 2	54

Глава 3. Программный комплекс построения карты проходимости пространства на основе комплексирования разнородных данных

3.1	Общие сведения	60
3.2	Назначение и функциональные возможности	60
3.3	Структура программного комплекса	61
3.3.1	Библиотека «occupancy_map_fusion»	65
3.3.2	Программа «evo_occupancy_map_fusion»	75
3.4	Заключение к главе 3	76

Глава 4. Неблокирующий алгоритм построения карты проходимости пространства на основе комплексирования разнородных данных

4.1	Introduction	78
4.2	Постановка задачи	79
4.3	Модификации базовой реализации	84
4.3.1	Base OGM	84
4.3.2	OGM with prefetched areas	87
4.3.3	OGM with lockfree cell value update	88
4.3.4	Fully lockfree OGM	92
4.4	Сравнение задержки обновления локальной карты проходимости пространства	95
4.4.1	Сдвиг карты	96
4.4.2	Добавление фрагмента в одном потоке	96
4.4.3	Добавление фрагмента в нескольких потоках	97
4.4.4	Получение карты	98
4.4.5	Стандартный цикл "публикации" карты	101

	Стр.
4.5 Реализация для 2D случая	102
4.6 Выводы	102
4.7 Заключение к главе 4	105
Заключение	106
Список сокращений и условных обозначений	107
Список литературы	108
Список рисунков	114
Список таблиц	117

Введение

Последние десятилетия характеризуются активным внедрением и распространением беспилотных транспортных средств (БТС) в различных сферах человеческой деятельности. Примеры таких сфер – логистика, промышленность, сельское хозяйство, горное дело, городская инфраструктура. Для осуществления успешной навигации БТС требуется модель окружающего пространства. В зависимости от выполняемых задач к данным моделям предъявляются различные требования, включая количество параметров, описывающих модель, разрешающую способность, вычислительную сложность построения. Построение модели производится на основании показаний различных датчиков, которыми оснащается БТС. Расположение, количество и тип используемых датчиков зачастую уникальны для каждой модели БТС. Эти характеристики должны быть учтены в выбираемой модели окружающего пространства для обеспечения своевременного учёта показаний всех доступных регистрируемых данных и, как следствие, достижения качественного и безопасного решения задач.

Задача построения модели окружающего пространства в робототехнике широко исследована, однако остаётся решённой не в полной степени и сегодня. Исследование этой задачи нашло отражение в работах ... Одна из таких моделей, нашедшей широкое применение в алгоритмах для колёсных роботов, – карта проходимости пространства, представляющая окружающее БТС пространство в виде сетки, каждой клетке, которой сопоставляется величина, характеризующая вероятность нахождения препятствия в соответствующей области. Данная модель, вместе с последующими модификациями была исследована в работах Э. Альберто, С. Труна, Л.М.Г. Гонсалвес, Л. Хеннинга. Для построения более полной модели, как правило, используется не один датчик, а их набор. Использование нескольких датчиков позволяет, во-первых, уменьшить слепую зону БТС, а, во-вторых, при использовании датчиков разного типа, компенсировать ограничения каждого типа сенсора по-отдельности. Поэтому, важным аспектом при разработке моделей окружающего пространства, является способ комплексирования разнородных данных регистрируемых датчиками для построения полного и непротиворечивого описания окружения. Среди исследований задачи комплексирования данных стоит выделить работы

Б. Халеки, Б.М. Миллера, П.П. Николаева, Ю.В. Визильтера. Однако вопрос применимости алгоритмов комплексирования при увеличении числа и/или типов комплексируемых датчиков не был в достаточной степени исследован к настоящему времени.

Целью данной работы является повышение надежности автономного управления за счёт разработки методов построения моделей окружающего пространства, которые работают на бортовом вычислителе БТС и используют алгоритмы комплексирования данных о проходимости, динамике объектов в сцене, а также слепых зонах БТС.

Для достижения поставленной цели необходимо было решить следующие **задачи**:

1. Исследовать существующие методы построения моделей окружающего пространства ТС
2. Разработать уникальный алгоритм оценки положения объектов в окружающем пространстве БТС
3. Разработать уникальные алгоритмы комплексирования данных в модель окружающего пространства ТС
4. Разработать комплекс технических средств, обеспечивающий построение модели окружающего пространства ТС, учитывающей наличие динамических и статических препятствий в сцене

Научная новизна:

1. Предложены новые алгоритмы проекции визуальных детекций и сегментации проезжей части на изображении на плоскость движения ТС на основе комплексирования данных с изображения и лидарного облака
2. Предложен новый метод построения карты проходимости пространства, позволяющий снизить время задержки между моментом регистрации данных сенсором и моментом обновления карты

Практическая значимость. Разработанные алгоритмы и комплекс технических средств, обеспечивающие построение модели окружающего пространства ТС, реализованные в данной работе, внедрены в автономные беспилотные транспортные средства “EVO.1” и “EVO.3”, разработанные компанией ООО “Эвокарго”. Внедрение разработанных в данной диссертации методов подтверждено соответствующими актами о внедрении. На результаты, полученные в работе были получены свидетельства о регистрации программ для ЭВМ:

1. №2025617212 Программное обеспечение «Система восприятия N1 V3».

2. №2025669744 Программное обеспечение «Способ построения карты проходимости в окрестности высокоавтоматизированного беспилотного грузового транспортного средства».

Методология и методы исследования. В диссертации используются методы вероятностной робототехники, методы байесовской оценки, вычислительной оптимизации, а также численный и натурный эксперимент.

Основные положения, выносимые на защиту:

1. Предложенный алгоритм оценки положения объектов в окружающем пространстве БТС по изображению с монокулярной камеры позволяет на 2.7% увеличить коэффициент Жаккара по сравнению с ближайшим аналогом.
2. Предложенный метод построения карты проходимости пространства, позволяющая выполнять неблокирующие операции сдвига, получения и обновления, позволяет в 13.07 раз уменьшить время обновления карты несколькими сенсорами за один цикл получения данных в одномерном случае
3. Предложенный алгоритм проекции визуальных детекций на изображении на плоскость движения ТС на основе комплексирования данных с изображения и облака точек лазерного дальномера позволяет на ?% увеличить коэффициент Жаккара по сравнению с ближайшим аналогом.
4. Интеграция предложенного алгоритма проекции сегментации проезжей части на изображении на плоскость движения ТС на основе комплексирования данных с изображения и лидарного облака в метод построения карты проходимости пространства позволяет уменьшить PathFinding Cost MSE (PFC-MSE) генерируемой карты проходимости на ?%

Достоверность полученных результатов обеспечивается использованием строгого математического аппарата и методов математической статистики. Помимо этого, достоверность подтверждается результатами вычислительных и натурных экспериментов и внедрением разработанных методов. Полученные результаты находятся в соответствии с результатами, полученными другими исследователями.

Апробация работы. Основные результаты работы докладывались на международной конференции 16-th International Conference on Machine Vision(ICMV 2023, Ереван, Армения)

Личный вклад. Все результаты диссертации, вынесенные на защиту, получены автором самостоятельно. Постановка задач и обсуждение результатов проводилось совместно с научным руководителем.

Публикации. Основные результаты по теме диссертации изложены в 2 печатных изданиях, 0 из которых изданы в журналах, рекомендованных ВАК. Зарегистрированы 1 патент и 1 программа для ЭВМ.

Объем и структура работы. Диссертация состоит из введения, 4 глав, заключения и 0 приложений. Полный объем диссертации составляет 117 страниц, включая 41 рисунок и 3 таблицы. Список литературы содержит 56 наименований.

Глава 1. Анализ методов построения моделей окружающего пространства БТС на основе комплексирования данных разной природы

1.1 Современное состояние беспилотных транспортных средств

Развитие беспилотных транспортных средств является одним из наиболее активных направлений интеллектуальных роботизированных систем. Разрабатываемые решения в этой области значительно отличаются по функционалу, а также условиям, в которых они применяются. Для классификации автономного транспорта Обществом автомобильных инженеров (англ. Society of Automotive Engineers, SAE) в 2014 году были предложены шесть уровней автоматизации (SAE J3016) [1].

1. Уровень 0. Отсутствие автоматизации (No Automation)

На данном уровне водитель полностью контролирует движение транспортного средства. Рулевое управление, ускорение/торможение и выполнение маневров осуществляется исключительно водителем. Автоматические системы, доступные на транспортных средствах данного уровня, осуществляют только информационные функции либо краткосрочно берут на себя вспомогательные функции управления транспортным средством. Примерами систем такого уровня являются системы предупреждения о сходе с полосы, СПСП (Lane Departure Warning, LDW), системы обнаружения объектов в слепых зонах транспортного средства (Blind Spot Monitoring, BSM), системы автоматического экстренного торможения, САЭТ (Autonomous Emergency Braking, AEB) и др.

2. Уровень 1. Вспомогательные системы (Driver Assistance)

Автоматические системы автомобиля на данном уровне могут выполнять одну из основных функций управления транспортным средством в определённых условиях, например ускорение/торможение или рулевое управление. При этом ответственность за безопасность остаётся на водителе, который обязан контролировать окружающую ситуацию и быть готовым взять управление транспортным средством на себя. Типичные

примеры систем этого уровня – адаптивный круиз-контроль (Addaptive Cruise Control, ACC) – система, позволяющая поддерживать фиксированные скорость и дистанцию позади впередиидущего транспортного средства, а также система удержания полосы движения, СУПД (Lane Keeping System, LKS).

3. **Уровень 2. Частичная автоматизация (Partial Automation)**

На данном уровне системы транспортного средства способны одновременно выполнять и рулевое управление, и торможение/ускорение. Однако, как и на предшествующих уровнях, водитель несёт полную ответственность за безопасность движения транспортного средства. К системам данного уровня относят системы удержания полосы, которые от аналогичных систем на уровне 1 контролируют не только рулевое управление, но и способны корректировать скорость транспортного средства. Также к этому уровню принадлежат системы автоматической парковки (Automatic Parking).

4. **Уровень 3. Условная автоматизация (Conditional Automation)**

Автоматические системы автомобиля на этом уровне способны полностью осуществлять управление транспортным средством без участия водителя в ограниченных условиях: как правило это определённый тип дороги (автомагистрали с отчётливо распознаваемой разметкой) и хорошие погодные условия. В случае возникновения нестандартной ситуации на дороге система должна своевременно передать управление транспортным средством водителю. Соответственно, в отличие от предшествующих уровней от водителя не требуется постоянное слежение за окружающей обстановкой, но он должен быть готов к управлению транспортным средством в нестандартных ситуациях. Пример такой системы – ассистент движения в пробках (Traffic Jam Assist).

5. **Уровень 4. Высокая автоматизация (High Automation)**

Аналогично предыдущему уровню, системы автомобиля уровня 4 способны осуществлять полное управление транспортным средством в чётко определённых сценариях. От систем 3-го уровня их отличает способность самостоятельного завершения поездки или возвращение в безопасное состояние, в случае возникновения внештатной ситуации и отсутствия реакции от водителя. Как следствие, на автомобилях данного уровня не накладывается требование к обязательному наличию

водителя. К системам этого уровня относят беспилотное такси и автоматизированные грузовые транспортные средства.

6. **Уровень 5. Полная автоматизация (Full Automation)**

На данном уровне автономности системы автомобиля способны осуществлять полное управление транспортным средством без ограничения на условия эксплуатации. Автомобили данного уровня не требуют наличия механизмов ручного управления, таких как педали торможения/ускорения, рулевое колесо и пр. На сегодняшний день не существует транспортных средств обладающих 5-м уровнем автоматизации, несмотря на количество работ, направленных на их создание.

В настоящее время идёт бурное развитие технологий, направленных, на усовершенствование беспилотных транспортных средств, обладающих 4-м уровнем автоматизации, а также создание транспортных средств с 5-м уровнем автоматизации. Среди проектов по разработке таких транспортных средств можно выделить Waymo, Tesla, Яндекс, Baidu Apollo и др. Несмотря на то, что данные проекты показывают высокую степень автономности, перед отраслью стоит множество задач, требующих решения для повышения уровня безопасности движения беспилотных транспортных средств, что будет способствовать внедрению данных технологий в повседневное использование. Одна из таких задач – разработка надёжных систем восприятия транспортного средства, отвечающих за построение точной и полной модели окружающего пространства. В рамках данной диссертационной работы исследуются и предлагаются алгоритмы построения таких моделей.

1.2 **Модели окружающего пространства БТС**

В робототехнике под моделью окружающего пространства понимается приближённое описание окружающих БТС объектов: их наличие и положение в некоторой окрестности транспортного средства. Помимо информации о положении объектов, такая модель может описывать их скорость и предполагаемую траекторию движения. Также модель может описывать форму и расположение проезжей части, слепых зонах транспортного средства (как вызванных окклюзией, так и обусловленных отсутствием показаний датчиков для

соответствующей области пространства), а также объектов транспортной инфраструктуры: полосы разметки, дорожные знаки, бордюры, светофоры и т.д.

В рамках данной работы исследуются двумерные модели – модели проецирующие объекты, окружающие БТС, на плоскость его движения. Такие модели не хранят информацию о высоте объекта и его положении над землёй, т.к., как правило, эти данные избыточны для решения задачи планирования движения колёсного транспорта. В то же время, следует отметить, что среди предложенных на сегодняшний день моделей существуют и трёхмерные. Так в работе [2] окружающая сцена представлена совокупностью полигонов, а в работе [3] в отдельный класс выделены модели, описывающие окружающее пространство как облако точек.

Среди существующих двумерных моделей можно выделить три категории:

1. **Объектно-ориентированные представления.** В них окружающая сцена описывается как множество объектов. Как правило, объекта представлен в виде ограничивающего параллелепипеда: положением его центра, ориентацией и габаритами. Дополнительно описание объекта может содержать тип (транспортное средство, пешеход, животное и т.д.), а также линейные и угловые скорость и ускорение, предсказываемую траекторию перемещения. Такие модели имеют достаточно краткую форму записи: множество векторов, описывающих отдельных объект, что делает их вычислительно эффективными. В то же время они ограничены в возможности описания объектов сложной формы: тягачи с полуприцепами, перевёрнутые машины, объекты нетипичной формы.
2. **Сеточные представления.** Модели этой категории призваны обойти ограничение предыдущей категории и описывать объекты с произвольной геометрией. В них пространство вокруг транспортного средства разбивается на сетку фиксированного разрешения, каждая клетка которой хранит информацию о наличии (или вероятности наличия) препятствия в соответствующей области. Такое представление получило название карты проходимости пространства (Occupancy Grid Map, OGM). К этой же категории относятся динамический карты проходимости пространства (Dynamic Occupancy Grid Map, DOGMa), в них для клеток, в которых обнаружено препятствия, добавляется информация

о его скорости и направлении движения. Модели этой категории также чаще, чем объектно-ориентированные модели, учитывают не классифицируемые объекты: мусор, открытые люки, бордюры и т.п., что делает их более надёжным представлением окружающей сцены. Однако, более детальное описание сцены приводит к большему размеру структуры данных в памяти компьютера, что негативно влияет на вычислительную сложность алгоритмов, работающих с моделями в этой категории. Ограничивающая разрешающая способность данных моделей также может рассматриваться как недостаток для некоторых задач автономной навигации.

3. **Непрерывные представления.** Как и модели прошлой категории модели непрерывного представления стремятся описывать объекты со сложной геометрией. Однако в отличие от сеточных представлений модели этой категории представляют пространство, как непрерывное отображение $\mathbb{R}^2 \rightarrow O$, где каждой точке плоскости движения транспортного средства сопоставляется некоторая характеристика: бинарная классификация принадлежности точки проезжей части дороги ($O = \{0,1\}$), класс объекта ($O = \{0, \dots, N\}$), плотность вероятности нахождения препятствия в данной точке ($O = \mathbb{R}^+$) и т.п.. Таким образом, модели данной категории имеют бесконечную разрешающую способность, что позволяет обойти главное ограничение сеточных представлений. Недостаток же моделей данной категории заключается в том, что алгоритмы для их построения и обновления сложнее в реализации, а вычислительная сложность может быть хуже чем у предыдущих двух категорий (для сеточных представлений последнее зависит от требований к размеру и разрешению сетки). Примерами таких представлений служат полигональные раскраски (Polygonal Random Field, PRF [4]), мультимодальное нормальное распределение [5].

В рамках данной диссертационной работы будут предложены алгоритмы для построения и обновления карты проходимости пространства из категории сеточных представлений. Данный выбор основан на широком распространении модели, вычислительной простоте и, как будет показано в работе, масштабируемости относительно числа датчиков, участвующих в обновлении модели. Опишем подробнее эту модель и методы её построения.

Представление окружающего пространства в виде карты проходимости пространства была впервые предложена в работах Альберто Элфеса и Ханса Моравека [6; 7]. В рамках данного представления окружающее пространство разбивается на двумерную сетку M , каждой клетке которой m_i сопоставляется вероятность нахождения препятствия в пространстве $p(m_i)$, ограниченной этой клеткой. В работе [8] те же авторы расширили модель карты проходимости пространства, включив в неё обратную модель наблюдения – вероятностную модель, моделирующая процесс измерений датчика и учитывающая принцип работы, а также особенности среды, в которой производятся наблюдения. Расширение модели позволило сформулировать алгоритм уточнения карты проходимости пространства на основании показаний датчика. В рамках данной модели дополнительно делается предположение, что каждая вероятность занятости клетки в карте проходимости не зависит от вероятностей занятости других клеток карты. Для каждой клетки вычисляется величина так называемого логарифма шансов:

$$l_{t,i} = \ln \frac{p(m_i|z_{1:t}, x_{1:t})}{1 - p(m_i|z_{1:t}, x_{1:t})}, \quad (1.1)$$

где $p(m_i|z_t, x_t)$ – вероятность занятости i -й клетки карты, при условии что к моменту времени t робот прошёл траекторию $x_{1:t}$, и его датчики давали показания $z_{1:t}$. Заметим, что здесь используется обратная причинно-следственная зависимость: измеряется вероятность причины (занятость пространства), при условии наступления его следствия (показания датчика), чем и обусловлено название модели. С получением новых данных значения в тех клетках, о которых появились новые данные (иными словами, которые попали в область видимости датчика), обновляются по следующей формуле:

$$l_{t,i} = l_{t-1,i} + \ln \frac{p(m_i|z_t, x_t)}{1 - p(m_i|z_t, x_t)} - l_{0,i}, \quad (1.2)$$

где $l_{0,i} = \ln \frac{p(m_i)}{1 - p(m_i)}$ логарифм шансов, посчитанный для априорной вероятности занятости клетки. Зачастую, о карте проходимости не делается априорных предположений и $p(m_i)$ устанавливаются равным 0.5, что приводит к $l_{0,i} = 0$.

Такой формат обновления удобен, так как позволяет независимо друг от друга посчитать значение $\ln \frac{p(m_i|z_t, x_t)}{1 - p(m_i|z_t, x_t)} - l_{0,i}$, на основании полученных дан-

ных от каждого сенсора, а затем просто добавить полученное значение к уже имеющейся карте проходимости (при условии атомарности данной операции).

1.3 Обзор используемых в колёсной робототехнике датчиков и их ограничения

Обновление любой модели окружающей сцены производится на основании показаний датчиков, установленных на БТС. Основные датчики, применяемые в колёсной робототехнике для построения модели, – лидары, радары, сонары и камеры [9].

1. **Радары (Radio Detection and Ranging, RADAR).** Технология основанная на использовании создании радиоволн и регистрации их отражений от объектов. По времени регистрации и степени затухания отраженной волны, датчик определяет расстояние и скорость до объекта. Радары разделяют на радары дальнего диапазона (РДД) и короткого диапазона (РКД) [10]. Первые обладают наибольшей дальностью видимости (250 м) среди упомянутых датчиков и используются для обнаружения препятствий на пути движения машины. Вторые обладают меньшей дальностью, но лучшим разрешением и могут быть использованы для реализации систем парковки и перестроения между полосами движения. В то же время радары обладают низким разрешением, требовательностью к погодным условиям и чувствительностью к металлическим препятствиям в поле видимости.
2. **Лидары (Light Detection and Ranging, LiDAR).** Технология, начавшая развитие в второй половине двадцатого века, в 1970х годах, в космической и авиастроительных сферах. Принцип работы схож с радаром, но вместо радиоволн использует пучки инфракрасного спектра. Существует два типа лидаров: механические и твердотельные. В механическом лидаре источники и приемники сигнала вращаются на высокой (от 600 оборотов в минуту) скорости. В твердотельном лидаре источник и приемник остаются неподвижны, а изменение направления сканирующего луча производится путём вращения призмы или зеркала, через которые он проходит. Помимо измерения расстояния

до объекта лидары способны регистрировать его светоотражательную способность, что может быть применено, например, для детекции разметки. Лидары обладают высокой точностью и лучшим разрешением, чем радары и хорошей дальностью обзора. Однако, как и радары, лидары требовательны к погодным условиям: качество измерений снижается в условиях тумана и/или осадков. Также ограничивающим фактором для использования лидаров является их дороговизна, из-за этого многие компании, занимающиеся разработкой беспилотных автомобилей вынуждены отказываться от лидаров в пользу альтернативных способов детекции местонахождения машины и построения маршрута движения. Лидары могут использоваться для распознавания и классификации объектов, создании трехмерных карт.

3. **Камеры.** Наиболее распространенные датчики, используемые в колёсной робототехнике. В отличие от остальных описанных датчиков, камеры – пассивный датчик, т.к. не излучают никаких сигналов, в отличие от активных датчиков, отдельной проблемой которых является борьба с интерференцией датчиков друг с другом. Это позволяет без опасений использовать большое количество камер без необходимости синхронизировать их работу друг с другом. В основном в БТС используют камеры видимого (400-780 нм) спектра, но также встречаются модели использующие камеры инфракрасного спектра. Камеры обладают наилучшим разрешением среди описанных датчиков, но не способны измерять расстояние до объектов. Использование нескольких камер в связке позволяет использовать алгоритмы стереозрения для измерения расстояния, однако точность и дальность этих измерений ниже и уступает лидарам и радарам. Камеры используются для захвата изображений, которые затем обрабатываются с помощью алгоритмов компьютерного зрения. Камеры в компьютерном зрении широко используются в различных областях, таких как робототехника, автоматическое управление производственными процессами, медицинская диагностика, безопасность, распознавание лиц, автоматическое управление транспортом, анализ изображений и много других.
4. **Сонары.** Как лидары и радары, принцип работы сонаров основан на измерении времени регистрации отражения волн от объектов, однако в сонарах используются ультразвуковые (40-70 кГц) волны. Данные

волны не слышны и не могут нанести вред слуху человека, что немаловажно, поскольку громкость создаваемого импульса достигает 100 дБ. Сонары обладают наиболее малой областью обзора (менее десятка метров), и потому как правило используются для систем парковки. Сонары используются для измерения расстояния до объектов и создания трёхмерной модели окружающей среды. Они работают на основе эхолокации, отправляя звуковые импульсы и затем регистрируя отражённые от объектов сигналы. Полученные данные могут быть обработаны компьютером для определения расстояния и формы объектов. Одним из наиболее распространённых применений сонаров является их использование при парковке БТС. Также сонары могут использоваться в системах обнаружения движущихся объектов, таких как пешеходов и других транспортных средств.

Из описания датчиков следует, что каждый тип датчика обладает рядом преимуществ и недостатков. Использование нескольких датчиков разных типов на одном транспортном средстве позволяет компенсировать недостатки отдельных датчиков и добиться повышения качества восприятия, тем самым повышая безопасность движения и открывая новые сценарии применения беспилотных транспортных средств.

Характеристика	Лидар	Радар	Камера	Сонары
Разрешение	Среднее	Низкое	Высокое	Низкое
Дальность области обзора	150 м.	250 м.	200 м.	5 м.
Устойчивость к погодным условиям	Средняя	Высокая	Низкая	Высокая
Чувствительность к условиям освещения	Нет	Нет	Высокая	Нет
Информация о дистанции	Да	Да	Нет	Да
Вычислительная нагрузка обработки	Средняя	Низкая	Высокая	Низкая
Стоимость	Высокая	Средняя	Низкая	Низкая

Таблица 1 — Сравнение характеристика датчиков, используемых в колёсной робототехники

1.4 Методы комплексирования данных

На сегодняшний день предложено множество алгоритмов построения карты проходимости пространства. Их реализация сравнительно проста в случае, если информация об окружающем пространстве поступает от единственного датчика. Примеры таких систем представлены в статьях [11] и [12]. Однако для большинства практических задач единственного датчика недостаточно, из-за ограниченного поля зрения, невозможностью полностью компенсировать зашумлённость входных данных, и общими ограничениями, обусловленными принципом его работы.

Чтобы получить более точную модель окружения и повысить безопасность движения БТС, на них устанавливаются несколько датчиков. Примеры систем с несколькими датчиками описаны в работах [13], [14]. Встречаются как системы, использующие несколько датчиков одного типа (например, камеры, как в работе [15]), так и комбинирующие различные датчики, например:

- лидары+радары. Пример такого робота представлен в [16];
- лидары+камеры [17]),
- и другие комбинации.

Алгоритмы построения модели окружающего БТС пространства для таких систем в общем случае генерируют модель достаточно полную и точную, но вынуждены решать задачу комплексирования данных для разрешения противоречий показаний с разных датчиков. Существуют разные алгоритмы, осуществляющие комплексированию данных, которые можно разделить на три группы [18]:

1. Раннее комплексирование необработанные данные с датчиков объединяются в один пакет данных и передаются системе для последующего анализа. Примером такой агрегации является построение карты глубины в стереокамере, как например в работе [19].
2. Промежуточное комплексирование – по сырым данным с датчиков вычисляются отдельные признаки (в общем случае отличные для каждого из датчиков), которые затем объединяются для дальнейшего решения задачи. Так в работах [20; 21] данные с разных датчиков обрабатываются разными нейросетевыми моделями, после чего выходные тензоры конкатенируются.

3. Позднее комплексирование – для каждого датчика целевая задача (в нашем случае построение модели окружающего БТС пространства) решается отдельно, выдвигая независимую гипотезу. На основе этих гипотез более высокоуровневый алгоритм вычисляет итоговое решение.

Для существующих алгоритмов комплексирования данных в контексте задачи построения карты проходимости пространства в работе [22] представлена классификация по трем группам:

1. Методы, использующие вероятностные модели. К ним относятся Байесовский вывод, теория Демпстера — Шафера, робастные методы и т.д.
2. Методы, использующие метод наименьших квадратов (МНК): фильтр Калмана, методы оптимизации, эллипсы неопределённости (англ. *uncertainty ellipsoids*) и пр.
3. Методы интеллектуального комплексирования: методы нечёткой логики, нейросетевые и генетические алгоритмы.

В настоящее время продолжается активная разработка алгоритмов каждого из трех типов. Примером актуальных исследований в первой группе – работа [23], в которой используется метод релевантных векторов (метод машинного обучения, использующий Байесовский вывод) для построения непрерывной карты проходимости пространства. Во второй группе можно выделить работу [24], в которой положение динамических препятствий на карте обновляется с помощью фильтра Калмана. Стоит отметить, что в данной работе обновление информации о статических препятствиях происходит с помощью Байесовского вывода, поэтому предлагаемый алгоритм является объединением методов из двух представленных групп. Наконец, не менее активно развиваются алгоритмы третьей группы, одним из которых является модель глубокого обучения BEVFusion, представленная в работе [25]

1.5 Метрики оценки качества моделей окружающего пространства

Одной из ключевых задач при построении моделей окружающей сцены БТС является оценка качества таких моделей. Учитывая сложность сенсорного восприятия в условиях реального мира, построенные модели нередко подвержены искажению информации об окружающей сцене из-за неоднородности,

шумов, неполноты и несогласованности данных, в частности при использовании разных типов датчиков, детектирующие данные разных модальностей. Поэтому разработка и выбор адекватных метрик, способных количественно охарактеризовать точность, полноту, согласованность и устойчивость моделей окружающей среды, становится неотъемлемым этапом при анализе и сравнении алгоритмов картографирования.

В данном разделе рассматриваются существующие подходы к оценке качества моделей окружающего пространства. Обсуждаются метрики, разработанные для моделей из классификации, представленной в разделе 1.2.

Среди метрик для объектно-ориентированных представлений широкое применение получил коэффициент Жаккара [26]. Пусть p_a – полигон, полученный в результате работы алгоритма построения модели по полученным с датчиков данным, описывающий форму и положение одного из окружающих БТС объектов. Пусть также p_r – полигон, описывающий эталонные форму и положение того же объекта. Тогда коэффициент Жаккара, равен:

$$J = \frac{S(p_a \cap p_r)}{S(p_a \cup p_r)},$$

где $S(p_a \cap p_r)$ – площадь полигона, полученного при пересечении p_a и p_r , $S(p_a \cup p_r)$ – площадь полигона, полученного при объединении p_a и p_r . Таким образом, значение J лежит в диапазоне $[0,1]$. Чем больше значение J , тем более p_a совпадает с p_r и тем качественнее модель. Отметим также, что зачастую в качестве сравниваемых полигонов выступают произвольно ориентированные ограничивающие прямоугольники. Для итоговой оценки построенной модели вычисляют усреднённое значение J :

$$mJ = \frac{1}{|C|} \sum_{c \in C} J_c,$$

где C – множество сопоставленных пар $= (p_a, p_r)$ вычисленных и эталонных полигонов, J_c – коэффициент Жаккара для пары полигонов c .

Помимо коэффициента Жаккара при оценке качества моделей можно встретить метрики, используемые при оценке решения задачи классификации. Например, F-score. Для этого вводится некоторый порог $j_{thr} \in (0,1)$. Корректно обнаруженными объектами (верными срабатываниями алгоритма – TP) считаются только те p_a , для которых $J > j_{thr}$, остальные p_a , считаются ложными срабатываниями алгоритма – FP , построившего модель. Наконец, эталонные p_r

для которых не нашлось p_a с достаточным коэффициентом Жаккара считаются ложными промахами алгоритма – FN . Итоговый F-score равен:

$$\text{F-score} = \frac{2}{P_c^{-1} + P_a^{-1}},$$

где $P_c = \frac{|TP|}{|TP|+|FN|}$ – полнота, $P_a = \frac{|TP|}{|TP|+|FP|}$ – точность.

Существует также метрика локализационной точности (localization accuracy), которая как и усреднённый коэффициент Жаккара оценивает качество вычисленных p_a , но только для верных срабатываний алгоритма:

$$\text{LocA} = \frac{1}{|TP|} \sum_{c \in TP} J_c$$

Для планирования движения БТС не редко важно не только точно определить положение объектов (задача детекции объектов) в сцене, но и корректно отследить их перемещение между сценами (задача отслеживания объектов). Для этого в модель к описанию объекта добавляется уникальный идентификатор, который в идеальном сценарии остаётся неизменным для одного и того же объекта в каждой сцене. Для оценки качества таких моделей предложено множество метрик. Одна из первых таких метрик – MOTA(Multi Object Tracking Accuracy) [27]. В рамках этой метрики вычисляется количество переключений идентификатора(Identity Switch, IDSW) Пусть i – номер сцены в последовательности сцен, упорядоченных по времени. $|\text{IDSW}|$ – число верных срабатываний, для которых алгоритм изменил идентификатор объекта, в то время как у эталонного объекта он остался неизменным:

$$|\text{IDSW}| = \sum_{i \in [1, N-1]} |(p_{a_{i+1}}, p_{r_{i+1}}) \in TP_{i+1}; id_p_{a_{i+1}} \neq id_p_{a_i}, id_p_{r_{i+1}} = id_p_{r_i}|,$$

где N – число анализируемых сцен.

Далее MOTA вычисляется по формуле:

$$\text{MOTA} = 1 - \frac{\sum_i |FN_i| + |FP_i| + |\text{IDSW}_i|}{\sum_i |TP_i| + |FN_i|}$$

Данная метрика хоть и зависит от качества решения задачи отслеживания остаётся больше сфокусированной на качестве детекции. Для решения этого ограничения в работе [28] была предложена метрика IDF1. В рамках этой

метрики выбирается такое сопоставление (id_p_a, id_p_r) реально полученных и эталонных идентификаторов объектов соответственно, для которых максимизируется величина:

$$IDF1 = \frac{2IDTP}{2IDTP + IDFP + IDFN},$$

где $IDTP$ – число пар (id_p_a, id_p_r) на всех сценах, совпавших с выбранным сопоставлением; $IDFP$ – число пар $(id_p_r, id_p'_a)$ с отличным от выбранного сопоставления эталонного идентификатора; $IDFN$ – число пар $(id_p'_r, id_p_a)$ с отличным от выбранного сопоставления вычисленного идентификатора. На рисунке 1.1 представлен пример последовательностей вычисленных и эталонных идентификаторов для подсчёта $IDF1$.

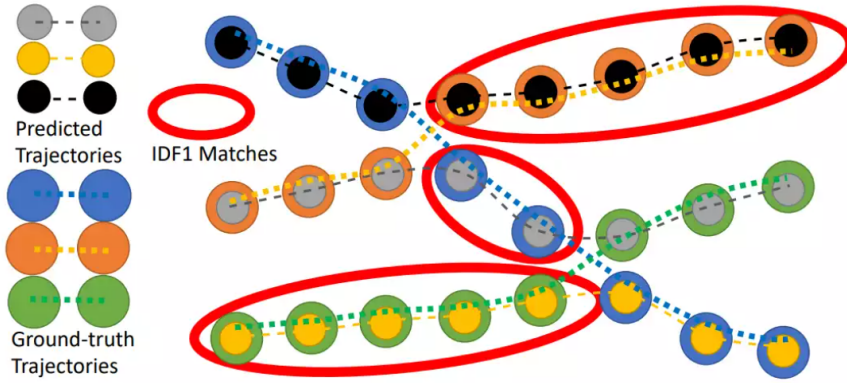


Рисунок 1.1 — Пример сопоставления вычисленных и эталонных идентификаторов объектов для $IDF1$ [27]

Чтобы получить более сбалансированную оценку, учитывающую как качество детекции объектов так и их отслеживание, была предложена метрика НОТА(Higher Order Tracking Accuracy) [29]. Для этого вводятся две вспомогательные метрики DetA (Detection Accuracy) и AssA (Association Accuracy), нацеленные на оценку качества решения соответствующих задач. Далее берётся среднее геометрическое полученных величин. Обе эти метрики зависят от выбираемого порога для коэффициента Жаккара. Чтобы не добавлять его в качестве гиперпараметра метрики, авторы предлагают интегрировать полученную величину по всем значениям порога:

$$NOTA(j_{thr}) = \sqrt{DetA_{j_{thr}} AssA_{j_{thr}}}$$

$$NOTA = \int_0^1 NOTA(j_{thr}) dj_{thr}$$

DetA равен доле верных срабатываний алгоритма для выбранного порога коэффициента Жаккара:

$$\text{DetA} = \frac{|TP|}{|TP| + |FN| + |FP|}$$

Для вычисления AssA для каждого верного сопоставления из TP вычисляется и усредняется доля верных срабатываний с такими же идентификатороами на всех кадрах:

$$\text{AssA} = \frac{1}{|TP|} \sum_{c \in TP} \frac{|TPA(c)|}{|TPA(c)| + |FPA(c)| + |FNA(c)|},$$

где $|TPA(c)|$ число TP с теми же идентификаторами, что и $c = (id_p_a, id_p_r)$; $|FPA(c)|$ – число вычисленных детекций с идентификатором id_p_a без сопоставленных эталонных объектов либо сопоставленных с объектом с идентификатором отличным от id_p_r ; $|FNA(c)|$ – число эталонных детекций с идентификатором id_p_r без сопоставленных вычисленных объектов либо сопоставленных с объектом с идентификатором отличным от id_p_a .

1.6 Заключение к главе 1

Глава 2. Алгоритмы проецирования визуальных данных на плоскость движения БТС

2.1 Алгоритм проекции визуальных детекций на плоскость движения БТС на основе модели L-Shape

В данном разделе описан метод определения положения транспортных средств (наиболее распространенного типа объектов окружения в городских условиях) в виде произвольно ориентированных ограничивающих рамок на виде сверху (birds'-eye view) по изображению, полученному с одной монокулярной камеры. Этот метод состоит из двух этапов. На первом этапе вычисляется проекция видимой границы транспортного средства в виде сверху на основе 2D-детекций препятствий и сегментации проезжей части на изображении. Предполагается, что полученная проекция представляет зашумленные измерения двух ортогональных сторон ТС. На втором этапе строится ориентированная ограничивающая рамка вокруг полученной проекции. Для этого этапа будет предложен новый алгоритм построения рамки на основе предположения об L-образности проекции: L-Shape алгоритм.

2.1.1 Этап 1. Проекция 2D-детекции изображения на вид сверху

Пусть имеется изображение, зарегистрированное камерой БТС, с известной внутренней и внешней калибровкой. На этом изображении в 2D производится детектирование ТС и семантическая сегментация проезжей части. Используя полученные данные, мы хотим достроить положение, ориентацию и габариты обнаруженных ТС на виде сверху. Далее описаны два подхода к определению границ окружающих ТС на плоскости движения БТС.

Наивный подход

Пусть дан набор ограничивающих рамок на зарегистрированном изображении, полученный с помощью нейросетевой модели YOLOP [30], осуществля-

ющей детекцию ТС на изображении. Для дальнейшего определения положения обнаруженных ТС вводятся два предположения:

1. Все объекты находятся на земле (проезжей части дороги).
2. Поверхность дороги плоская. Далее в работе используется система координат, в которой уравнение плоскости, в которой лежит данная поверхность, имеет виде $z = 0$.

Для оценки 3D-координат ТС, мы проецируем два нижних угла ограничивающей рамки на плоскость дороги (рис. 2.1). Мы считаем известными параметры внутренней калибровки и внешней калибровок, поэтому данная операция происходит в два шага: сперва мы находим однородные координаты p_u интересующих нас точек в системе локальной системе координат камеры, а затем, используя внешнюю калибровку, проецируем полученные точки на плоскость движения БТС: $z_l = 0$ в системе O_l , получая окончательный результат p_l . Алгоритмы планирования пути зачастую работают с представлением объектов в виде геометрических фигур с ненулевой площадью или объёмом. Для таких алгоритмов мы производим достраивание полученного отрезка до прямоугольника, добавлением к нему ортогональных отрезков фиксированной длины.

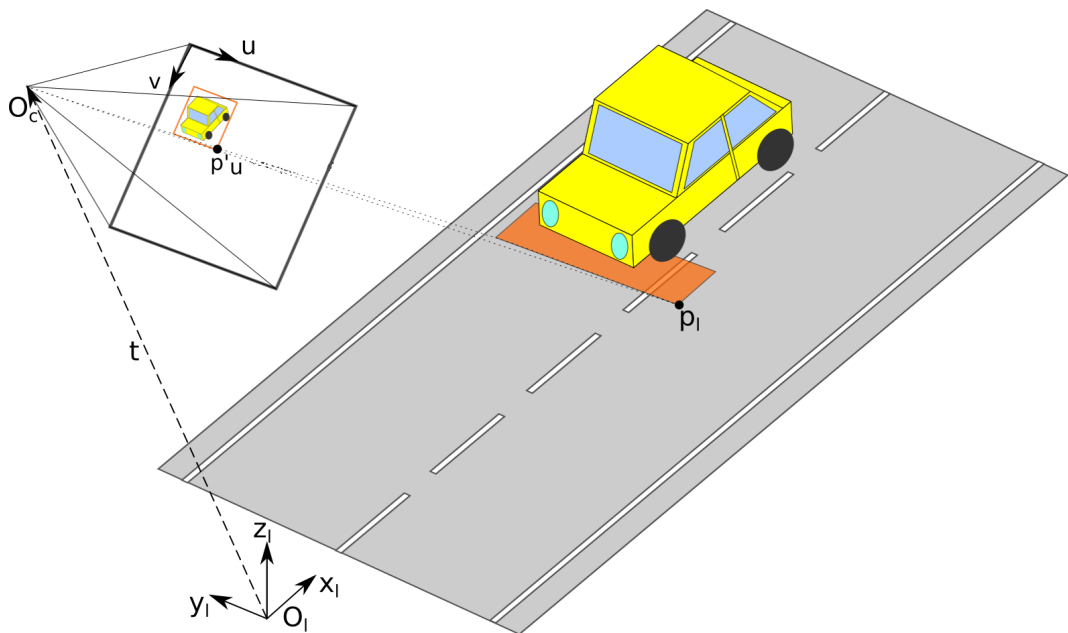


Рисунок 2.1 — Наивный подход к проецированию ограничивающей рамки в 2D на вид сверху.

Данный алгоритм достаточно хорошо справляется с проецированием небольших объектов, например, людей, а также объектов, которые находятся в центре кадра и ориентированы вдоль оптической оси камеры. Однако, если объект большой и его границы не параллельны осям изображения, алгоритм

проецирует его неверно, что может вызвать в алгоритмах планирования движения, использующих выходные данные из нашего алгоритма решение произвести остановку там, где она не требуется, т.к. возможен объезд. Пример такой ситуации показан на рис. 2.2.

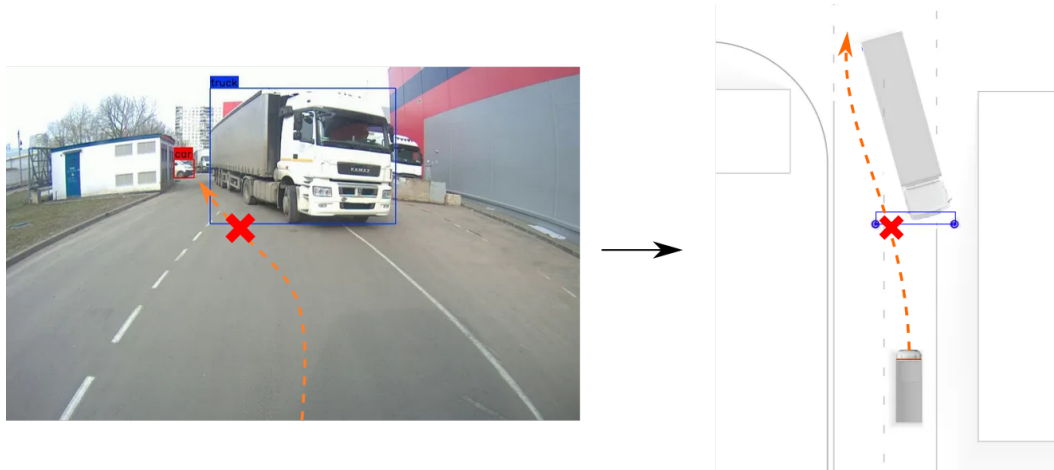


Рисунок 2.2 — Неверная проекция грузовика. Зона слева от грузовика воспринимается, как часть объекта, и мешает объехать его.

Уточнение положения объекта с помощью семантической сегментации проезжей части

Для решения продемонстрированной проблемы, мы дополнительно уточняем положение детектируемых объектов с учетом сегментации проезжей части (также полученной с помощью нейросетевой модели YOLOP). В результате наш алгоритм выглядит следующим образом:

1. Берем N равномерно распределенных точек на нижней границе ограничивающей рамки.
2. Каждая полученная точка сдвигается к верху изображения пиксель за пикселем, пока она не выйдет за пределы проезжей части либо не достигнет верхнего края ограничивающей рамки.
3. Выпуклая оболочка полученных точек проецируется на дорогу аналогично наивному подходу.

На рис. 2.3 приведена демонстрация принципа работы алгоритма.

Несмотря на недостатки этого метода, включая ошибки, вызванные изменяющимся зазором между ТС и дорогой, он может быть применен как способ построения одного из кандидатов для более сложных детекторов, учитывающих результаты работы нескольких алгоритмов, как это сделано в работе [31]

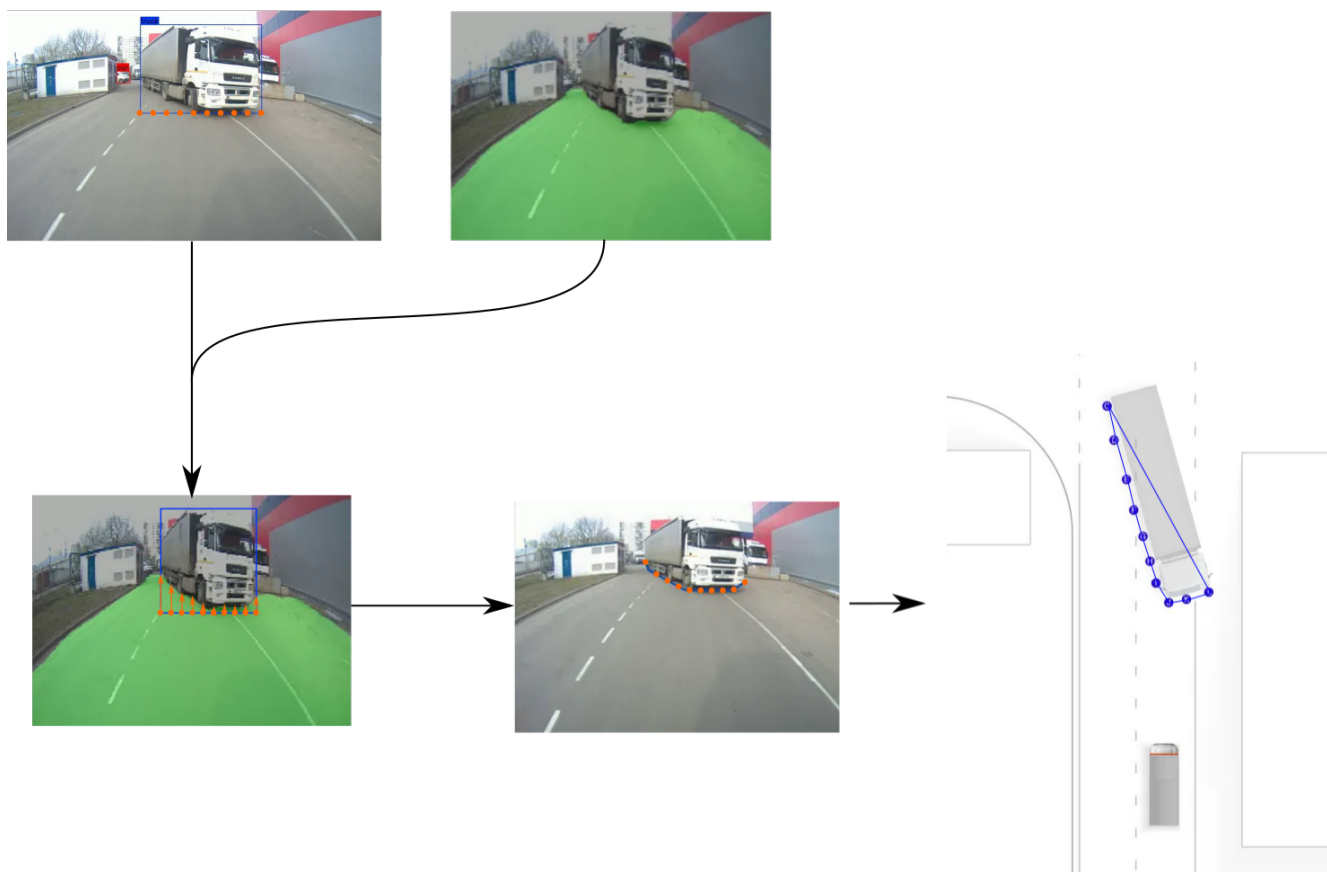


Рисунок 2.3 — Проекция двумерной ограничивающей рамки на вид сверху с учетом сегментации проезжей части.

2.1.2 Этап 2. Вписывание проекции объекта на виде сверху в ориентированную ограничивающую рамку.

На следующем этапе алгоритма полученная на прошлом шаге проекция вписывается в ориентированную рамку на плоскости дороги. Такое представление упрощает работу последующим алгоритмам, работающим с полученными трехмерными детекциями, в частности некоторым алгоритмам планирования пути и многим алгоритмам отслеживания объектов. Помимо этого, из-за окклюзии мы не можем видеть некоторые части детектируемого ТС из-за чего получаемая проекция покрывает только часть реально занимаемого им пространства.

Для построения ограничивающей рамки мы делаем дополнительные предположения об L-образности вписываемых точек:

1. Вписываемые в рамку точки – зашумлённые точки, принадлежащие двум смежным и ортогональным сторонам ТС

2. Точки упорядочены таким образом, что сперва идут точки принадлежащие одной стороне ТС, затем точки второй стороны.

Далее в работе данную модель мы будем именовать L-Shape моделью. Используя данную модель, нахождение ограничивающей рамки сводится к поиску двух перпендикулярных прямых: $l : y = kx + d_1$ и $l' : x = -ky + d_2$, таких что минимизируется среднеквадратичное отклонение (mean squared error, MSE) вписываемых точек до соответствующей прямой. Согласно выбранной модели:

$$\begin{aligned} \exists m \in \{1, \dots, N-1\} \hookrightarrow \forall i \in \{1, \dots, m\}, p_i = (x_i, y_i) \in l; \\ \forall i \in \{m+1, \dots, N\}, p_i = (x_i, y_i) \in l' \end{aligned}$$

При этом заранее значение m неизвестно. Задачу минимизации MSE можно записать в следующем виде:

$$\operatorname{argmin}_{k, d_1, d_2} \left(\begin{pmatrix} x_1 & 1 & 0 \\ x_2 & 1 & 0 \\ \vdots & \vdots & \vdots \\ x_m & 1 & 0 \\ -y_{m+1} & 0 & 1 \\ \vdots & \vdots & \vdots \\ -y_N & 0 & 1 \end{pmatrix} \begin{pmatrix} k \\ d_1 \\ d_2 \end{pmatrix} - \begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_m \\ x_{m+1} \\ \vdots \\ x_N \end{pmatrix} \right)^2$$

Введя дополнительные обозначения, мы можем переписать это выражение следующим образом:

$$\operatorname{argmin}_a \left(\begin{pmatrix} \hat{x} & 1 & 0 \\ -\hat{y} & 0 & 1 \end{pmatrix} a - \begin{pmatrix} \tilde{y} \\ \tilde{x} \end{pmatrix} \right)^2$$

И далее:

$$\operatorname{argmin}_a (Xa - y)^2$$

Решением данной задачи минимизации является:

$$a^* = (X^T X)^{-1} X^T y \quad (2.1)$$

$$\text{MSE} = a^{*T} X^T X a - 2a^{*T} X^T y - y^T y \quad (2.2)$$

Таким образом, неэффективная реализация модели L-Shape следующая:

1. Перебираем все значения $m \in \{1, \dots, N-1\}$
2. Пересчитываем матрицы X, y
3. Находим a^* , MSE для каждой задачи минимизации по формулам (2.1), (2.2)
4. В качестве финального решения для l, l' выбираем a^* с наименьшим MSE.
5. Обрамляем вписываемый набор точек линиями параллельными l и l' . Область ограниченная полученными линиями – искомая ограничивающая рамка.

Вычисление шагов 2 и 3 занимает $\Theta(N)$ операций. Эти шаги повторяются в цикле $N-1$ раз. Итоговая сложность такой реализации – $\Theta(N^2)$. Покажем, как ускорить данный алгоритм. Для этого вычислим некоторые промежуточные матрицы, используемые при вычислении a^* и MSE (для удобства чтения введём обозначение $\bar{m} = N - m$):

$$\begin{aligned}
 X^T X &= \begin{pmatrix} \sum \hat{x}^2 + \sum \hat{y}^2 & \sum \hat{x} & -\sum \hat{y} \\ \sum \hat{x} & m & 0 \\ -\sum \hat{y} & 0 & \bar{m} \end{pmatrix} \\
 \Delta_{X^T X} &= m\bar{m}(\sum \hat{x}^2 + \sum \hat{y}^2) - m\left(\sum \hat{y}\right)^2 - \bar{m}\left(\sum \hat{x}\right)^2 \\
 (X^T X)^{-1} &= \frac{1}{\Delta_{X^T X}} \\
 &\begin{pmatrix} m\bar{m} & -\bar{m}\sum \hat{x} & m\sum \hat{y} \\ -\bar{m}\sum \hat{x} & \bar{m}(\sum \hat{x}^2 + \sum \hat{y}^2) - (\sum \hat{y})^2 & -\sum \hat{x}\sum \hat{y} \\ m\sum \hat{y} & -\sum \hat{x}\sum \hat{y} & m(\sum \hat{x}^2 + \sum \hat{y}^2) - (\sum \hat{x})^2 \end{pmatrix} \\
 X^T y &= \begin{pmatrix} \hat{x}^T \tilde{y} - \hat{y}^T \tilde{x} \\ \sum \tilde{y} \\ \sum \tilde{x} \end{pmatrix}
 \end{aligned}$$

Можно заметить, что если вычислить значения $\sum \hat{x}^2 + \sum \hat{y}^2$, $\sum \hat{x}\tilde{y}$, $\sum \tilde{x}\hat{y}$, $\sum \hat{x}$, $\sum \hat{y}$, $\sum \tilde{x}$, $\sum \tilde{y}$ один раз для $m = 1$ (за $\Theta(N)$ операций), то для каждого последующего m при «перемещении» точки (x, y) между прямыми l и l' , перечисленные значения пересчитываются следующим образом:

$$\begin{aligned}
\sum \hat{x}^2 + \sum \hat{y}^2 + &= x^2 - y^2 \\
\sum \hat{x} + &= x \\
\sum \hat{y} - &= y \\
\sum \tilde{x} - &= x \\
\sum \tilde{y} + &= y \\
\hat{x}^T \tilde{y} + &= xy \\
\hat{y}^T \tilde{x} - &= xy
\end{aligned}$$

Таким образом, за фиксированное число шагов мы можем пересчитать выписанные ранее матрицы. Зная эти матрицы, вычисление (2.1), (2.2) так же занимает фиксированное, независящее от N , количество операций. Итоговая временная сложность алгоритма: $\Theta(N) + (N-1)\Theta(1) = \Theta(N)$. Алгоритм, вычисляющий ориентированную ограничивающую рамку, описанным выше способом мы далее будем называть L-Shape алгоритм.

2.2 Исследование точности алгоритма проекции визуальных детекций на основе модели L-Shape

Для оценки качества второго этапа алгоритма мы сравнили его с подходом, минимизирующем площадь вычисляемой рамки, а также с алгоритмами построения рамок для кластеров точек из статьи [31], адаптированными под проекции на вид сверху.

2.2.1 Набор данных

Сравнение алгоритмов выполнялось на наборе данных, полученном с БТС. Данный набор включает в себя реальные сцены из 4-х различных локаций:

складов открытого типа и закрытых территорий, и содержит 12 последовательностей кадров, снятых в разные сезоны и время суток. Суммарный размер набора составил 93 кадра. Примеры сцен из набора приведены на рис. 2.4. Каждая сцена, помимо RGB-изображений с монокулярной камеры содержит облака точек, полученные с двух 32-х лучевых лидаров, в которых были вручную аннотированы (обрамлены ограничивающими параллелепипедами) все автомобили и пешеходы. Проекция полученных таким образом ограничивающих параллелепипедов автомобилей на вид сверху были взяты в качестве эталонных положения объектов.



Рисунок 2.4 — Проекция двумерной ограничивающей рамки на вид сверху с учетом сегментации проезжей части.

2.2.2 Сравнимые алгоритмы

Для сравнения работы L-Shape алгоритма были взяты пять альтернативных подходов к решению задачи вписывания полигона в ориентированную ограничивающую рамку: подход, минимизирующий площадь итоговой рамки,

и четыре подхода, описанные в работе [31]. Последние были разработаны для трехмерной детекции, в виде ориентированных ограничивающих параллелепипедов вокруг кластеров облаков точек. Отметим, что в статье [31] получаемые данными подходами детекции затем передавались в нейросетевую модель для создания итоговой трехмерной детекции. В рамках данной работы мы не рассматривали данную модель, ограничиваясь исследованием вычислительно эффективных алгоритмов, не требующих высоких вычислительных ресурсов (в частности, не требующих, наличия графических ускорителей). В работе Ванга [31] точки кластеров проецировались на вид сверху, затем для полученных проекций строилась выпуклая оболочка, и далее выполнялось её вписывание в ориентированную рамку на виде сверху (которая позже может быть достроена до параллелепипеда минимальной высоты, включающего все точки исходного кластера). Так как мы в качестве входа алгоритма использовали не кластеры точек, а проекции объектов на вид сверху, то мы адаптировали эти подходы под наши входные данные.

Построение рамки минимальной площади

Тривиальным алгоритмом, решающим описанную задачу является построение рамки, которая включает в себя все вписываемые точки и имеет наименьшую площадь. Пусть $(x_i, y_i), i \in \{1, \dots, N\}$ – вершины выпуклой оболочки, указанные в порядке обхода. Алгоритм построения рамки минимальной площади опирается на то, что хотя бы одна сторона выпуклой оболочки принадлежит одной из сторон рамки. Для двумерного случая существует алгоритм с линейной временной сложностью по числу вершин N [32]. Этот алгоритм правильно определяет положение транспортного средства (ТС) на дороге, только если среди точек оболочки есть хотя бы по одной точке каждой стороны ТС. На практике нам редко удастся получить достаточное количество точек двух сторон, не говоря уже о трех сторонах. В таком случае, алгоритм выдаст неверное положение объекта, что может привести к нежелательным остановкам автономного ТС, использующего такой алгоритм, или, что еще хуже, автономная система может неправильно спланировать траекторию объезда препятствия. Пример такой ситуации представлен на рис. 2.5. Несмотря на недостатки метода, он может быть применен как способ построения одного из кандидатов для более сложных детекторов, учитывающих результаты работы нескольких алгоритмов, как это сделано в работе [31].

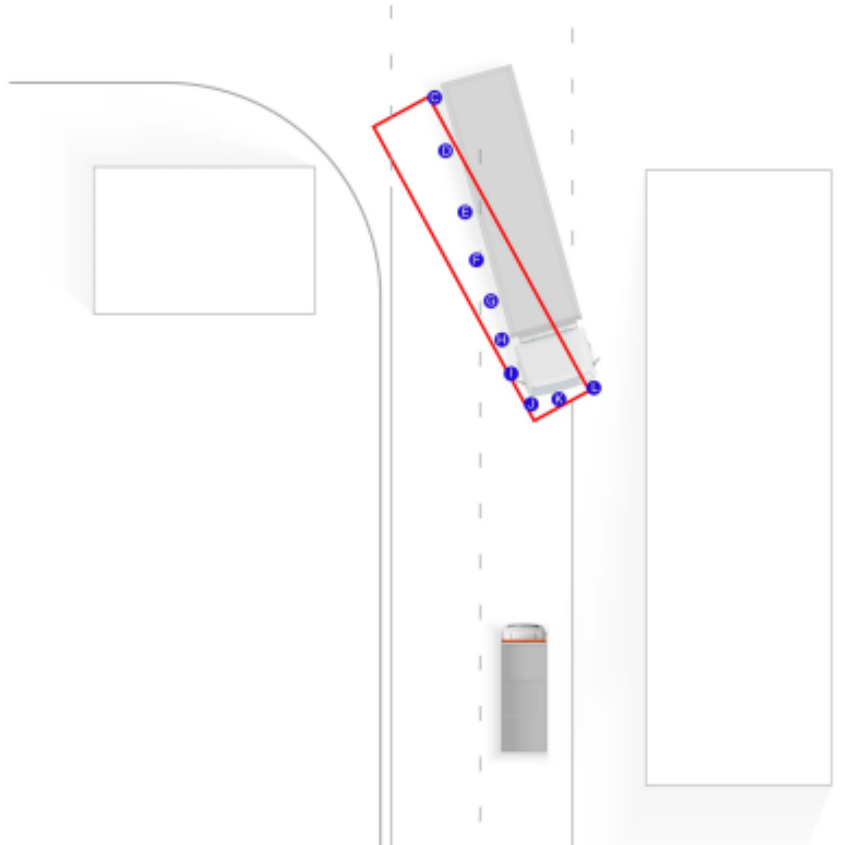


Рисунок 2.5 — Пример неправильно построенной ограничивающей рамки.

Алгоритмы построения рамок для кластеров точек, адаптированные под проекции на вид сверху

Из статьи Ванга [31] были взяты и адаптированы следующие алгоритмы вписывания полигона в ориентированную ограничивающую раму:

- **Rotating Principal Component Analysis Algorithm.** Идея алгоритма заключается в том, чтобы взять собственные векторы, полученные в результате анализа главных компонент, и построить ограничивающую рамку минимального размера так, чтобы стороны этой рамки были параллельны полученным векторам.
- **Diagonal Principal Component Analysis Algorithm.** Этот алгоритм похож на предыдущий с той разницей, что собственный вектор, соответствующий наибольшему собственному значению, берется за основу диагонали рамки, а не ее сторон. Поэтому после того, как собственный вектор найден, алгоритм подбирает две точки оболочки, которые образуют прямую, наиболее близкую к направлению вектора. Затем алгоритм выбирает самую дальнюю от этой прямой точку и принимает эту точку за угол ограничивающей рамки. Ограничивающая рамка за-

тем подгоняется и расширяется таким образом, чтобы заключить в себе все точки оболочки.

- **Longest Diameter Algorithm.** И снова этот алгоритм похож на предыдущий, но на этот раз диагональ ограничивающей рамки строится по самым дальним точкам оболочки, а собственный вектор не берется во внимание.
- **Longest Diameter Algorithm.** И снова этот алгоритм похож на предыдущий, но на этот раз диагональ ограничивающей рамки строится по самым дальним точкам оболочки, а собственный вектор не берется во внимание. Исходя из этого, можно достроить и расширить рамку, как это делалось при работе с другими алгоритмами.

2.2.3 Описание эксперимента

На изображениях из нашего набора данных запускался первый этап алгоритма, вычисляющий проекцию препятствия на вид сверху. Для сопоставления полученных проекций с эталонными данными вычислялись коэффициент Жаккара [26] для каждой пары проекция-эталон, после чего применялся Венгерский алгоритм [33].

Для каждой полученной проекции затем запускались все сравниваемые алгоритмы для второго шага алгоритма (L-Shape, рамка минимальной площади и алгоритмы из работы Ванга [31]), и для результирующего положения препятствия измерялся коэффициент Жаккара. Примеры полученных проекций, полученных ограничивающих рамок и эталонных положений представлены на рис. 2.6.

2.2.4 Результаты экспериментов.

В таблице 2 представлена статистика измеренных коэффициентов Жаккара для каждого алгоритма. Можно видеть, что представленный алгоритм L-Shape превосходит по качеству другие исследуемые алгоритмы. Худшие ре-

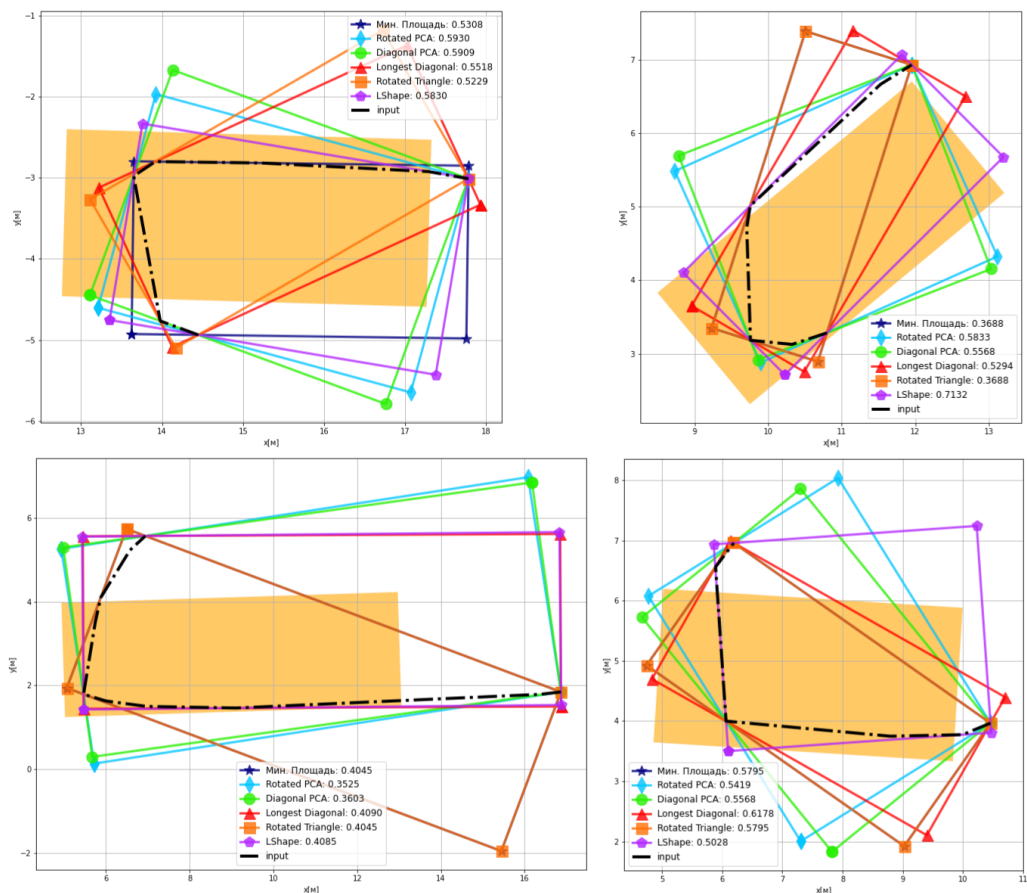


Рисунок 2.6 — Примеры работы сравниваемых алгоритмов построения ограничивающих рамок.

зультаты показал алгоритм Rotated Triangle. Однако само значение среднего коэффициента Жаккара сравнительно невелико в том числе и для представленного алгоритма L-Shape. Объясняется это в первую очередь ограниченной применимостью предположения о плоскости поверхности дороги, а также не идеальностью внешней калибровки камеры, вызванной наклоном дороги, загруженностью транспортного средства, изменившимся объемом шин и т.п. В результате предполагаемая плоскость дороги отклоняется от плоскости идеальной аппроксимации дороги и для удаляющихся препятствий наблюдается накопление ошибки. Помимо этого, погрешность первой части алгоритма: вычисления проекции объекта на вид сверху, выше для более удалённых объектов из-за линейной перспективы изображения. Чтобы учесть это, было предложено ограничить рабочее расстояние алгоритмов: не учитывать препятствия далее некоторого фиксированного порога. На Рис. 2.7 представлен график зависимости среднего коэффициента Жаккара в зависимости от данного порога. Видно, что качество предсказываемого положения препятствия начинает падать после

10 метров для всех рассматриваемых алгоритмов. Также можно видеть, что при всех значениях L-Shape алгоритм сохраняет первенство по измеряемой метрике.

Характеристика	Мин. пло- щадь	Rotating PCA	Diagonal PCA	Longest Diagonal	Rotated Triangle	L-Shape
Среднее	0.294 (−12.8%)	0.328 (−2.7%)	0.321 (−4.7%)	0.323 (−4.2%)	0.301 (−10.7%)	0.337 (−0.0%)
Среднеквадратичное отклонение	0.146	0.144	0.149	0.156	0.141	0.141
Минимум	0.059 (−24.4%)	0.075 (−3.8%)	0.078 (−0.0%)	0.074 (−5.1%)	0.059 (−24.4%)	0.078 (−0.0%)
25%	0.198 (−13.9%)	0.219 (−4.8%)	0.202 (−12.2%)	0.199 (−13.5%)	0.208 (−9.6%)	0.230 (−0.0%)
50%	0.257 (−21.6%)	0.316 (−3.7%)	0.316 (−3.7%)	0.287 (−12.5%)	0.264 (−19.5%)	0.328 (−0.0%)
75%	0.383 (−15.1%)	0.448 (−0.7%)	0.445 (−1.3%)	0.442 (−2.0%)	0.425 (−5.8%)	0.451 (−0.0%)
Максимум	0.649 (−9.0%)	0.607 (−14.9%)	0.622 (−12.8%)	0.651 (−8.7%)	0.579 (−18.8%)	0.713 (−0.0%)

Таблица 2 — Статистики полученных коэффициента Жаккара для сравниваемых алгоритмов

2.3 Алгоритм проекции визуальных детекций на плоскость движения БТС на основе комплексирования данных с данными лазерных дальномеров

Одним из фундаментальных ограничений описанного в прошлом разделе L-Shape алгоритма является предположение о том, что поверхность проезжей части лежит в плоскости. В реальном мире это предположение не выполняется как в силу рельефа местности, в которой проложена дорога, так и согласно государственным стандартам и сводам правил, регулирующим конструктивные особенности дорожного покрытия. Так согласно СП 34.13330.2012[34] дорожное покрытие имеет двускатный поперечный профиль на прямых участках дороги и односкатный на виражах (см. рис. 2.8).

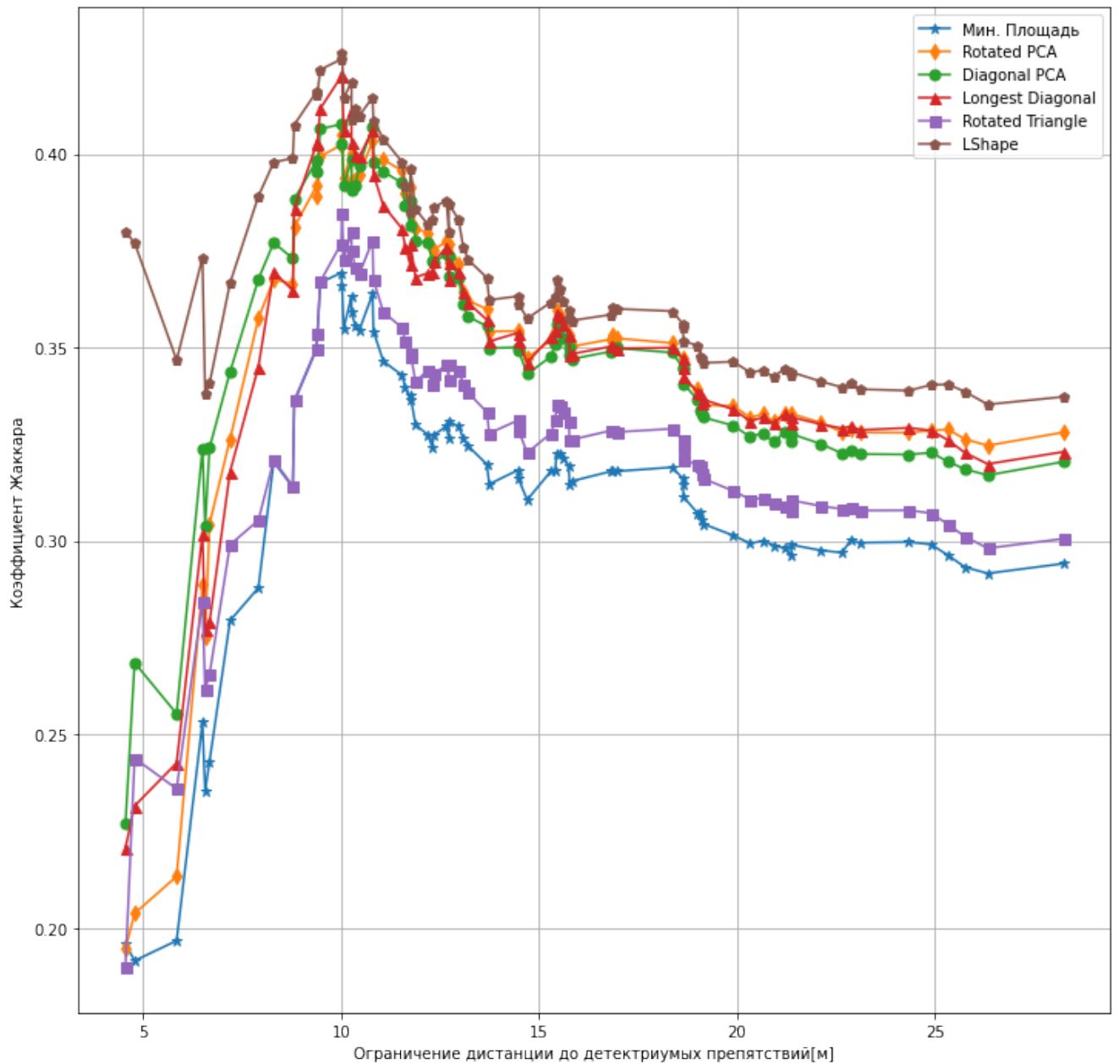


Рисунок 2.7 — График зависимости среднего коэффициента Жаккара от ограничения дистанции до детектируемых препятствий.

Для преодоления данного ограничения необходимо добавить источник информации, позволяющий оценить расстояние до наблюдаемых объектов без использования упомянутого предположения. В качестве такого источника информации были использованы механические лазерные дальномеры (лидары), установленные на БТС. Был разработан алгоритм комплексирования данных, полученных с камеры и лазерных дальномеров (лидаров) для оценки положения окружающих БТС объектов. Облако точек лидара представляет собой набор точек в трёхмерной системе координат лидара: $P_s = \{(x_i, y_i, z_i), i \in \{1, \dots, N_s\}\}$, где s — идентификатор лидара, N_s — количество зарегистрированных им точек.

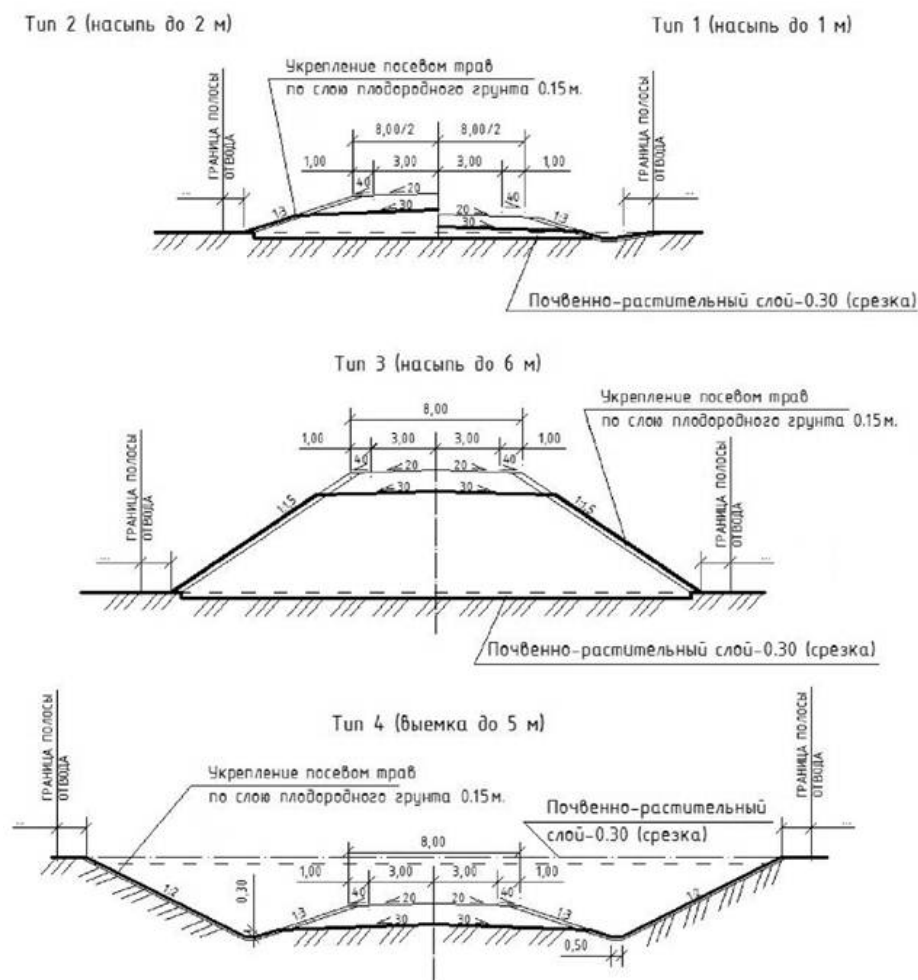


Рисунок 2.8 — Примеры оформления поперечного профиля из ГОСТ 21.701-2013 [35]

Все облака точек, а также изображения, полученные камерой, имеют временную метку момента регистрации датчиком. Сенсоры работают с фиксированной частотой и синхронизированы друг с другом. Это позволяет однозначно сопоставить данные из последовательности измерений с датчиков в единые «кадры», описывающие окружающую БТС сцену в конкретный момент времени. Далее будем считать, что все облака точек с каждого лидара:

1. Приведены в локальную систему координат БТС O_l .
2. Точки попадающие на БТС удалены, что возможно при известной геометрии БТС.
3. Положение точек скорректировано и приведено к одному моменту времени, чтобы компенсировать собственное движение БТС во время регистрации облака точек.
4. После предыдущих манипуляций облака объединены в одно общее облако точек P .

После этих преобразований, один «кадр» представляет собой пару изображение–облако точек. В Листинге 1 представлен алгоритм осуществляющий комплексирование данных одного кадра для оценки положения препятствий в окружающей БТС сцене: из облака выделяется подмножество точек, не принадлежащих земле, производится объединение близко расположенных точек в кластеры, которые затем сопоставляются в 2D детекциями, найденными на изображении. Наконец, кластеры, которым нашлось сопоставление среди детекций дополнительно фильтруется от точек, проецируемых за пределы соответствующей детекции и находится ограничивающий параллелепипед описанный вокруг полученного набора точек. На рисунке 2.9 показан результат работы представленного алгоритма на данных, полученных с датчиков БТС. Отметим, что шаг 3 алгоритма 1 был реализован в виде алгоритма с использованием параллельных вычислений на платформе CUDA (англ. Compute Unified Device Architecture) [36]. Данная реализация была предложена в работе Нгуена [37].

2.3.1 Разделение облака точек на точки земли и препятствий

Отметим нетривиальность шага 2 алгоритма 1. Зачастую такое разделение производится поиском с помощью RANSAC плоскости, хорошо описывающей часть точек в облаке. Однако, такой подход возвращает нас к предположению о совпадении поверхности дороги с некоторой плоскостью, от которого мы нацелены уйти. Поэтому в рамках данного алгоритма используется подход без использования поиска плоскостей. В рамках данного подхода мы сперва производим поиск точек, для которых мы уверены, что они принадлежат земле. Далее мы решаем задачу регрессии: мы хотим найти зависимость высоты земли z_l от координаты в плоскости движения БТС (x_l, y_l) . В качестве обучающей выборки используются выбранные нами в начале точки. В качестве тестовых точек берутся координаты (x_l, y_l) входного облака точек. Мы решаем данную задачу регрессии на основе гауссовских процессов (Gaussian Process Regression, GPR) [39].

Регрессия на основе гауссовских процессов представляет собой байесовский непараметрический подход к аппроксимации функции, задающей

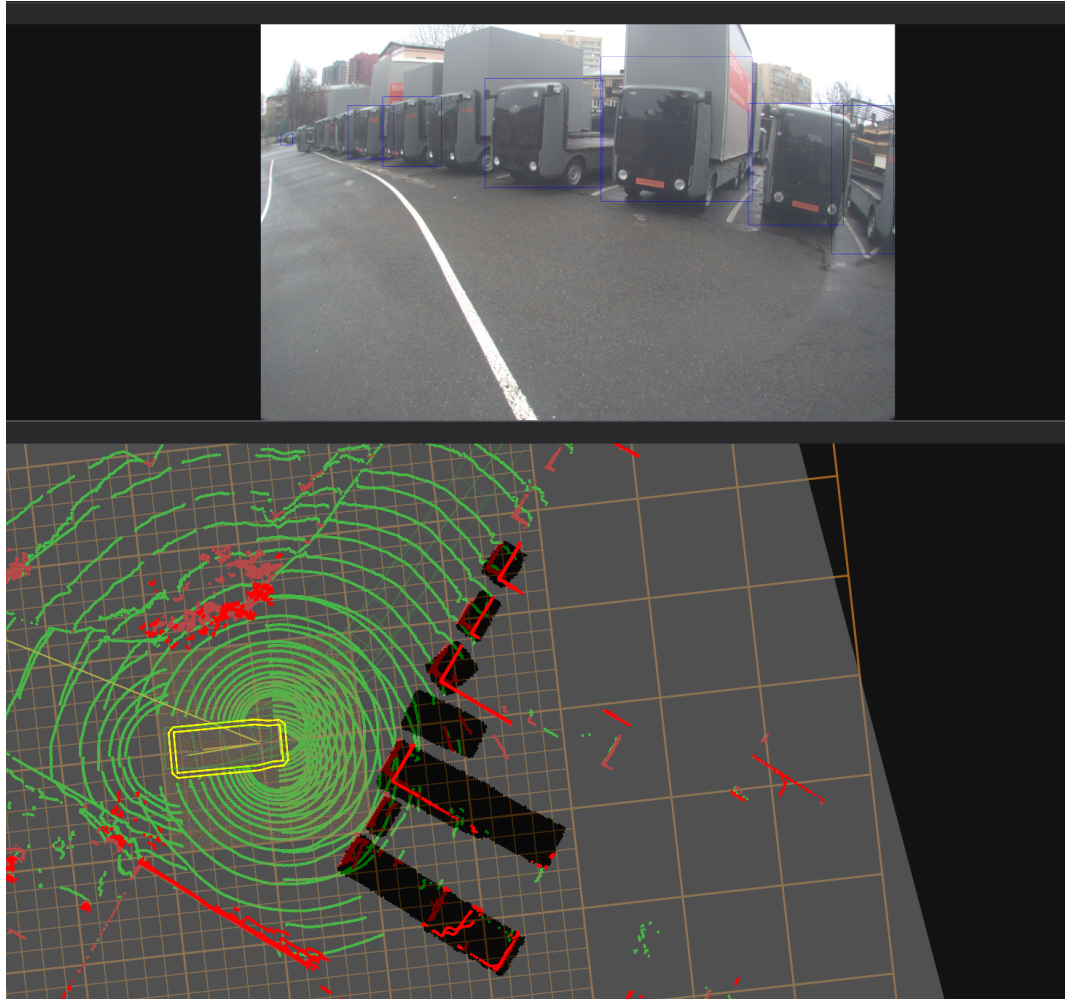
Input: I – изображение с RGB-камеры, P – облако точек.

Output: b_i – ориентированные обрамляющие рамки на виде сверху окружающих БТС объектов.

- 1 Произвести поиск 2D-детекции объектов на I : $\{d_1, \dots, d_n\}$;
- 2 Выделить в облаке точек P точки, не принадлежащие поверхности земли: P_o ;
- 3 Кластеризовать полученное облако точек P_o с расстоянием Евклида:

$$P_o = \bigcup_{i=1}^l C_i; C_i \cap C_j = \emptyset, i \neq j;$$
- 4 Спроецировать с учётом дисторсии все кластеры на изображение I : $C_i \rightarrow \hat{C}_i$. Все точки проецируемые за границы I , а также находящиеся позади оптического центра камеры отбрасываются. Пустые после проекции кластеры – удаляются;
- 5 Для каждого спроецированного кластера найти обрамляющую рамку минимального размера: d_{C_i} ;
- 6 Используя Венгерский алгоритм найти наилучшее сопоставление найденных детекций d_i с построенными ограничивающими рамками d_{C_i} , максимизирующий суммарный коэффициент Жаккарда. Не сопоставленные кластеры – удаляются.;
- 7 В оставшихся кластерах удалить точки, не попадающие при проекции в сопоставленную детекцию d_i : $C_i \rightarrow C_i^-$.;
- 8 **foreach** C_i^- **do**
 - 9 | Используя RANSAC(RANdom SAMple Consensus) [38] найти прямую l_i , хорошо описывающую большую часть точек «очищенного» кластера C_i^- .;
 - 10 | Аналогично L-Shape алгоритму построить ограничивающую рамку b_i , со сторонами параллельными l_i и ортогональной ей l'_i .;
- 11 **end**
- 12 **return** $\{b_i\}$

Алгоритм 1: Алгоритм определения положения объектов на основе комплексов данных с камеры и лазерных дальномеров



На верхнем изображении с камеры BTS прямоугольниками выделены 2D детекции, полученные моделью YOLOP. На нижнем красными точками отмечены точки, зарегистрированные лазерным дальномером и классифицированные как точки препятствия. Полученные алгоритмом детекции показаны в виде чёрных прямоугольников.

Рисунок 2.9 — Демонстрация работы алгоритма 1 на данных, полученных с BTS

зависимость значения целевой переменной от вектора признаков. В рассматриваемой задаче целью является аппроксимация функции $f : (x_l, y_l) \rightarrow z_l$, где (x_l, y_l) – координаты точки в плоскости движения BTS, а z_l – соответствующая высота земли.

Гауссовский процесс задаётся как распределение на множестве функций и полностью определяется средним значением $m(\mathbf{x})$ и ковариационной функцией (ядром) $k(\mathbf{x}, \mathbf{x}')$, где $\mathbf{x} = (x_l, y_l)$. В нашем сценарии мы использовали ядро Матерна ($\nu = 3/2$) [40], которое лучше справляется с аппроксимацией в точках разрыва функции (рисунок 2.10). Такие разрывы часто присутству-

ют в реальных сценах вокруг БТС, например, в местах соединения проезжей части и тротуаров.

$$k(\mathbf{x}, \mathbf{x}') = \sigma_f^2 \left(1 + \frac{\sqrt{3}d}{l} \right) \exp \left(-\frac{\sqrt{3}d}{l} \right),$$

где $d = \|\mathbf{x} - \mathbf{x}'\|$ — евклидово расстояние между точками, σ_f^2 — вариация функции, а l — гиперпараметр длины масштаба.

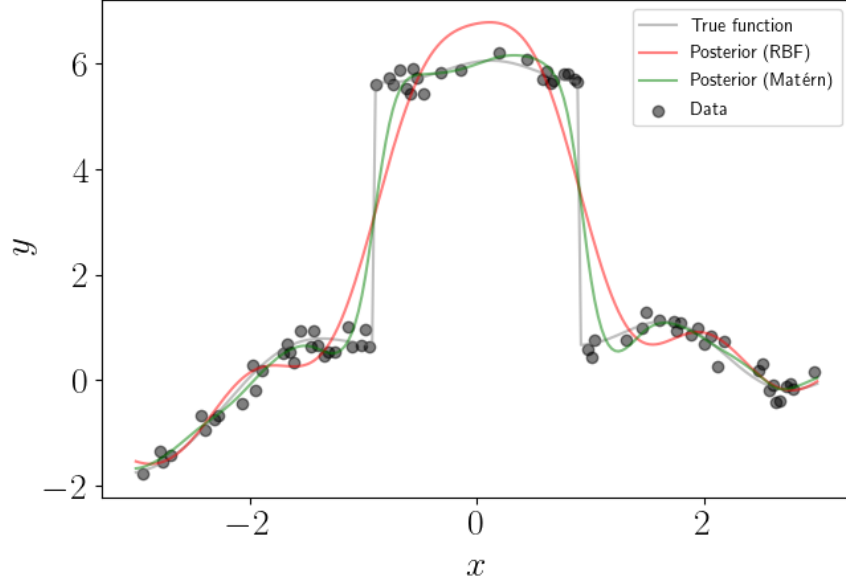


Рисунок 2.10 — Сравнение регрессии на основе гауссовских процессов с использованием в качестве ядра радиальной базисной функции и ядра Матерна [41]

Для обучающей выборки размером N_t с входами $\mathbf{X} = \{\mathbf{x}_i\}_{i=1}^{N_t}$ и соответствующими наблюдениями высоты $\mathbf{z} = \{z_i\}_{i=1}^{N_t}$ задача регрессии сводится к вычислению апостериорного распределения значений z_* для тестовых точек $\mathbf{X}_* = \{\mathbf{x}_*\}$.

Обозначим ковариационные матрицы:

$$K = [k(\mathbf{x}_i, \mathbf{x}_j)]_{i,j=1}^{N_t}, \quad K_* = [k(\mathbf{x}_*, \mathbf{x}_i)]_{i=1}^{N_t}, \quad K_{**} = [k(\mathbf{x}_*, \mathbf{x}_*)]_{i,j=1}^N, \quad (2.3)$$

тогда условное распределение предсказаний $p(z_* | \mathbf{X}, \mathbf{z}, \mathbf{x}_*)$ является гауссовским с параметрами:

$$\hat{z} = \mathbb{E}[z_*] = K_* K^{-1} \mathbf{z}, \quad (2.4)$$

$$\text{Var}[z_*] = K_{**} - K_* K^{-1} K_*^\top.$$

Таким образом, предсказанное значение высоты земли для любой точки (x_l, y_l) получается как взвешенная сумма наблюдений обучающей выборки с учётом ковариации между точками.

Заметим, что регрессия на основе гауссовских процессов – вычислительно дорогая операция: в стандартных реализациях она занимает $O(N_t^3)$, где N_t – количество точек в обучающей выборке, т.к. включает разложение Холецкого. В реальных данных присутствуют десятки тысяч точек, принадлежащих земле. Чтобы алгоритм имел возможность работать в режиме реального времени мы вычисляем усреднённые по областям значения координат точек, которые мы считаем принадлежащими земле и берём их подвыборку в размере нескольких сотен точек.

Наконец, мы для каждой точки входного облака мы сравниваем её измеренную координату z_l и предсказанную $\hat{z}_l$: если z_l больше $\hat{z}_l$ на некоторый порог z_{thr} , мы считаем данную точку не принадлежащей земле. Отметим, что ввиду принципа формирования лидарных данных плотность точек уменьшается при отдалении от машины, ввиду чего растёт и дисперсия предсказываемого значения $\hat{z}_l$. Чтобы учесть этот факт мы делаем величину z_{thr} зависимой от расстояния до БТС: $z_{thr} = f(\|(x_l, y_l)\|_2)$. В Листинге 2 представлена реализация описанного алгоритма. На рисунке 2.11 представлен результат работы алгоритма.

Отметим, что данный алгоритм может быть реализован с помощью параллельных вычислений, в частности используя программно-аппаратную архитектуру CUDA, что позволяет снизить время выполнения данного алгоритма.

2.4 Алгоритм проекции сегментации проезжей части на изображении на плоскость движения ТС

Помимо определения положения препятствий в окружающей БТС сцене важную роль в осуществлении безопасного решения задач планирования пути играет включение в модель описание областей свободных для перемещения

Input: P – облако точек.

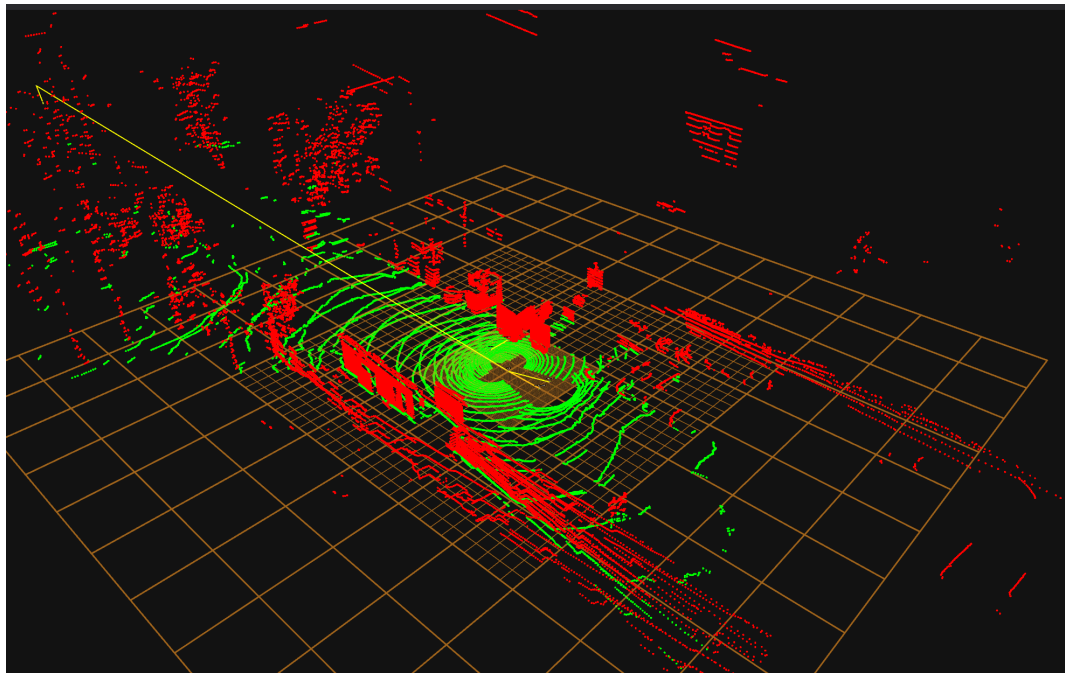
Output: P_g – облако точек, принадлежащей земле, P_o – облако точек, не принадлежащих земле

```

1 Копируем облако  $P$  в  $P_c$ ;
2  $P_c$  разделяется на воксели  $V_i$  – области пространства, получаемые при
   накладывании трёхмерной сетки фиксированного разрешения;
3  $P_c$  разделяется иным образом сеткой в полярных координатах на сектора  $S_i$ ;
4 foreach  $V_i$  do
5     Для точек в вокселе вычисляется среднее  $m$  и среднеквадратичное отклонение
        $\sigma$  координаты  $z_l$  в локальной системе координат БТС  $O_l$ ;
6     Из  $P_c$  удаляются точки с  $|z_l - m| > \alpha\sigma$ , где  $\alpha$  – фиксированный параметр
       алгоритма;
7 end
8 foreach  $S_i$  do
9     Находится минимальное значение  $z_{min}$   $z_l$  точек в  $S_i$ ;
10    Из  $P_c$  удаляются точки, у которых  $z_l > z_{min} + T$ , где  $T$  – фиксированный
       параметр алгоритма;
11 end
12 Из вокселей  $V_i$  случайным образом выбираются  $N_{train}$  вокселей, где  $N_{train}$  –
    параметры алгоритма;
13 Для выбранных вокселей вычисляется центр масс  $(mx_i, my_i, mz_i)$  точек внутри
    вокселя. Полученные точки образуют множество  $P_{train}$ ;
14 Применяется регрессия на основе гауссовских процессов для оценки высоты  $\hat{z}_l$ , в
    точках  $(x_i, y_i)$ ;  $\forall p_i = (x_i, y_i, z_i) \in P$ , используя в качестве обучающей выборки
     $((x_i, y_i), z_i)$ ;  $\forall p_i = (x_i, y_i, z_i) \in P_{train}$   $P_g = \{\}$ ;
15 foreach  $p_i = (x_i, y_i, z_i) \in P$  do
16      $S(p_i)$  – сектор, в котором находится точка  $p_i$ ;
17     if  $z_i < \hat{z}_i + z_{thr}(S(p_i))$  then
18         // (
19          $z_{thr}(S(p_i))$  – массив порогов по высоте для каждого сектора)  $P_g + = p_i$ ;
20     end
21 end
22  $P_o = P \setminus P_g$ ;
23 return  $P_g, P_o$ ;

```

Алгоритм 2: Алгоритм фильтрации точек земли



а) Результат работы алгоритма.

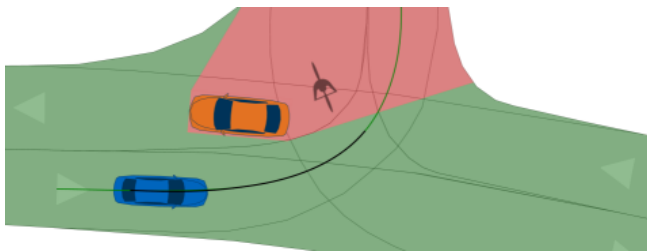


б) Изображение сцены с камеры BTS.

Рисунок 2.11 — Пример работы алгоритма разделения облака точек земли и препятствий.

БТС. Это позволяет алгоритмам планирования пути различать области, для которых достоверно известно отсутствие в нём каких либо объектов, от областей, для которых отсутствует информация, позволяющая судить о его проходимости ввиду окклюзии либо недостоверности регистрируемых данных. На Рисунке 2.12 приведён пример, демонстрирующий важность этого различия. Мы можем обучить нейросетевую модель YOLOP или другую модель, осуществля-

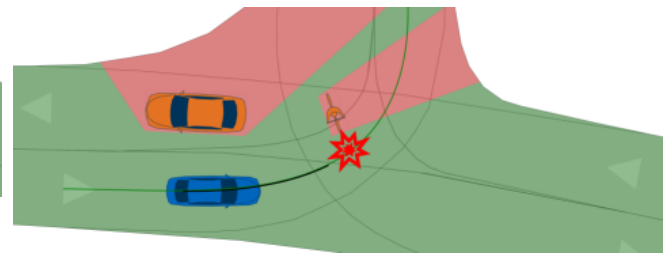
ющую семантическую сегментацию двухмерных изображений, сегментировать проезжую часть на изображении. Однако аналогично задаче проецирования препятствий в трехмерную систему координат БТС O_l , описанной в разделе 2.1, мы столкнёмся с необходимостью спроецировать полученную сегментацию на вид сверху. В данном разделе предложен алгоритм, осуществляющий данное проецирование на основе комплексирования данных с камеры и лазерных дальномеров, установленных на БТС. Дополнительно будет предложена модификация алгоритма, позволяющая добавлять при проецировании области, содержащие некрупные препятствия.



а) БТС (синий) не видит

велосипедиста позади автомобиля.

Зелёным выделена территория в поле видимости датчиков БТС, красным – области, о которых нет информации ввиду их окклюзии



б) При продолжении движения

велосипедист детектируется

датчиками БТС, но из-за близости

объекта, у систем управления остаётся

мало времени для реагирования.

Рисунок 2.12 — Демонстрация влияния окклюзии на безопасность движения БТС [42].

В отличие от типичных препятствий в окружающей БТС сцене: людей, автомобилей; размеры и положение которых с высокой точностью описываются ориентированными ограничивающими рамками, проезжая часть после проекции имеет сложную геометрию (рисунок 2.13). Из-за этого векторное представление проезжей части на виде сверху может быть неудобным для построения. Вместо него мы воспользуемся представлением в виде сетки, фиксированного разрешения, где каждому элементу будет сопоставлен класс «свободно» – в случае если в соответствующей области БТС может безопасно проезжать, или «неизвестно» – в случае если информации об области недостаточно (а далее и дополнительный класс «занято» для областей, где обнаружен объект).



Рисунок 2.13 — Пример семантической сегментации проезжей части моделью YOLO на изображении с камеры БТС

Прежде чем приступить к описанию алгоритма опишем интуицию, которая будет лежать в его основе без использования информации с лазерных дальнометров. Воспользуемся предположением в наивном подходе проецирования препятствий из раздела 2.1 о совпадении поверхности дорожной части с плоскостью $z_l = 0$. В такой модели оценка проходимости элемента сводится к проекции центров сетки на плоскость изображения (как и прежде мы считаем внутреннюю и внешнюю калибровки камеры известными), если при проецировании центр элемента попал на пиксель, классифицированный как проезжая часть – соответствующий элемент сетки классифицируется «свободным», в противном случае «неизвестно» (рисунок 2.14). Отметим, что в последнем случае нельзя утверждать, что область не свободна для проезда, т.к. она может быть заслонена другим объектом – см. рисунок 2.15). Листинг 3 приводит описание данного алгоритма.

Однако, мы уже показали низкое качество моделей, использующих предположение о совпадении поверхности проезжей части с плоскостью. Ослабим это предположение и обобщим алгоритм на этот случай. Будем считать, что поверхность описывается некоторой известной функцией $z_l = z(x_l, y_l)$. Тогда в ранее описанный алгоритм достаточно добавить вычисление этой координаты перед проецированием на изображение (Листинг 4). На рисунке 2.16 представлена иллюстрация обобщённого алгоритма. Очевидно, для корректной работы алгоритма необходимо, чтобы более близкие к БТС участки проезжей части не перекрывали обзор камеры для более удалённых участков. При достаточ-

Input: S_I – семантическая сегментация проезжей части на изображении I ; ppm – разрешение сетки[пикс/м], на которую производится проецирование; l – ширина и высота сетки[м]

Output: S_{BEV} – проекция сегментации проезжей части на вид сверху

```

1  $s \leftarrow \frac{1}{\text{ppm}}$  ;
2  $L \leftarrow l \cdot \text{ppm}$  ;
3  $S_{BEV} \leftarrow$  матрица размера  $L \times L$ ;
4 for  $i \leftarrow 0$  to  $L$  do
5   for  $j \leftarrow 0$  to  $L$  do
6      $x \leftarrow -\frac{l}{2} + js$ ;
7      $y \leftarrow -\frac{l}{2} + is$ ;
8      $(u, v) \leftarrow$  проекция  $(x, y, 0)$  на изображение (с учётом дисторсии);
9     if  $(u, v)$  попадает на изображение AND  $S_I[u][v] == \text{«проезжая часть»}$  then
10       $S_{BEV}[u][v] \leftarrow \text{«свободно»}$ ;
11    else
12       $S_{BEV}[u][v] \leftarrow \text{«неизвестно»}$ ;
13    end
14  end
15 end
16 return  $S_{BEV}$ ;

```

Алгоритм 3: Алгоритм проецирования проезжей части на вид сверху (наивный)

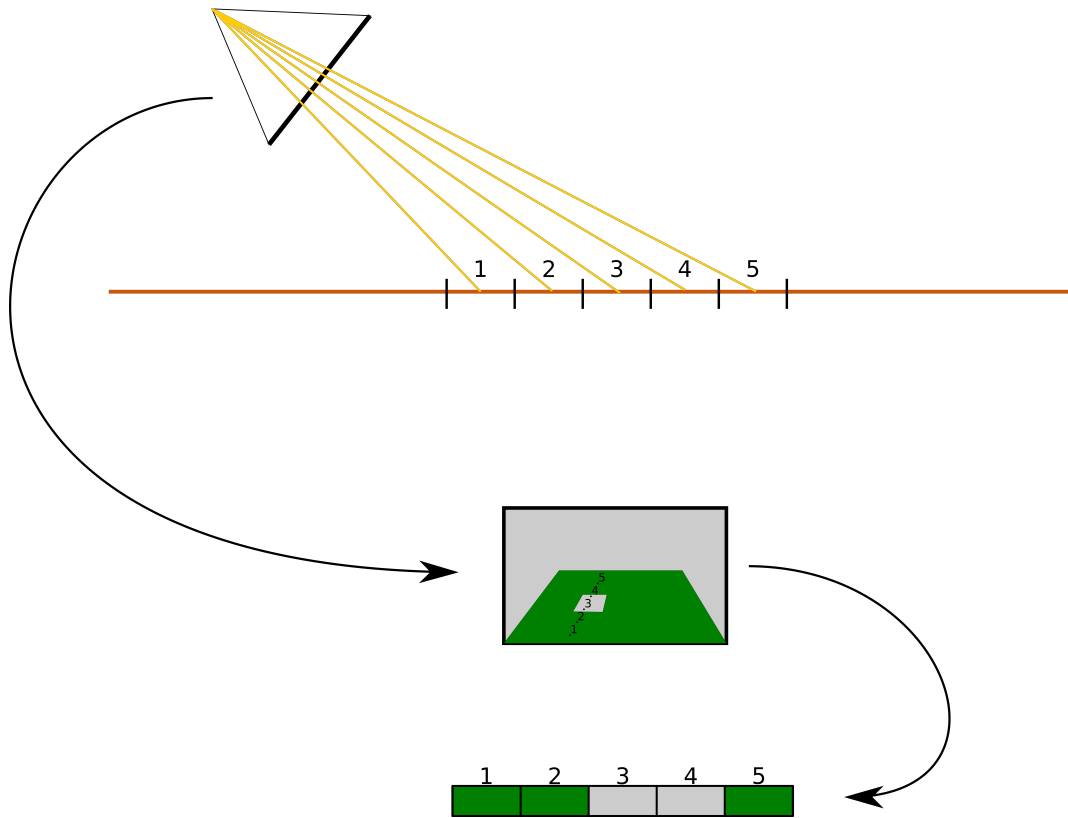


Рисунок 2.14 — Иллюстрация алгоритма проецирования в сценарии, когда проезжая часть лежит в известной плоскости. Для облегчения восприятия проецируемая сетка представлена одним рядом

ной гладкости проезжей части, разумном разрешении итоговой сетки на виде сверху и достаточной высоте расположения камеры, это требование будет выполняться.

- 1 Аналогично шагам 1-7 из Листинга 3;
- 2 $(u; v) \leftarrow$ проекция $(x, y, z(x, y))$ на изображение (с учётом дисторсии);
- 3 Аналогично шагам 9-16 из Листинга 3;

Алгоритм 4: Обобщение алгоритма проецирования проезжей части на вид сверху

2.4.1 Вычисление функции $z(x, y)$.

Для завершения разработки алгоритма необходимо предложить способ оценить значения $z(x, y)$ в центрах сетки, на которую производится проекция

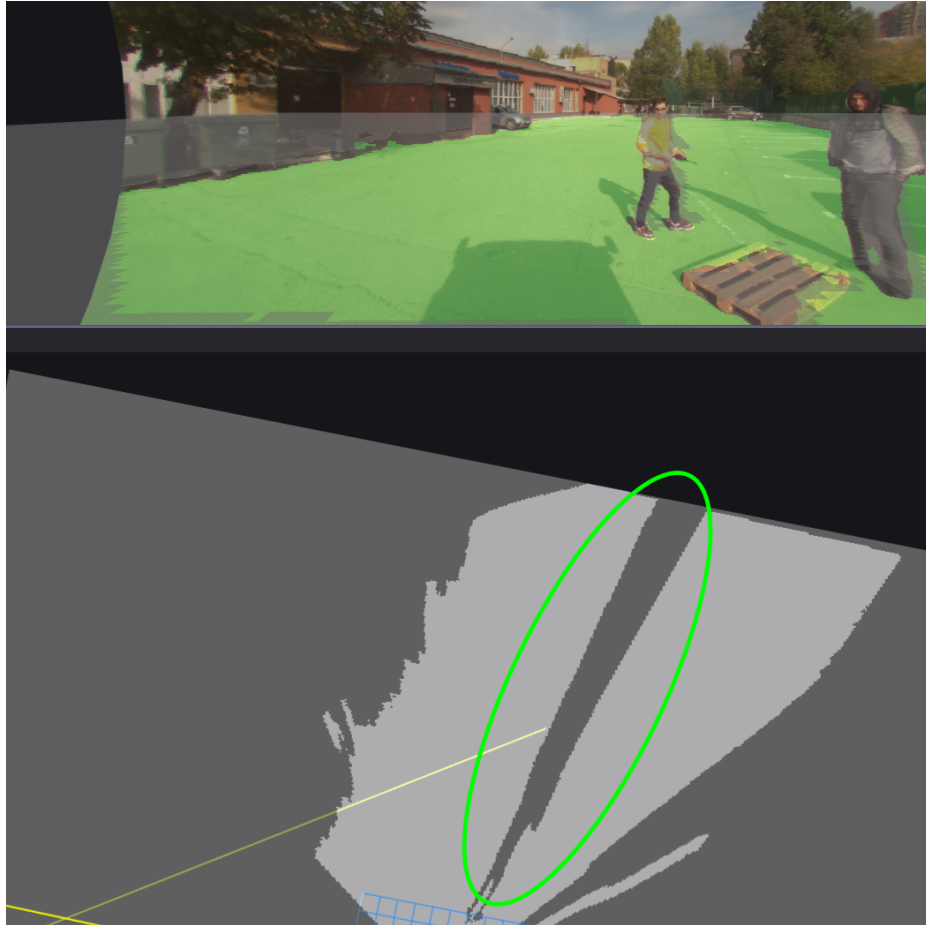


Рисунок 2.15 — Пример заслонённой области свободной для проезда

сегментации. Для этого аналогично разделу 2.3 мы воспользуемся комплексированием данных, зарегистрированными лидарами. Как и прежде мы будем считать, что облака точек с лидаров прошли предварительную обработку и представлены единым облаком точек с компенсированным движением БТС во время регистрации облаков. Для дальнейшего вычисления $z(x,y)$ предложим два подхода:

1. **Использовать облако точек земли, полученных алгоритмом 2.**

В этом случае облако вычисляются прямоугольные проекции на сетку. За z тех элементов сетки, в которых оказалась проекции точек, берётся среднее, минимум, максимум или произвольное из координат z этих точек. Для оставшихся не инициализированных точек необходимо произвести триангуляцию точек попавших на сетку. Далее для каждой точки сетки находится треугольник, содержащий её. Высота анализируемой точки получается путём линейной интерполяции. Пусть v_1, v_2, v_3 — вершины выбранного треугольника на виде сверху, z_1, z_2, z_3 — координата z этих вершин соответственно. Пусть вершина v , координата z

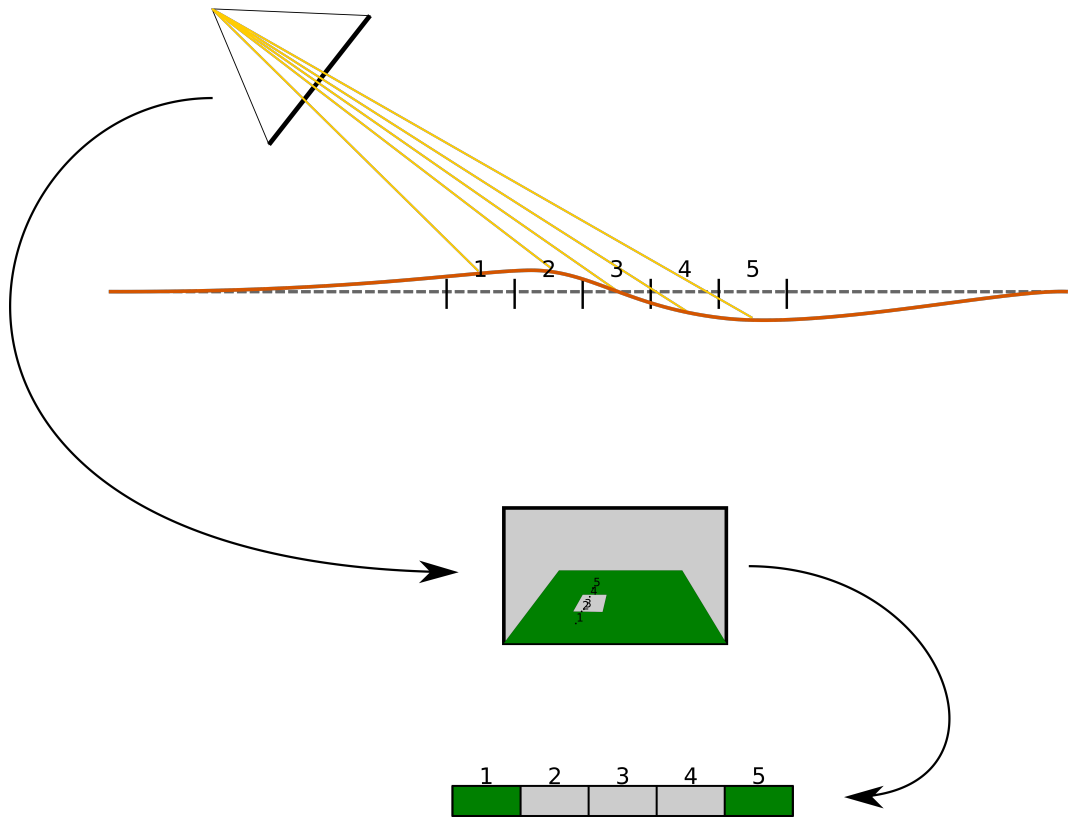


Рисунок 2.16 — Иллюстрация алгоритма проецирования в сценарии, когда поверхность проезжей части удовлетворяет уравнению $z = z(x, y)$. Для облегчения восприятия проецируемая сетка представлена одним рядом

которой интерполируется, образует с вершинами выбранного треугольника три треугольника площадью S_1, S_2, S_3 . Тогда искомая координата вычисляется по формуле: $z = \frac{z_1 S_1 + z_2 S_2 + z_3 S_3}{S_1 + S_2 + S_3}$ [43] (рисунок 2.17). В качестве триангуляции используется триангуляция Делоне [44]. По определению, при такой триангуляции треугольники обладают тем свойством, что описанные вокруг них окружности не содержат других триангулируемых точек. Поскольку лидарные точки земли по принципу работы датчика образуют приближённо набор дуг (так называемых «колец»), это свойство гарантирует, что интерполяция будет происходить по точкам, принадлежащим разным кольцам, что делает её более точной. Рисунок 2.18 демонстрирует разницу в выборе интерполируемых трёх точек при триангуляции Делоне и трёх ближайших соседей.

2. **Добавить в алгоритм 2 дополнительные тестовые точки.** В качестве этих точек будут выступать центры искомой сетки. Недостатком такого подхода станет увеличение времени работы алгоритма 2, что замедлит время обработки алгоритмы, использующие его результаты, в

частности, алгоритма 1. Однако при наличии вычислительных ресурсов на ГПУ вычисление регрессии можно распараллелить с использованием механизма мультиточности CUDA Streams [45]. В этом случае в регрессии можно переиспользовать матрицу K из уравнения (2.3). Затем вычислить ковариационные матрицы $\widetilde{K}_*$, $\widetilde{K}_{**}$ для новых тестовых точек, и аналогично (2.4) вычислить аппроксимацию z в тестовых точках. На рисунке 2.19 представлены результаты профилирования ресурсов на ГПУ с помощью Nsight Systems [46] изменившейся части программы, реализующей алгоритм 2 при последовательном и параллельном решении задач регрессии. Программа реализована на языках C++ и CUDA C++. Профилирование происходило на ПК с процессором AMD Ryzen 5 5600x с 6-ю ядрами и тактовой частотой 4.6 ГГц, ОЗУ объёмом 32 Гб, а также ГПУ Nvidia GeForce 3070 с графической памятью 8 Гб и тактовой частотой 1.5 ГГц. Количество основных точек составило порядка 100000 точек, количество дополнительных точек составило 10000. На рисунке 2.20 представлен график сравнения распределения времени работы алгоритма разделения при последовательном и параллельном вычислениях.

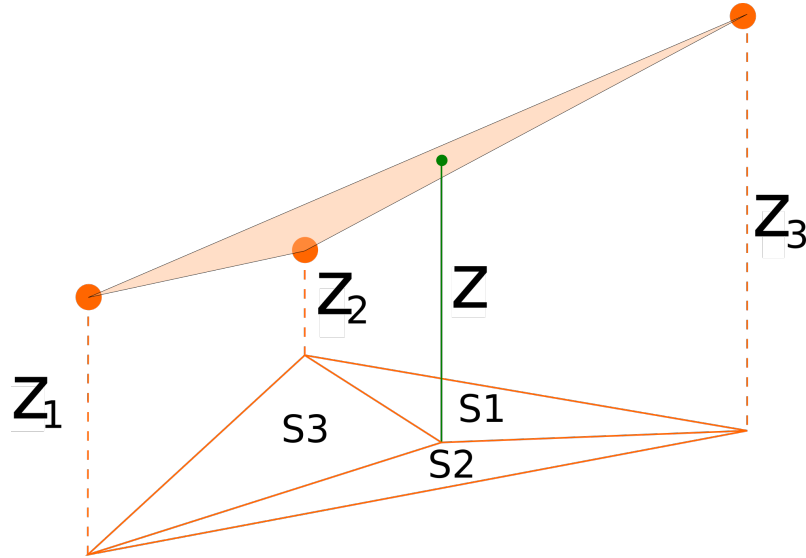


Рисунок 2.17 — Линейная интерполяция z в треугольнике.

Отметим, что второй подход возможно реализовать с использованием параллельных вычислений средствами CUDA: т.к. после регрессии на основе гауссовских процессов каждый элемент сетки, на который проецируется сегментация обрабатывается независимо от других элементов. В то же время первый подход требует произвести триангуляцию Делоне, для которой реализации на

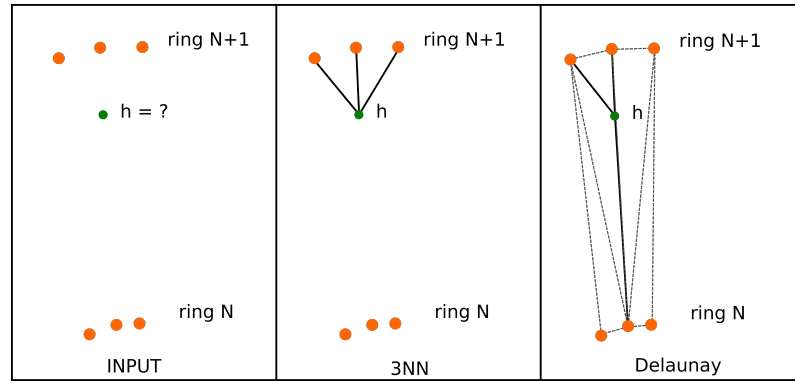


Рисунок 2.18 — Линейная интерполяция z в треугольнике.

ГПУ существуют [47], но эксперименты с данной реализацией показали замедление алгоритма на 10 мс или 20% в сравнении с реализацией на ЦПУ, которая занимает в среднем 50 мс.

2.4.2 Модификация алгоритма для обнаружения препятствий на проезжей части

В разделе 2.3 был представлен алгоритм оценки положения детекций на изображении в трехмерной системе координат БТС O_l . Ограничением этого алгоритма является необходимость алгоритма детекции объектов быть обученным на распознавание конкретных типов объектов. Как правило, такие модели способны надёжно распознавать людей, машин, животных, дорожные знаки. Однако многие типы объектов в реальном мире встречаются редко либо их объекты сильно различаются между собой из-за чего обучение модели для них представляется возможным ввиду малого размера обучающей выборки. В то же время в контексте решаемой нами задачи можно заметить, что проезжая часть редко содержит «вырезы» малого размера особенно на пути следования БТС. Воспользовавшись этим предположением можно модифицировать алгоритм 4: после завершения проецирования сегментации на вид сверху произвести поиск замкнутых областей, классифицированных как «неизвестно» внутри полученной проекции, площадью в некотором интервале $[a_{low}, a_{high}]$ и задать им новый класс «занято» (Листинг 5). Ограничение сверху требуется для того, чтобы избежать неверной классификации как «занято» вырезов образованных заслонением крупным объектом, например, разделителем дороги или челове-

ком (рисунок 2.21). Ограничение снизу используется для защиты от возможных артефактов в сегментации. Рисунок 2.22 демонстрирует результат работы алгоритма на изображении, полученном с камеры БТС.

```

1 Аналогично шагам Листинга 4;
2 Произвести поиск «вырезов»  $C_i$  на полученной проекции  $S_{BEV}$ ;
3 foreach  $C_i$  do
4   | if  $area(C_i) \in [a_{low}, a_{high}]$  then
5   |   |  $S_{BEV} \cap C_i \leftarrow$  «занято»;
6   | end
7 end
8 return  $S_{BEV}$ ;

```

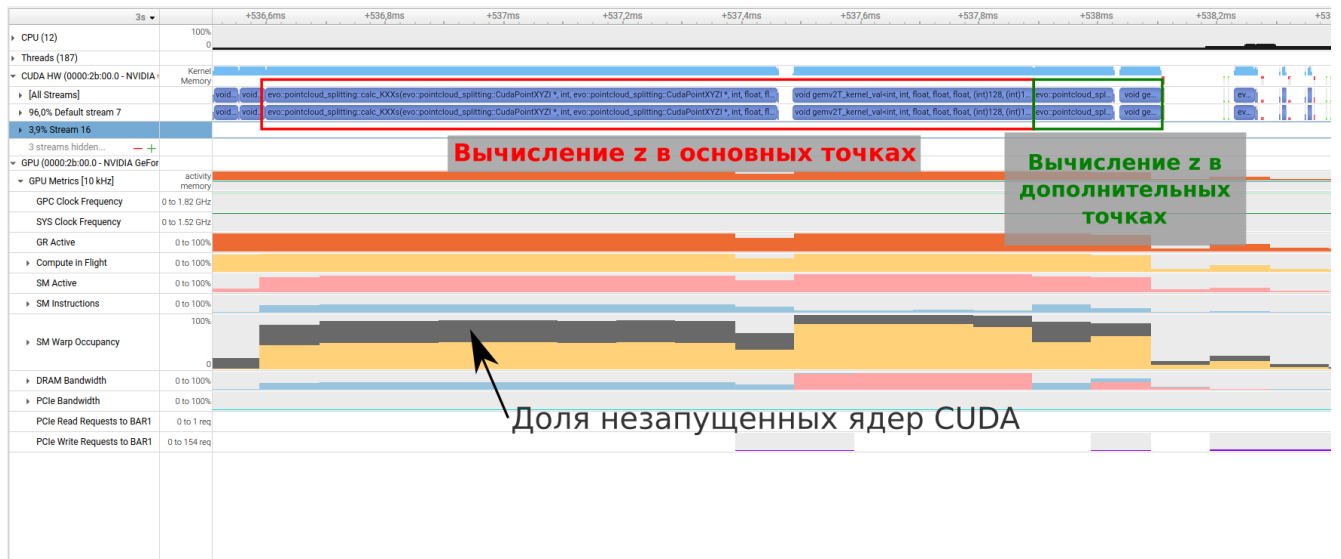
Алгоритм 5: Обобщение алгоритма проецирования проезжей части на вид сверху

2.5 Заключение к главе 2

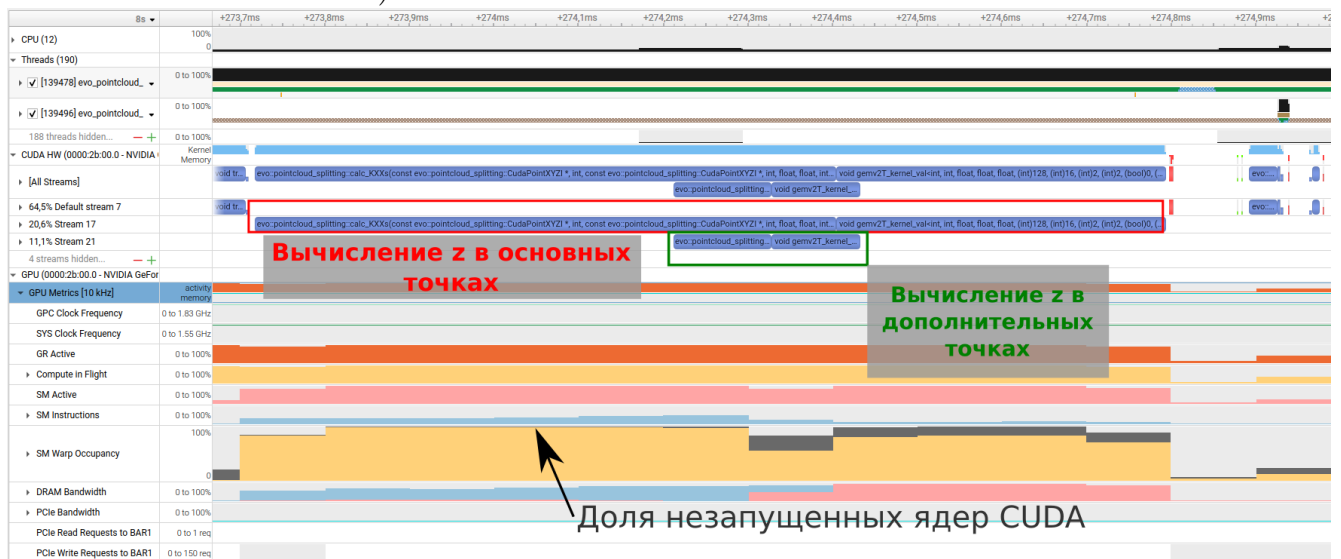
Итак, в данной главе получены следующие основные результаты:

1. Предложен алгоритм проекции визуальных детекций с изображения на плоскость движения БТС с использованием модели L-shape;
2. Проведено сравнение алгоритма L-Shape с альтернативными подходами. Результаты экспериментов демонстрируют, что:
 - на реальных данных L-Shape алгоритм показал наивысший средний коэффициент Жаккара;
 - алгоритм показывает наилучшее качество, если ограничить дальность проецируемых препятствий до 10 метров;
3. Предложен альтернативный алгоритм проекции визуальных детекций с изображения на плоскость движения БТС на основе комплексирования данных с камеры и лазерных дальномеров, установленных на БТС
4. Предложен алгоритм проекции семантической сегментации проезжей части с изображения на плоскость движения БТС на основе комплексирования данных с камеры и лазерных дальномеров, установленных на БТС

Перечисленные основные результаты и другие авторские материалы главы опубликованы в работах[48; 49]



а) Без использования CUDA Streams



б) С исправленной конфигурацией запуска ядер CUDA и использование CUDA Streams

Красной рамкой выделены вычисления аппроксимации прежних тестовых точек, в зелёной – дополнительных точек. Во вкладке «SM Warp Occupancy» жёлтым показана доля вычисляемых ядер относительно максимальной теоретической пропускной способности архитектуры в данный момент времени, серым – доля ядер, которым не хватило ресурсов для вычисления и в результате простаивающих в ожидании их появления.

Рисунок 2.19 — Профилирование в Nsight Systems изменившейся части программы, реализующей алгоритм 2, при добавлении дополнительных точек для аппроксимации.

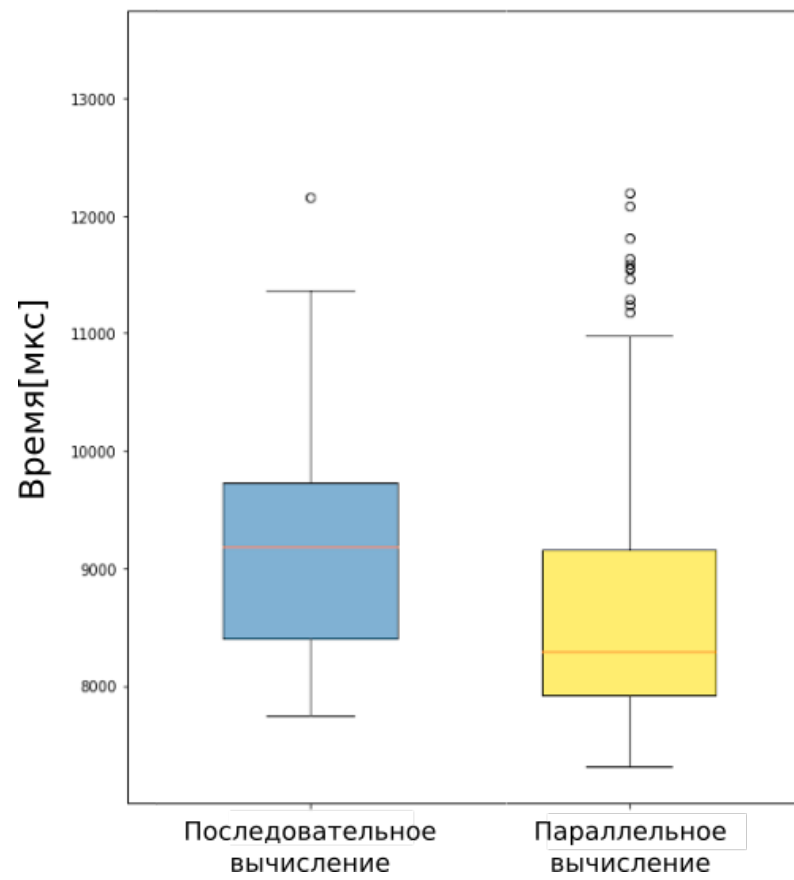
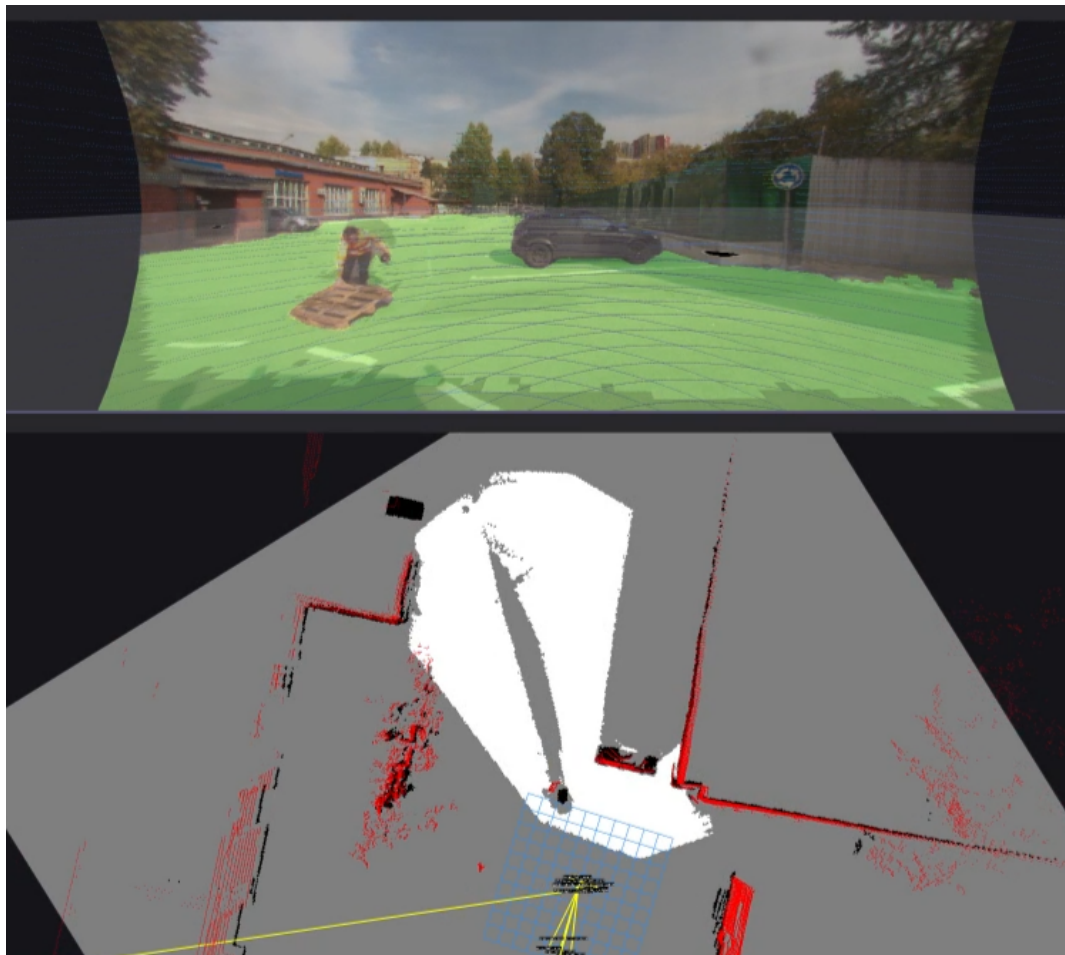
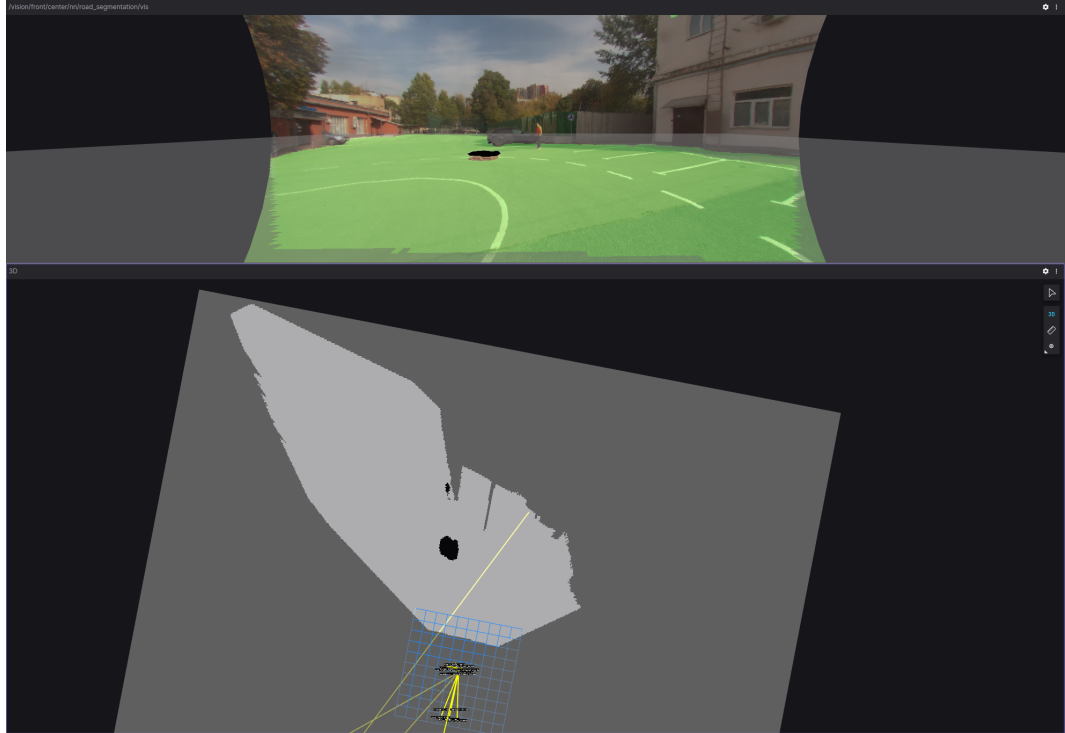


Рисунок 2.20 — График полного времени работы алгоритма 2 с дополнительными тестовыми точками при последовательном и параллельном вычислении.



Вверху изображена семантическая сегментация проезжей части на изображении с камеры БТС. Внизу белым показана проекция данной сегментации на вид сверху алгоритмом. Черным изображены препятствия, полученные не только описанным алгоритмом, но и другими источниками данных о препятствиях (подробнее о них будет рассказано в разделе [3](#))

Рисунок 2.21 — Пример «выреза» слишком большого размера, классификация которого как «занято» была бы некорректна.



Вверху изображена семантическая сегментация проезжей части на изображении. Внизу белым показана проекция данной сегментации на вид сверху алгоритмом

Вверху изображена семантическая сегментация проезжей части на изображении с камеры БТС. Внизу изображён результат проекции сегментации на вид сверху. Цветами обозначен класс элемента сетки: белое – «свободно», чёрное – «занято», серым – «неизвестно»

Рисунок 2.22 — Демонстрация работы алгоритма проецирования сегментации проезжей части на вид сверху.

Глава 3. Программный комплекс построения карты проходимости пространства на основе комплексирования разнородных данных

В рамках работы реализован программный комплекс для сбора разнородных данных с датчиков, установленных на БТС, и их комплексирование в единую модель окружающей БТС сцены – карту проходимости пространства. Данная глава содержит подробное описание разработанного программного комплекса: структуру, функциональные возможности, формат входных и выходных данных.

3.1 Общие сведения

Полное наименование комплекса программ – "Программный комплекс построения карты проходимости в окрестности высокоавтоматизированного беспилотного грузового транспортного средства". Условное обозначение – OssurancуMapFusion. Программный комплекс реализован на языках C++ и CUDA C++. Средством разработки является редактор Microsoft Visual Studio Code.

3.2 Назначение и функциональные возможности

Программный комплекс Ossurancу Map Fusion предназначен для построения карты проходимости пространства – двумерной сетки фиксированного разрешения, элементы которой описывают возможность проезда БТС в пространстве, ограниченном границами данного элемента. Построение производится на основе сырых показаний датчиков, а также признаков извлечённых из них.

Программный комплекс предоставляет следующие функциональные возможности:

1. многопоточное обновление карты проходимости пространства на основе различных данных:

облака точек, принадлежащих препятствиям;

показаний сонара;

комплексирования двухмерных детекций на RGB изображении с камеры и облака точек с лазерного дальномера, принадлежащих препятствиям;

комплексирования семантической сегментации проезжей части на RGB изображении с камеры и облака точек с лазерного дальномера, принадлежащих земле;

трёхмерных детекций препятствий;

2. расширение поддерживаемых источников данных о проходимости окружающего пространства путём самостоятельной генерации пользователем фрагментов карты проходимости пространства;
3. публикация генерируемой карты проходимости пространства в виде ROS(Robot Operatin System) [50] сообщений;
4. опциональное удаление из карты проходимости пространства информации об областях, занимаемых динамическими препятствиями;
5. отключение и подключение отдельных источников информации, учитываемой в карте проходимости пространства, по ходу выполнения программы;

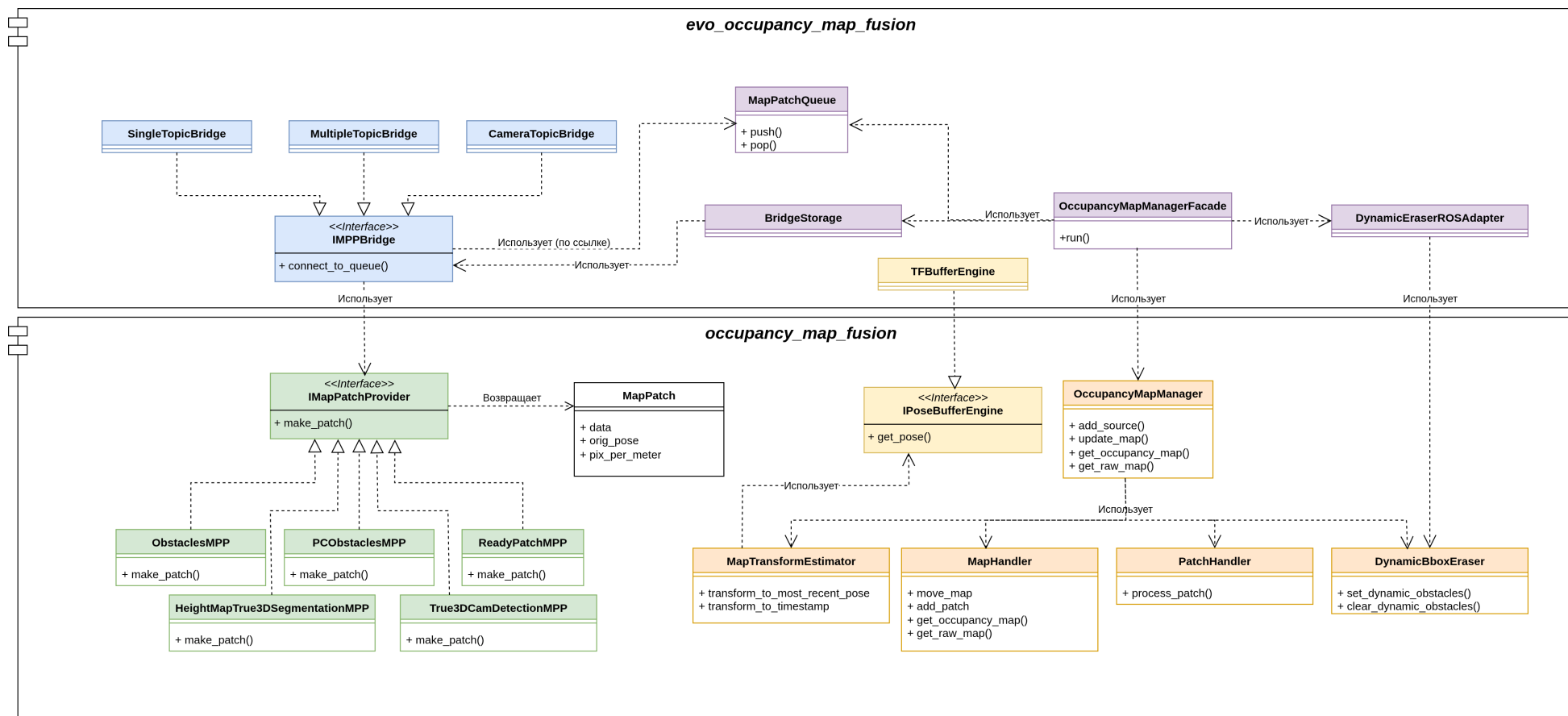
3.3 Структура программного комплекса

На рисунке 3.1 показана диаграмма основных компонентов описываемого программного комплекса. На рисунке 3.2 схематично изображена последовательность шагов при обработке данных приходящих от источников данных (т.е. датчиков либо их предварительных обработчиков, извлекающих полезные признаки). Комплекс состоит из двух крупных компонентов:

1. `ossurancу_map_fusion` – библиотека, реализующая логику работы с картой проходимости пространства: добавление дополнительных источников данных, актуализация положения карты, обработка и учёт входных данных от источников. Компонент реализован на языках C++ и CUDA C++

2. `evo_occupancy_map_fusion` – программа интегрирующая библиотеку `occupancy_map_fusion` в систему ROS: производит соединение с остальными компонентами систем БТС через механизмы ROS-топиков и ROS-сервисов, преобразует входные данные в формат, соответствующий интерфейсам библиотеки, и преобразует генерируемую карту проходимости в ROS сообщение, передаваемое последующим компонентам ПО БТС, в частности компоненту планирования пути БТС. Компонент реализован на языке C++.

Далее в разделе представлено подробное описание структуры данных компонентов, их назначение и форматы входных и выходных данных.



Цветом выделены «идейно связанные» компоненты: базовый класс и его наследники, либо класс и классы хранимых в нём (путём композиции) объектов.

Рисунок 3.1 — Диаграмма компонентов программного комплекса Occupancy Map Fusion на языке UML(Unified Modeling Language).

3.3.1 Библиотека «occupancy_map_fusion»

Библиотека «occupancy_map_fusion» реализует функционал для построения карты проходимости пространства на основе обратной модели наблюдения. Учёт новых данных в карте происходит в два этапа (рисунок 3.2):

1. Построение фрагмента карты проходимости по данным от одного из источников данных.
2. Добавление построенного фрагмента на общую карту проходимости пространства с учётом:

положения системы координат фрагмента относительно системы координат карты,

изменения положения БТС за время между временем зарегистрированных данных и последнего актуального положения карты в глобальной системе координат,

времени прошедшего с момента регистрации данных.

Первый шаг реализуется через интерфейс «IMapPatchProvider» и его наследующих его классов-реализаций. Второй – классом «OccupancyMapManager». Опишем их подробнее.

Интерфейс «IMapPatchProvider»

«IMapPatchProvider» – шаблонный класс, параметризуемый типом входных данных $InputT$. Интерфейс задаёт единственный абстрактный метод `make_patch`, реализующий преобразование входных данных во фрагмент карты проходимости пространства.

Инициализация включает параметры, используемые всеми наследуемыми классами:

1. pix_per_meter – разрешение генерируемого фрагмента,
2. $p(TP) \in (0,1)$ – вероятность класса верно определить наличие препятствия в клетке генерируемого фрагмента,
3. $p(FN) \in (0,1)$ – вероятность класса неверно ошибочно проигнорировать препятствие в клетке генерируемого фрагмента.

Входом метода «make_patch» являются входные данные источника типа *InputT*.

Выходом метода «make_patch» является фрагмент карты проходимости пространства. Фрагментом является сетка фиксированного разрешения *pix_per_meter*. Те элементы сетки, которые согласно реализуемому алгоритму, заняты препятствиям равны логарифму шанса вероятности корректности алгоритма:

$$l_+ = \ln \frac{p(TP)}{1 - p(TP)}.$$

Те элементы сетки, которые согласно реализуемому алгоритму, не заняты какими-либо препятствиями равны логарифму шанса вероятности ошибки алгоритма:

$$l_- = \ln \frac{p(FN)}{1 - p(FN)}.$$

Все остальные значения сетки заполнены 0, что соответствует вероятности занятости пространства 0.5:

$$p(occ) = \frac{\exp(l)}{1 + \exp(l)} \stackrel{l=0}{=} \frac{1}{2}$$

Помимо самой сетки результирующий фрагмент хранит её разрешение, а также положение источника данных относительно системы координат генерируемой сетки. Всё это описание хранится в виде структуры «MapPatch»:

```
struct MapPatch{
    cv::Mat data;
    float ppm;
    cv::Point2f origin;
};
```

Здесь сетка хранится в виде структуры «cv::Mat» из библиотеки Open Computer Vision Library (OpenCV) [51], это позволяет упростить дальнейшие манипуляции с фрагментом при добавлении на общую карту проходимости: такие как поворот и квантование. Такая форма представления может быть полезна и для классов наследников при построении фрагмента, например для добавления на фрагмент геометрических примитивов.

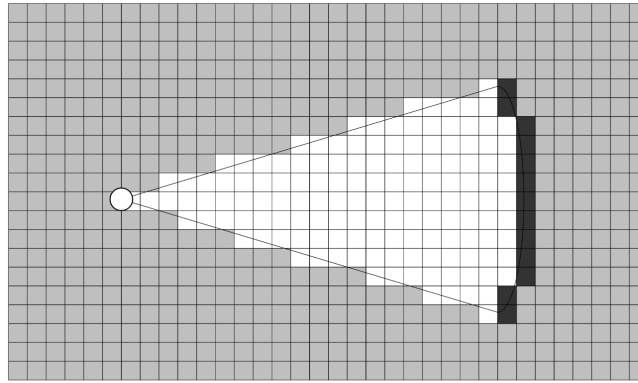


Рисунок 3.3 — Иллюстрация поля зрения, занимаемого препятствием.

Класс «ObstaclesMPP»

Класс реализует интерфейс «IMapPatchProvider». Входным типом для класса является набор полигонов, ограничивающих области, занятые препятствиями. Такие данные могут приходить от сонаров или алгоритмов, представленных в разделе 2.1.

Инициализация класса полностью совпадает с инициализацией «IMapPatchProvider».

Входом метода «make_patch» являются полигоны, ограничивающие области, занимаемыми препятствиями. Координаты вершин полигонов описаны в системе координат источника данных.

```
| using InputT = std::vector<std::vector<cv::Point2f>>;
```

Выходом метода «make_patch» является фрагмент карты проходимости пространства в формате «MapPatch». Области ограниченные входными полигонами заполнены значениями l_+ . Дополнительно метод может (если задать $p(FN) \neq 0.5$) заполнить поле зрения занимаемого входными полигонами значениями l_- (рисунок 3.3). Это повышает качество генерируемой карты проходимости пространства, если наблюдаемые препятствия не могут заслонять друг-друга и следовательно можно судить об отсутствии препятствий в пространстве между источником данных и препятствием.

Класс «True3DCamDetectionMPP»

Класс «True3DCamDetectionMPP» реализует интерфейс «IMapPatchProvider». Входной тип «InputT» – облако точек препятствий с лазерного дальномера, полученного алгоритмом разделения облака точек (раздел 2.3.1) и 2D детекции с RGB камеры БТС.

```
struct BBox2D{
    int min_x = 0;
    int min_y = 0;
    int max_x = 0;
    int max_y = 0;
};
using InputT = std::pair<pcl::PointCloud<PointT>, std::vector<
    BBox2D>>;
```

Здесь облако точек хранится в формате «pcl::PointCloud<PointT>» библиотеки Point Cloud Library (PCL) [52]. Данная библиотека широко распространена и предлагает функционал по вычислительно эффективной обработке облаков точек. Также её преимуществом являются готовые библиотеки, предоставляющие возможность конвертации классов библиотеки PCL (в частности, облаков точек) в ROS сообщения и обратно, например, pcl_conversions [53]. В качестве структуры точки, хранимой в облаке точек «PointT» может выступать любая структура с обязательными полями «x», «y» и «z», которые являются числами с плавающей точкой «float».

Метод «make_patch» реализует алгоритм представленный в разделе 2.3 проекции 2D детекций с камеры БТС в систему координат БТС O_l с использованием комплексирования данных с лазерного дальномера с последующей растеризацией, полученных ограничивающих рамок. В случае отсутствия кластера облака точек препятствий, соответствующего 2D детекции (см. шаг 6 алгоритма 1) метод опционально может спроецировать детекцию в систему координат, используя наивный алгоритм из раздела, чтобы хоть и с меньшей точностью, но учесть объект в генерируемом фрагменте карты проходимости.

Инициализация класса помимо стандартных параметров «IMapPatchProvider» включает следующие параметры:

1. параметры кластеризации облака точек:

min_size, max_size – минимальное и максимальное число точек кластеров (кластеры за пределами этих границ игнорируются);

tolerance – максимальное расстояние Евклида между точками, при котором они считаются точками одного кластера;

distance_clip – максимальное расстояние Евклида от центра системы координат O_l до кластеризуемых точек (точки, расположенные далее удаляются из облака);

z_squeeze – коэффициент сжатия координат вдоль оси Oz_l (ввиду принципа формирования облака точек точки располагаются реже вдоль этой оси чем в плоскости Oxy_l);

2. параметры метода RANSAC для вписывания кластера в ограничивающую рамку:

ransac_tolerance – максимальное расстояние Евклида от линии-гипотезы RANSAC до точек кластера, чтобы считать их соответствующей гипотезе;

iterations_num – число итераций метода RANSAC;

3. настройки наивного алгоритма проекции детекций в случае отсутствия сопоставленного кластера:

enable – булев тип для включения алгоритма;

bev_depth – глубина прямоугольника, строящегося на отрезке после проекции нижней границы препятствия;

Входом метода «make_patch» является пара: облако точек препятствий в формате «pcl::PointCloud» и 2D детекции с камеры в формате «std::vector<BBox2D>».

Выходом метода «make_patch» является фрагмент карты проходимости в формате «MapPatch», на котором области внутри полученных алгоритмами из разделов 2.3 и 2.1.1 (опционально) ограничивающими рамками заполнены l_+ , остальные элементы фрагмента заполнены 0.

Класс «HeightMapTrue3dSegmentationMPP»

Класс «True3DCamDetectionMPP» реализует интерфейс «IMapPatchProvider». Метод «make_patch» реализует алгоритмы из раздела 2.4 проецирования семантической сегментации на вид сверху в окрестности БТС.

Инициализация класса помимо стандартных параметров «IMapPatchProvider» включает следующие параметры:

1. параметры кластеризации облака точек:

height_map_creator_type – параметр задающий способ построения карты высот над плоскостью движения БТС. Доступны варианты:

ortho_projection – использование готовой карты высоты, которая прямоугольно проецируется на плоскость движения БТС;

delaunay – карта высот инициализирована частично, неизвестные значения высоты линейно интерполируются согласно триангуляции Делоне;

use_gpu – булев тип, указывающий использовать ли алгоритм проекции с использованием параллельных вычислений на архитектуре CUDA.

Входом метода «make_patch» является пара: облако точек земли в формате «pcl::PointCloud» и 2D семантическая сегментация проезжей части с камеры в формате одноканального изображения «cv::Mat».

```
| using InputT = std::pair<pcl::PointCloud<PointT>, cv::Mat>;
```

Выходом метода «make_patch» является фрагмент карты проходимости «MapPatch», построенный алгоритмом 5. Области, согласно алгоритму соответствующие свободному для проезда БТС пространству заполнены l_- , занятые препятствиями – l_+ , остальные – 0.

Класс «PCObstaclesMPP»

Класс «PCObstaclesMPP» реализует интерфейс «IMapPatchProvider». Метод «make_patch» производит прямоугольную проекцию облака точек препятствий на плоскость карты проходимости пространства.

Инициализация класса помимо стандартных параметров «IMapPatchProvider» включает следующие параметры:

1. параметры кластеризации облака точек:

min_z, max_z – интервал координаты z точек, попадающих на фрагмент карты проходимости пространства (необходим для фильтрации точек, находящихся выше БТС и не препятствующих его движению);
 $inplane_max_dist$ – максимальное расстояние Евклида в плоскости Oxy от центра координат до точек, попадающих на итоговый фрагмент.

Выходом метода «make_patch» является фрагмент карты проходимости пространства «MapPatch», элементы фрагмента, в которые попали проекции точек инициализированы l_+ , остальные – 0.

Класс «ReadyPatchMPP»

Класс «ReadyPatchMPP» реализует интерфейс «IMapPatchProvider». Класс позволяет расширить количество источников информации, с которыми может взаимодействовать библиотека без изменения содержащегося в ней кода. Это упрощает для пользователей возможность экспериментировать с дополнительными алгоритмами, учитываемыми в карте проходимости пространства БТС.

Инициализация класса полностью совпадает с инициализацией «IMapPatchProvider».

Входом метода «make_patch» является готовый фрагмент карты проходимости пространства «MapPatch». Элементы фрагмента заполнены значениями в интервале $[-1, 1]$, значения в интервале $[-1, 0)$ интерпретируются как области свободные для проезда БТС, в интервале $(0, 1]$ – занятые препятствиями.

```
| using InputT = MapPatch;
```

Выходом метода «make_patch» является фрагмент карты проходимости пространства «MapPatch» того же размера и разрешения, что и входной фрагмент. Значения элементов определяются по следующей формуле:

$$l_{out} = l_{in} \cdot \begin{cases} l_- & , \text{если } l_{in} < 0 \\ 0 & , \text{если } l_{in} = 0 \\ l_+ & , \text{если } l_{in} > 0 \end{cases}$$

где l_{out} – итоговое значение итогового фрагмента, l_{in} – оригинальное значение входного фрагмента.

Класс «DynamicEraser»

В некоторых задачах планирования пути БТС важно различать статические и динамические препятствия (люди, машины, велосипедисты и пр.), окружающие БТС и обрабатывать их разными алгоритмами. Чтобы сделать это возможным в рамках библиотеки «оссурансу_мар_fusion» был создан класс «DynamicEraser», который осуществляет удаление информации об областях, занимаемых динамическими препятствиями.

Инициализация класса осуществляется одним параметром:

bbox_expand_factor – задающий степень увеличения входящих динамических препятствий, чтобы учесть погрешность в определении их положения.

Класс обладает двумя методами:

1. «set_dynamic_obstacles» – сохраняет в объекте класса наблюдаемые на данный момент динамические препятствия в окрестности БТС. Препятствия описываются повернутыми ограничивающими рамками на плоскости карты проходимости пространства:

```
struct DynamicObstacle{
    struct {
        float x;
        float y;
    } center;
    struct {
        float x;
        float y;
    } dims;
    float heading;
};
```

2. «clear_dynamic_obstacles» – осуществляет удаление динамических препятствий из карты проходимости пространства. На вход алгоритму подаётся текущая карта проходимости пространства: для всех областей занимаемыми сохранёнными динамическими препятствиями соответствующие значения элементов карты становятся равными 0.

Класс «OccupancyMapManager»

Класс «OccupancyMapManager» осуществляет хранение и обработку итоговой карты проходимости пространства вокруг БТС на основе фрагментов, генерируемых реализациями «IMapPatchProvider», и информации о перемещении БТС в глобальной системе координат.

Класс имеет следующие основные методы:

1. «add_source». Метод, регистрирующий новый источник информации, учитываемый в карте проходимости пространства. Аргументом функции является положение поля генерируемых источником фрагментов «MapPatch». Положение описывается структурой:

```
struct Pose2f{
    float x;
    float y;
    float heading;
};
```

x, y – положение *origin* генерируемых «MapPatch» в системе координат O_l БТС. *heading* – угол поворота генерируемых фрагментов в той же системе координат.

Метод возвращает уникальный идентификатор источника, по которому в дальнейшей будет производиться преобразование фрагмента в общую систему координат перед тем, как он будет добавлен на карту проходимости пространства.

2. «map_update». Существует две перегрузки данного метода. Первая – без каких либо аргументов. При вызове этой перегрузки, метод находит последнее известное положение БТС в глобальной системе координат и сдвигает карту с учётом положения, в котором в последний раз была

сохранена карта. Вторая перегрузка имеет на вход три параметра: фрагмент карты проходимости пространства «MapPatch», идентификатор источника данных, который сгенерировал данный фрагмент, и время которому этот фрагмент соответствует. При вызове этой перегрузки метод корректирует положение карты проходимости (если время регистрации фрагмента позднее времени последнего сохранения карты), осуществляет сдвиг и поворот фрагмента, чтобы сопоставить его с картой проходимости пространства и добавляет его путём суммирования элементов фрагмента с элементами карты проходимости (см. «Поток N+1» на рисунке 3.2). Дополнительно после этого шага может быть произведено удаление динамических препятствий с карты проходимости. Также в обеих перегрузках данного метода происходит экспоненциальное затухание элементов карты проходимости или фрагмента, а также их отсечение, чтобы уменьшить количество артефактов, полученных в результате ошибок источников информации. Подробнее об этих двух операциях будет рассказано в главе 4.

3. **«get_raw_map»**. Метод не принимает никаких параметров и возвращает последнюю сохранённую карту проходимости пространства без квантования значений элементов карты.
4. **«get_occupancy_map»**. Метод не принимает никаких параметров и возвращает последнюю сохранённую карту проходимости пространства, полученную после квантования:

$$l_{quant} = \begin{cases} -1 & , \text{если } l_{raw} < -l_{thresh} \\ 1 & , \text{если } l_{raw} > l_{thresh} \\ 0 & , \text{в остальных случаях} \end{cases},$$

где l_{quant} итоговое значение элемента после квантования, l_{raw} – оригинальное значение элемента, $l_{thresh} > 0$ – порог квантования, задаваемый параметром класса.

Для реализации описанного функционала класс хранит путём композиции несколько специализированных классов:

1. **«MapTransformEstimator»**. Класс отвечает за сбор, хранение и вычисление положений БТС в глобальной системе координат в течение времени работы класса. Часть функционала класса делегирована

интерфейсу «IPoseBufferEngine» для реализации принципа инверсии зависимостей [54], поскольку в системе ROS уже существует механизм для оценки относительных перемещений систем координат, используемых в описании движения БТС [55].

2. **«MapHandler»**. Класс отвечает за хранение, модификацию и извлечение карты проходимости пространства. К модификациям карты относятся: сдвиг, экспоненциальное затухание, клиппинг и добавление фрагментов приведённых в систему координат карты.
3. **«PatchHandler»**. Класс отвечает за преобразование фрагментов карты (сдвиг, поворот, экспоненциальное затухание фрагментов) в систему координат карты.
4. **«DynamicBboxEraser»**. Опциональный класс реализующий удаление информации об участках, занятых динамическими препятствиями, как описано в предыдущем разделе.

3.3.2 Программа «evo_occupancy_map_fusion»

Программа «evo_occupancy_map_fusion» осуществляет интеграцию функционала в систему ROS, предоставляя классы осуществляющие преобразование ROS-сообщений в классы и структуры, используемые библиотекой «occupancy_map_fusion» (тем самым реализуя структурный шаблон проектирования «Адаптер» [56]). Программа включает следующие основные классы:

1. **«IMPPBridge»**. Класс «Адаптер» класса «IMapPatchProvider». Реализации этого класса в отдельном программном потоке подписывается на один («SingleTopicBridge») или несколько («MultipleTopicBridge») ROS-топиков, в которые поступают сообщения на обработку программой, преобразует их в формат библиотеки «occupancy_map_fusion» и передаёт адаптируемому объекту класса «IMapPatchProvider», к сформированному фрагменту добавляется метainформация: идентификатор источника данных и время регистрации отражаемых им данных, после чего структура добавляется в очередь «MapPatchQueue».

2. **«MapPatchQueue»**. Класс реализующий очередь, гарантирующий потоковую безопасность при добавлении и извлечении фрагментов с метайнформацией.
3. **«DynamicEraserROSAdapter»**. Класс аналогично «IMPPBridge», подписывающийся на сообщения с динамическими препятствиями, конвертирует их в «std::vector<DynamicObstacle>» и передающий результат в хранимый внутри класса «DynamicEraser».
4. **TFBufferEngine**. Класс реализует функционал необходимый для работы «MapTransformEstimator» адаптируемой библиотеки.
5. **OccupancyMapManagerFacade** Класс исполняющий основной цикл обновления и публикации карты проходимости пространства вокруг БТС («Поток N+1» на рис. 3.2). Метод «run» в цикле пытается извлекать новые фрагменты из очереди «MapPatchQueue», при успехе фрагмент и метайнформация передаются в метод «update_map» хранимого в классе объекта «OccupancyMapManager», после чего, если класс «DynamicEraserROSAdapter» инициализирован, происходит удаление областей занимаемыми динамическими препятствиями. Далее если с момента последней публикации карты проходимости пространства прошло больше заданного в параметрах класса времени происходит извлечение карты проходимости через метод «get_occupancy_grid» класса «OccupancyMapManager», который затем преобразуется в ROS-сообщение типа «nav_msgs/OccupancyGridMap» и публикуется в отдельный топик для сообщений данного типа.

3.4 Заключение к главе 3

В данной главе разработан программный комплекс построения карты проходимости пространства на основе комплексирования разнородных данных от датчиков, установленных на БТС, и их обработчиков. Комплекс реализует алгоритмы описанные в главе 2, а также позволяет пользователям интегрировать собственные алгоритмы, построения фрагментов карты проходимости пространства. Для представленного в программного комплекса были получены свидетельства о регистрации программ для ЭВМ:

1. №2025617212 Программное обеспечение «Система восприятия N1 V3».
2. №2025669744 Программное обеспечение «Способ построения карты проходимости в окрестности высокоавтоматизированного беспилотного грузового транспортного средства».

Глава 4. Неблокирующий алгоритм построения карты проходимости пространства на основе комплексирования разнородных данных

4.1 Introduction

1. Преобразование входных данных к единому формату представления данных о проходимости пространства. Как правило, это фрагмент карты проходимости, содержащий вероятности занятости клеток, основываясь на последних показаниях данного источника данных.
2. Комплексирование полученных фрагментов с ранее построенной картой проходимости пространства. Для обратной модели измерения этот шаг включает в простом суммировании полученного фрагмента с соответствующей областью карты.

Вычисление первого шага может быть выполнено с помощью асинхронного программирования, так как для построения фрагмента используются данные только одного источника данных, и синхронизация с другими источниками/подсистемами не требуется. Этот шаг может быть «дорогим» с точки зрения времени выполнения в зависимости от сложности используемого алгоритма, но мы считаем, что уменьшение этого времени труднореализуемо, так как требует индивидуального анализа и подхода к оптимизации для каждого уникального источника данных. В то же время второй шаг, как правило, является блокирующим: созданный фрагмент добавляется в очередь и включается в карту только после того, как были добавлены все фрагменты, стоящие раньше в очереди. Таким образом происходит задержка, даже в случае добавляемые фрагменты не имеют пересечений и параллельное обновление теоретически возможно.

Целью данной работы является предложить асинхронный алгоритм построения карты проходимости пространства, который позволяет снизить итоговое время задержки обновления карты.

Основной вклад данной работы заключается в следующем:

1. Предложены три последовательные модификации базового алгоритма картирования на основе Байесовского вывода с использованием обратной модели измерения для случая одномерного пространства.

2. Проведено сравнение вычислительной сложности базовой реализации и модификаций для всех выполняемых операций над картой проходимости пространства (и их комбинации) при различных размерах картируемой области. Доказано, что применение всех трёх модификаций имеет лучшую вычислительную сложность, чем базовая реализация.

4.2 Постановка задачи

Пусть задана фиксированная относительно Земли система координат, расположенная таким образом, что движение робота происходит в плоскости $z = 0$, положение робота в этой системе координат задаётся тремя параметрами: $\mathbf{x} = x, y, \theta$, где x, y координаты фиксированной точки робота (см. рис 4.1), θ – угол поворота робота от Ox вокруг вертикальной оси. Введём также локальную систему координат робота, начало которой находится в той же фиксированной точке x, y , ось Ox направлена вдоль оси движения робота. Координаты в этой системе координат, мы будем обозначать x_l, y_l . Наконец, пусть у нас есть S источников данных. Каждый источник имеет собственную систему координат, положение которой известно и фиксировано относительно локальной системы координат робота. Источник s независимо от остальных источников генерирует информацию о занятом и/или свободном пространстве в окрестности робота в виде наборов $(P_{s,k}, \mathbf{x}_{s,k}, t_{s,k})$, $s = \overline{1, S}$, $k \in \mathbb{N}$, где $P_{s,k}$ – фрагмент карты проходимости в виде двумерного массива логарифмов шансов занятости пространства с фиксированным разрешением ppm_s (pixel per meter, пикселей на метр), оси массива ориентированы вдоль осей системы координат источника s ; $\mathbf{x}_{s,k}$ – положение начала системы координат датчика относительно левого верхнего угла массива; $t_{s,k}$ – время регистрации сгенерированного фрагмента, k – порядковый индекс сгенерированного набора.

На основе получаемых данных необходимо сформировать локальную карту проходимости пространства в виде набора $(M_i, \mathbf{x}_i, t_i)$; $i \in \mathbb{N}$, где M_i – карта проходимости пространства в момент времени t_i в виде двумерного массива логарифмов шансов занятости пространства с фиксированным разрешением ppm ; $\mathbf{x}_i$ – координаты левого верхнего угла карты, заданные таким образом, чтобы начало локальной системы координат в момент времени t_i совпадало с центром

карты M_i , оси карты параллельны осям глобальной системы координат. Такая ориентация выбрана, чтобы не вносить погрешность в оценку положения свободного/занятого пространства в результате многократных поворотов на малую величину двумерного массива, которая возникала бы при ориентации карты параллельно осям локальной системы координат.

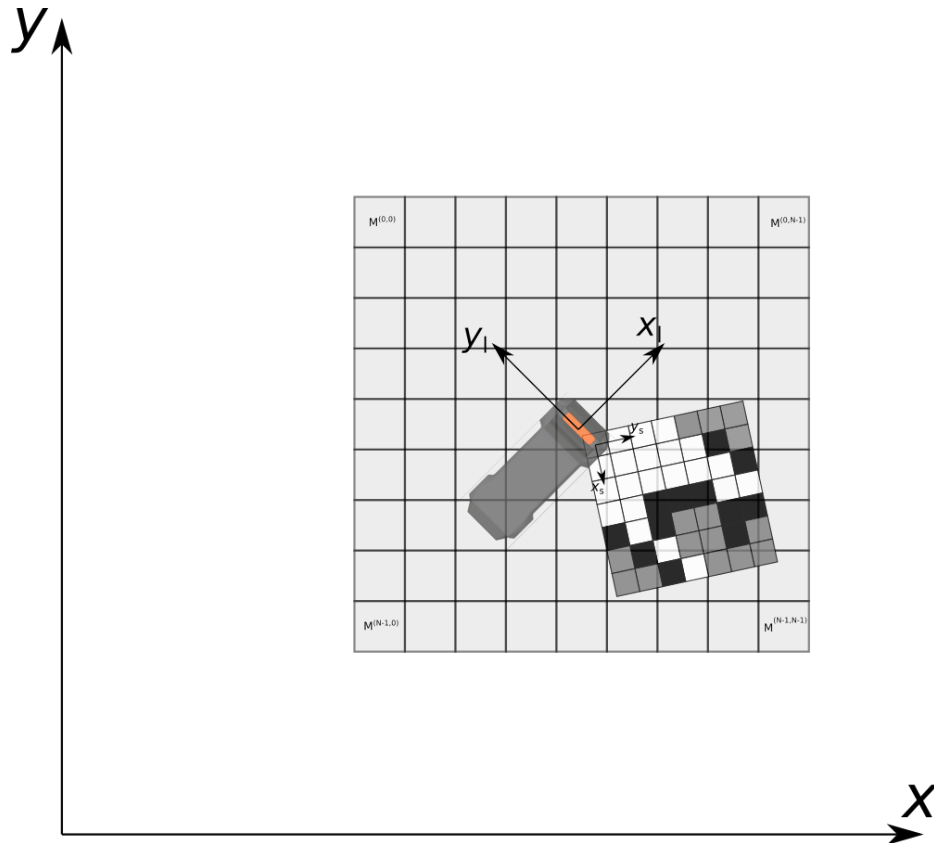


Рисунок 4.1 — Расположение локальной карты проходимости и её фрагмента от одного из источников данных относительно робота и глобальной системы координат

Для построения локальной карты проходимости пространства все генерируемые фрагменты P_s карты проходимости пространства добавляются в очередь в порядке их готовности. Заметим, что время регистрации добавляемых таким образом фрагментов в общем случае не является монотонным (см. рис 4.2). Далее локальная карта проходимости последовательно обновляется добавлением фрагментов из очереди. Алгоритм одной итерации обновления представлена листинге 6: если $t_{s,k} > t_i$, карта актуализируется к моменту времени фрагмента (строки 3 – 6): вычисляются новые координаты левого верхнего угла карты, время карты заменяется временем регистрации извлечённого фрагмента, содержимое карты сдвигается и «затухает» (механизм и

цель «затухания» мы опишем далее в этом разделе). В противном случае значения x_i, t_i остаются неизменными: x_{i-1}, t_{i-1} (строки 8 – 9). Далее извлечённый фрагмент преобразуется: «доворачивается» до ориентации локальной карты проходимости пространства и масштабируется до разрешения локальной карты проходимости, создавая временное представление $P_{aligned}$. Также вычисляется сдвиг левого верхнего угла фрагмента относительно левого верхнего угла локальной карты проходимости пространства $patch_shift$ (строка 12). Если время регистрации фрагмента меньше времени карты, к фрагменту применяется «затухание» (строка 13). Наконец фрагмент добавляется к соответствующей ему области на локальной карте проходимости пространства (строка 14).

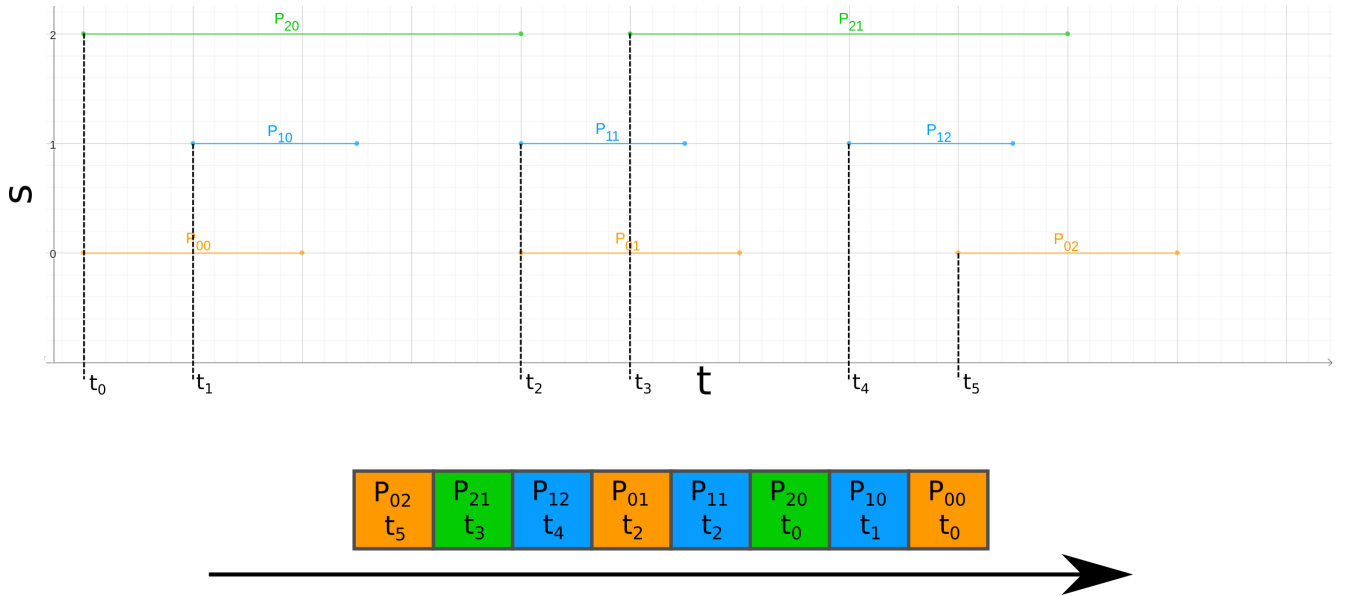


Рисунок 4.2 — Демонстрация формирования очереди фрагментов. Горизонтальные отрезки обозначают время формирования фрагмента каждым источником: начало отрезка соответствует времени регистрации данных $t_{s,k}$, конец отрезка соответствует времени завершения формирования фрагмента и момент добавления его в очередь фрагментов. Внизу иллюстрации продемонстрирован итоговый порядок фрагментов в очереди: фрагменты извлекаются из очереди в порядке справа налево

Недостатком обратной модели является предположение о статичности сцены: в случае наличия динамических объектов в сцене, такие объекты оставляют за собой «след», который не удаляется с локальной карты проходимости пространства, если нет источника данных способного отмечать области в которых препятствия отсутствуют. Даже при наличии такого источника, «след» может попасть в его слепую зону и остаться на карте. Для решения данной проблемы,

```

1  $P_{s,k}, x_{s,k}, t_{s,k} \leftarrow \text{pop PatchQueue};$ 
2 if  $t_{s,k} > t_{i-1}$  then
3    $x_i \leftarrow \text{get\_map\_pose}(t_{s,k});$ 
4    $t_i \leftarrow t_{s,k};$ 
5    $M_{actual} \leftarrow \text{shift\_map}(M_{i-1}, x_i - x_{i-1});$ 
6    $M_{actual} \leftarrow \text{dim}(M_{actual}, t_i - t_{i-1});$ 
7 else
8    $x_i \leftarrow x_{i-1};$ 
9    $t_i \leftarrow t_{i-1};$ 
10   $M_{actual} \leftarrow M_{i-1};$ 
11 end
12  $P_{aligned}, \text{patch\_shift} \leftarrow \text{align\_patch}(P_{s,k}, t_{s,k}, t_i, \text{ppm}/\text{ppm}_s);$ 
13  $P_{aligned} \leftarrow \text{dim}(P_{aligned}, t_i - t_{s,k});$ 
14  $M_i \leftarrow \text{add\_patch}(M_{actual}, P_{aligned}, \text{patch\_shift});$ 

```

Алгоритм 6: update_map

мы вводим механизм «затухания» – показания каждой клетки экспоненциально затухают, возвращаясь в состояние "неизвестно" без повторных обновлений от источников данных (см. листинг 7)

```

1  $a \leftarrow \text{some float-number hyper-param};$ 
2  $\widehat{M}_i \leftarrow \text{2D array of size: rows}(\widehat{M}_{i-1}) \times \text{cols}(\widehat{M}_{i-1});$ 
3 for  $j = 0$  to  $\text{rows}(M)$  do
4   for  $k = 0$  to  $\text{cols}(M)$  do
5      $\widehat{M}_i^{j,k} \leftarrow \exp(-a\Delta t) \widehat{M}_{i-1}^{j,k};$ 
6   end
7 end
8 return  $\widehat{M}_i$ 

```

Алгоритм 7: $\text{dim}(\widehat{M}_{i-1}, \Delta t)$

Также на шаге добавления фрагмента на карту мы вводим механизм против "перезарядки" клеток карты: если динамическое препятствие долго стояло неподвижно, а потом сдвинулось, то логарифм шансов освободившегося пространства останется отрицательным, и даже с механизмом затухания будут

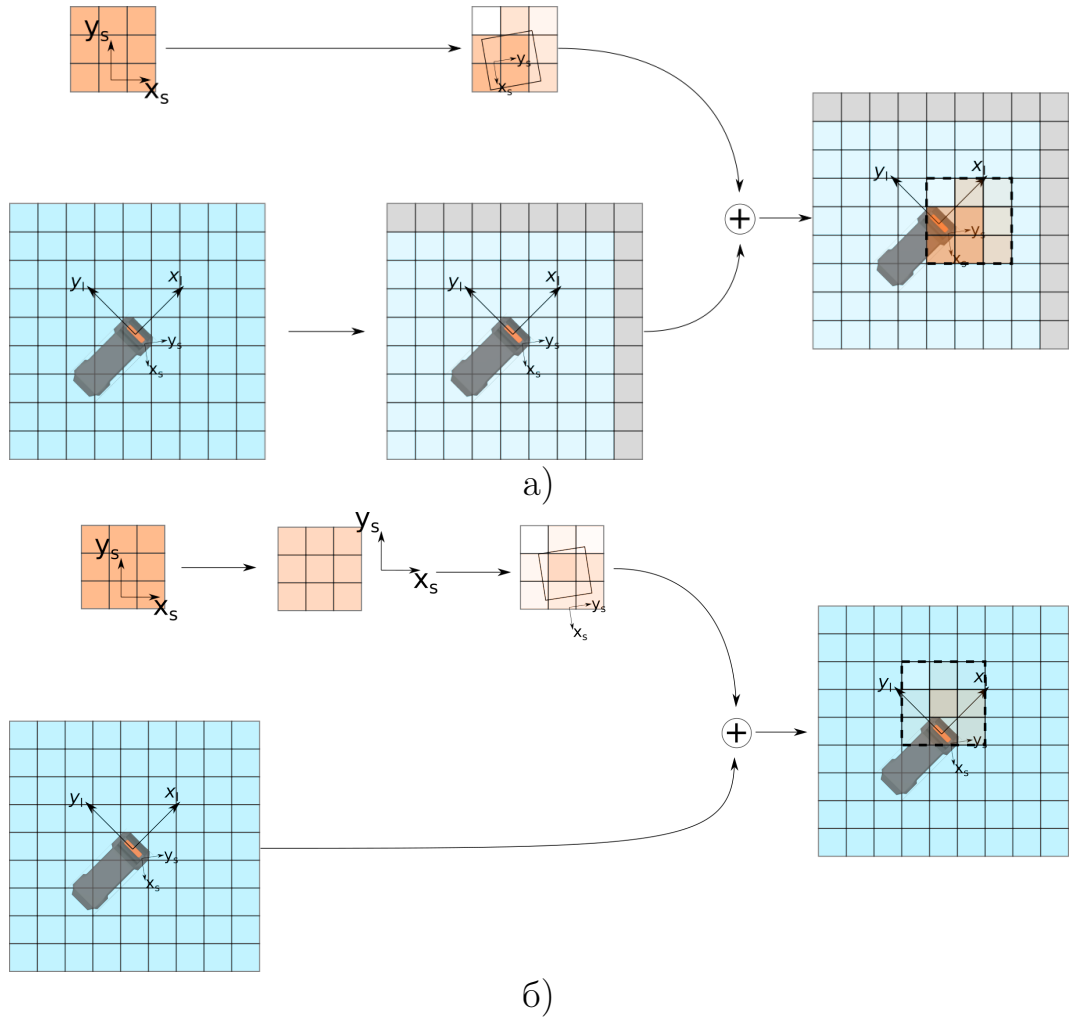


Рисунок 4.3 — Иллюстрация алгоритма обновления локальной карты проходимости пространства фрагментом от источника данных s . 4.3а: время регистрации фрагмента новее последнего времени обновления карты. 4.3б: Сценарий, когда время регистрации фрагмента старше последнего времени обновления карты

долго оставаться в состоянии «занято». Чтобы этого избежать, мы ограничиваем абсолютную величину максимального значения, которое может принимать клетка:

$$l_t = \max(\min[\text{inv_mes_model}(t), l_{thresh}], -l_{thresh}),$$

где $\text{inv_mes_model}(t)$ – значение логарифма шансов, полученное по формуле 1.2, l_{thresh} – пороговое значение логарифма шансов по абсолютному значению.

4.3 Модификации базовой реализации

В данном разделе мы предлагаем модификации базовой реализации для снижения задержки между появлением информации о проходимости у одного из источников данных и её добавлением на итоговую карту проходимости пространства. Для простоты восприятия, мы упростим задачу и будем считать пространство одномерным: карта в таком случае представляет собой одномерный массив M размера N , а движение возможно только влево или вправо. Также для простоты мы будем считать размер одной клетки равным 1 метру, чтобы опустить операции с учётом разрешения карты ppm . В прошлом разделе мы показали, что карта должна поддерживать операции сдвига и обновления. Помимо этого мы отдельно добавим к описанию реализаций операцию получения карты, поскольку в модификациях данная операция перестаёт быть тривиальным копированием массива и требует отдельного обсуждения.

4.3.1 Base OGM

Для начала опишем исходную реализацию в одномерном случае. Карта представляет собой массив M размера N , текущую координаты центра x_i и время t_i , которому текущая карта соответствует (рис. 4.4).

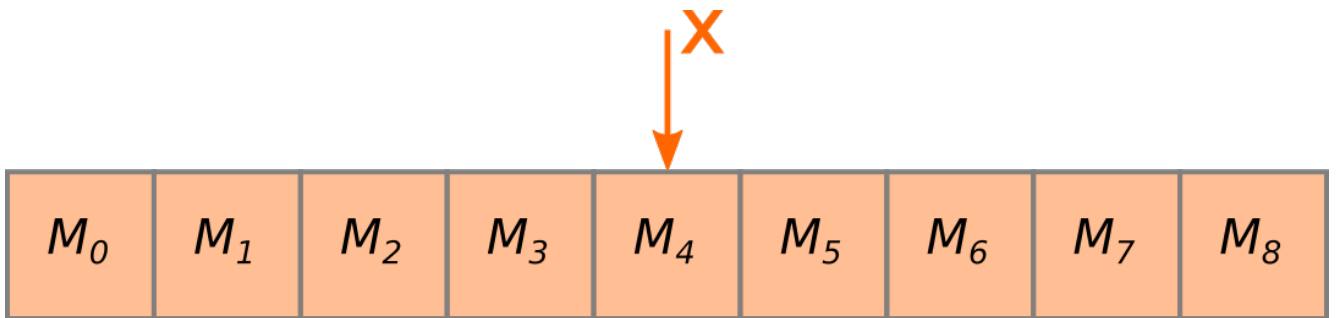


Рисунок 4.4 — Оригинальная реализация локальной карты проходимости пространства

Сдвиг

При сдвиге в новую точку x_{i+1} , в момент времени t_{i+1} массив со значениями логарифма шанса проходимости сдвигается на $\lfloor x_{i+1} - x_i \rfloor$ клеток. Новые клетки инициализируются 0. Ниже приведён код данной операции. Вычислительная сложность такой операции $\Theta(N)$. Итоговая реализация алгоритма представлена в листинге 8.

```

1  $\Delta x \leftarrow \text{round}(x_{i+1} - x_i);$ 
2  $j \leftarrow 0;$ 
3 for  $j = 0$  to  $N$  do
4   if  $j + \Delta x \in [0; N)$  then
5      $M[j] \leftarrow M[j + \Delta x];$ 
6   else
7      $M[j] \leftarrow 0;$ 
8   end
9 end
10 return  $M$ 

```

Алгоритм 8: *shift_map*(M, x_{i+1}, t_{i+1}) (base 1D case)

Обновление

Процесс обновления в одномерном случае во многом повторяет алгоритм, представленный в 4.2. Отметим только, что в данном случае «доворачивание» фрагмента не требуется, и необходимо только учесть сдвиг карты за промежуток времени между моментом регистрации фрагмента и итоговым временем регистрации карты после обновления. Дополнительно введём обозначение размера фрагмента P : N_P . Итоговая реализация алгоритма представлена в листинге 9.

Первая часть алгоритма (до цикла добавления фрагмента на карту) имеет вычислительную сложность $\Theta(1)$, если $t_{patch} \leq t_M$, и, как показано в преды-

```

1  $x_{t_{s,k}} \leftarrow$  local occupancy map pose at  $t_{s,k}$ ;
2 if  $t_{s,k} > t_i$  then
3    $M \leftarrow shift\_map(M, x_{t_{s,k}}, t_{s,k})$ ;
4    $M \leftarrow dim(M, t_{s,k} - t_i)$ ;
5 else
6    $x_{s,k} \leftarrow x_{s,k} - (x_{t_i} - x_{t_{s,k}})$ ;
7    $P_s \leftarrow dim(P_s, t_i - t_{s,k})$ ;
8 end
9  $\Delta x_s \leftarrow \text{round}(x_{s,k} - x_i)$ ;
10 for  $j \leftarrow \max(0, \Delta x_s)$  to  $\min(N_P; N + \Delta x_s)$  do
11    $M[j - \Delta x_s] \leftarrow M[j - \Delta x_s] + P_s[j]$ ;
12 end

```

Алгоритм 9: $update_map(P_s, t_{s,k}, x_{s,k})$ (base 1D case)

дущем пункте, $\Theta(N)$ (операция затухания карты имеет ту же вычислительную сложность) в противном случае. Если патч целиком помещается во второй части алгоритма будет выполнено $\Theta(N_P)$ операций. Если же фрагмент больше карты и полностью её покрывает будет выполнено $\Theta(N)$ операций. В то же время вторая часть алгоритма может не выполняться вовсе в случае если фрагмент не пересекается с картой. Таким образом вычислительная сложность второй части алгоритма: $O(\min(N_P, N))$, а суммарная вычислительная сложность всего алгоритма: $O(N + \min(N_P, N))$.

Получение карты

Для получения карты необходимо просто вернуть копию текущего массива. Сложность операции: $\Theta(N)$. Копия требуется для того, чтобы не блокировать последующие операции обновления и сдвига. В силу тривиальности алгоритма мы не приводим здесь его реализацию.

4.3.2 OGM with prefetched areas

Первая модификация локальной карты проходимости пространства направлена на снижение стоимости сдвига. Для этого массив для хранения карты увеличивается до размера ($\hat{N} > N$), чтобы описывать область больше, чем отображаемая карта.

В такой реализации к уже упомянутому массиву и координате x добавляется координата, соответствующая некоторой фиксированной точке "полного" массива $\hat{x}$. Возьмём в качестве такой точки центр массива (Рис. 4.5).

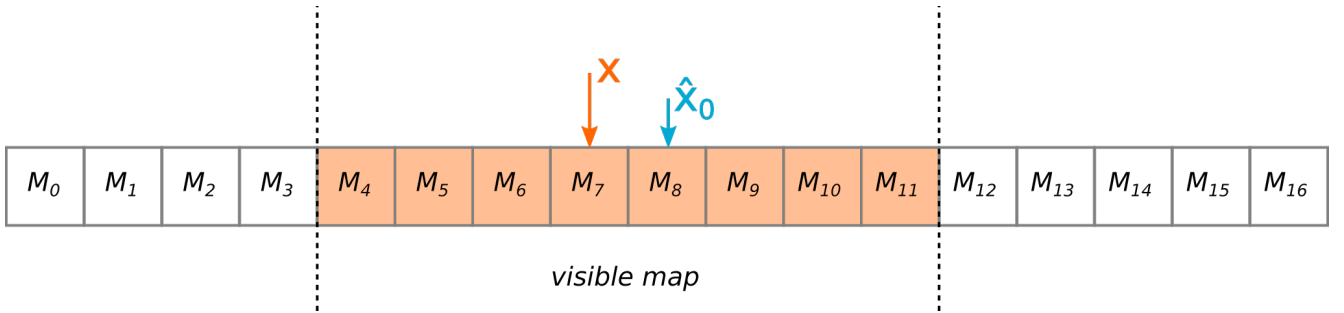


Рисунок 4.5 — Карта с подготовленными окрестностями

Сдвиг

После описанной модификации для сдвига, в общем случае будет достаточно только обновить значение x , соответственно эта операция требует константное время для исполнения $\Theta(1)$. Если же при сдвиге "отображаемая" часть карты выходит за границы полного массива, его содержимое сдвигается так, чтобы новый $\hat{x}$ совпал с актуальным x . В этом случае сложность, как и в оригинальной реализации, $\Theta(N)$. Как итог, при правильно выбранном размере массива, амортизационная сложность операции: $\Theta(1)$.

```

1 if  $(\hat{x} - x_{i+1}) \in [N/2; \hat{N} - N/2)$  then
2   |   return M;
3 end
4 ...
5 // Previous map_shift implementation but for whole prefetched
   map
6 ...

```

Алгоритм 10: *shift_map*(M, x_{i+1}, t_{i+1}) (prefetched)

Обновление

Операция обновления карты в точности повторяет обновление из оригинальной реализации. Сложность алгоритма: $O(\min(N_P, N))$, если не происходит затухания карты и $O(\hat{N} + \min(N_P, N))$ в противном случае (мы должны учитывать клетки за границами видимой карты, т.к. они могли быть ранее обновлены и вернуться в область видимости в будущем).

Получение карты

Операция получения карты заключается в копировании подмассива, соответствующего видимой части карты. Как и прежде, сложность операции — $O(N)$. Все операции по-прежнему остаются блокирующими.

4.3.3 OGM with lockfree cell value update

Блокировки обновления карты становятся узким местом алгоритма при увеличении числа источников данных, используемых для заполнения карты. Каждый источник вынужден ждать своей очереди для доступа к карте, даже если добавляемые фрагменты не пересекаются между собой. Чтобы снизить

задержку в таком случае, мы предлагаем вторую модификацию локальной карты проходимости пространства, изменяющую структуру самой клетки карты. Ранее клетка содержала одно число с плавающей точкой. Теперь это будет атомарная пара чисел: значение логарифма шансов занятости клетки и время, которому оно соответствует (таким образом, мы более не требуем, чтобы все клетки массива соответствовали одному моменту времени). Заметим, что при разработке атомарной структуры данных (т.е. структуры данных, не использующей примитивы блокировки: мьютексы, семафоры, сигналы и т.п. при чтении/записи) важно учитывать её размер и функциональные возможности аппаратного обеспечения. В нашем случае структура состоит из двух 32-битных полей и имеет итоговый размер – 64 бита. Большинство современных архитектур имеют инструкции, обеспечивающие атомарные операции с такими структурами данных.

Сдвиг

Операция сдвига не претерпела значительных изменений в сравнении с предыдущей реализацией, с тем уточнением что при копировании клеток во время «полноценного» сдвига копируется не только значение логарифма шансов, но и время, а также в данной реализации необходимо инициализации времени клетки (мы используем значение 0, т.к. оно не влияет на последующие операции обновления клетки). Сложность операции остаётся амортизированной $\Theta(1)$ и в худшем случае имеет сложность $\Theta(\hat{N})$. Заметим однако, что реальная константа сложности увеличится из-за использования атомарных структур, копирование и инициализации, которых значительно продолжительнее своих неатомарных аналогов.

Обновление

При обновлении карты многие шаги из предыдущего раздела повторяются. Сперва мы определяем необходим ли сдвиг карты. Если нет, обновление

может быть выполнено одновременно с другими такими же операциями обновления, не требующими сдвига. В противном случае операция ждёт возможности взять уникальные права на модификацию карты и произвести требуемый сдвиг.

Далее мы корректируем координаты, куда требуется добавить фрагмент карты проходимости пространства. Для каждой клетки мы атомарно считываем её значения, аналогично предыдущим версиям алгоритма пересчитываем новое значение клетки с учётом «затухания» и пытаемся атомарно записать в клетку, если её значение не изменилось за то время, что мы вычисляли новое значение клетки (сравнение с обменом, Compare and Swap, CAS). Если клетка поменяла своё значение, мы пересчитываем новое значение клетки с учётом обновившегося состояния обновляемой клетки до тех пор, пока не произведём успешную запись. Листинг описанного алгоритма приведён в листинге ??

Теоретически данная операция может выполняться неограниченно долго, если на между попытками сравнения с обменом происходит обновление клетки в другой операции обновления. На практике незавершаемость операции практически невозможна ввиду в первую очередь ограниченного числа потоков, работающих одновременно (в нашем случае их 12, на современных вычислителях используемых в робототехнике их число варьируется от 8 до 24). Если пренебречь единичными пересчётами обновляемых значений, сложность такой операции становится $O(\min(N_P, N))$ ($O(\hat{N} + \min(N_P, N))$), если во время выполнения операции произошёл «полноценный» сдвиг карты). Однако константа в этом случае будет заметно больше, чем в предыдущих вариантах, из-за использования а) атомарных операций, которые в несколько раз вычислительно дороже, неатомарных аналогов и б) `shared_mutex`, который имеет значительно большее время блокировки в сравнении с обычным `mutex`-ом. Поэтому эта реализация становится выгоднее предыдущих вариантов при условии достаточного количества источников производящих одновременное обновление карты.

Получение карты

При получении карты, копируются все клетки видимой части карты, чтобы избежать состояния гонки с другими операциями. В то же время находится самое позднее значение времени в клетках t_{newest} . После чего создаётся кар-

```

1 // Enter section with allowed simultaneous update
2 shared_lock(map_access_mutex);
3  $x_{t_{s,k}} \leftarrow$  local occupancy map pose at  $t_{s,k}$ ;
4 if (Shift is required) then
5     // Critical section start
6     unique_lock(map_access_mutex);
7     if Shift is still required and was not performed by other update then
8          $M \leftarrow \text{shift\_map}(M, x_{t_{s,k}}, t_{s,k})$ 
9     else
10         $x_{s,k} \leftarrow x_{s,k} - (x_{t_i} - x_{t_{s,k}})$ ;
11    end
12    // Critical section end
13 else
14     $x_{s,k} \leftarrow x_{s,k} - (x_{t_i} - x_{t_{s,k}})$ ;
15 end
16  $\Delta x_s \leftarrow \text{round}(x_{s,k} - x_i)$ ;
17 for  $j \leftarrow \max(0, \Delta x_s)$  to  $\min(N_P; N + \Delta x_s)$  do
18      $t_{prev} \leftarrow 0$ ;
19      $val_{prev} \leftarrow 0$ ;
20      $t_{new} \leftarrow t_{s,k}$ ;
21      $val_{new} \leftarrow P_s[j]$ ;
22     while  $\text{CAS}(M[j - \Delta x_s], \{val_{prev}, t_{prev}\}, \{val_{new}, t_{new}\})$  failed do
23         // Compare and swap values until success.  $\{val_{prev}, t_{prev}\}$ 
           are actualized in case CAS fails
24         if  $t_{prev} < t_{s,k}$  then
25              $t_{new} \leftarrow t_{s,k}$ ;
26              $val_{new} \leftarrow val_{prev} \exp(-a(t_{s,k} - t_{prev})) + P_s[j]$ ;
27         else
28              $t_{new} \leftarrow t_{prev}$ ;
29              $val_{new} \leftarrow val_{prev} + P_s[j] \exp(-a(t_{prev} - t_{s,k}))$ ;
30         end
31     end
32 end

```

та «старого» формата – массив чисел с плавающей точкой, куда записываются значения скопированных клеток, приведённых к моменту времени t_{newest} . Сложность такой операции $O(N)$, однако как и в случае с операцией обновления, константа, скрываемая за O , превышает старое значение. Заметим, что эта операция может выполняться одновременно с операциями обновления локальной карты проходимости пространства, если допускается частично обновлённое состояние карты. Рассмотрим, например, такой сценарий

1. Операция получения карты считала значение $M[0]$
2. Источник данных обновил значения клеток $M[0]$, $M[1]$
3. Операция получения карты считала значение $M[1]$

В этом случае только в $M[1]$ будут учтены показания источника данных. Естественно в последующих вызовах эти показания будут учтены в обеих клетках. Если такая несогласованность данных считается недопустимой, операция получения карты должна быть блокирующей аналогично сдвигу.

4.3.4 Fully lockfree OGM

В рамках последней модификации мы избавляемся от последнего ограничения карты и делаем операцию сдвига так же не блокирующей. Для этого мы разделим карту на два равных примыкающих друг к другу фрагмента, добавим к каждому координату, описывающую его положение (как и ранее это будет центр фрагмента) и будем хранить их в умных, разделяемых указателях (рис. 4.6) pM_1, pM_2 . Для нашей реализации важно, чтобы аппаратное обеспечение позволяло работать с такими указателями атомарно. Большинство современных архитектур поддерживают атомарные операции для стандартной C++ реализации `std::shared_ptr`. Примем размер одного фрагмента равным $\hat{N}$.

Сдвиг

Прежде чем приступить к описанию операции важно сделать следующее замечание: предлагаемая реализация сдвига является неблокирующей относи-

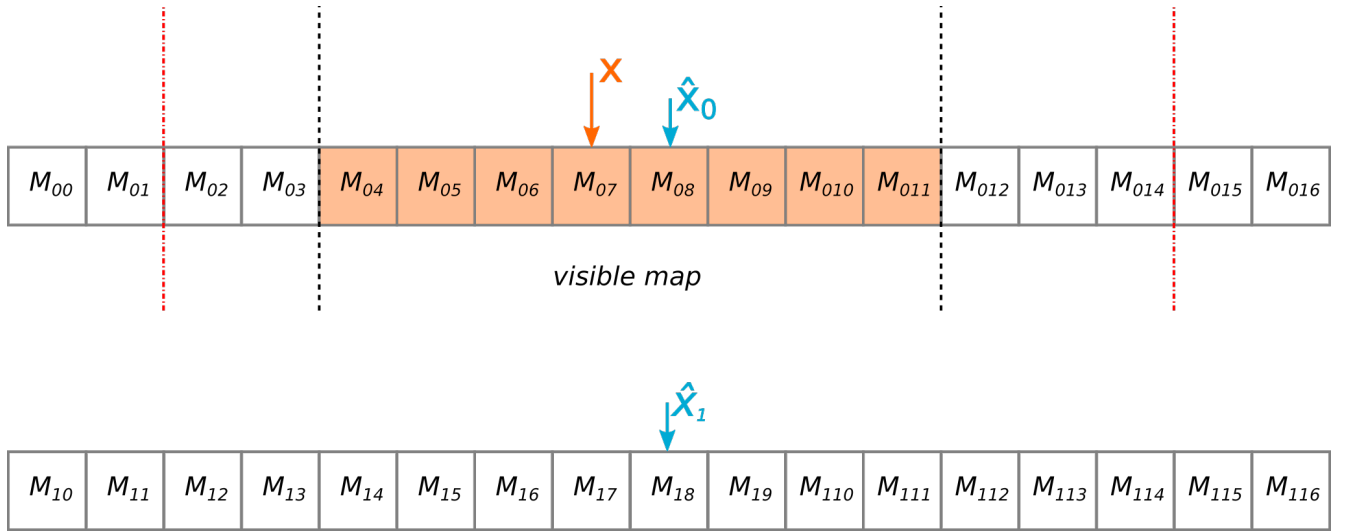


Рисунок 4.6 — Карта с неблокирующей операцией сдвига

тельно обновления и/или получения карты, но в то же время, эта версия не может выполняться одновременно с другими операциями сдвига.

При сдвиге мы определяем положение отображаемой карты после сдвига. Если отображаемая часть карты присутствует целиком на одном фрагменте, и в результате сдвига отдаляется от его центра более заданного порога Δ_{lim} (см. рис. 4.6) и при этом так же отдаляется и от второго фрагмента (проще говоря, если дальнейшее движение карты в том же направлении приведет к выходу за границы всех фрагментов), то тот фрагмент, на котором сейчас не находится карты заменяется на новый: все значения клеток (как и ранее это атомарные пары значения логарифма шансов и времени) инициализируются $\{0,0\}$, а его координата меняется таким образом, чтобы при дальнейшем движении в том же направлении отображаемая часть карты «перешла» на этот фрагмент. В противном случае, как и ранее мы просто обновляем положение отображаемой части карты (см. листинг 11). В данном алгоритме важно правильно подобрать Δ_{lim} таким образом, чтобы отображаемая часть карты не могла выйти за границы фрагмента, когда производится замена второго фрагмента (в противном случае это может повлиять на операцию получения карты, выполняемой одновременно со сдвигом).

Вычислительная сложность, как и ранее, $\Theta(1)$, в общем случае, когда обновляется только координата отображаемой части локальной карты проходимости пространства. $\Theta(\hat{N})$, если происходит замена второго фрагмента.

```

1 if Current map center is between  $M_1$  and  $M_2$  centers then
2   | return  $M$ 
3 end
4  $M_c \leftarrow$  fragment with current map;
5  $\hat{x} \leftarrow M_c$  center;
6 if  $|\hat{x} - x_{i+1}| > \Delta_{lim}$  then
7   |  $M_{new} \leftarrow$  new fragment of size  $\hat{N}$ ;
8   | for  $i \leftarrow 0$  to  $\hat{N}$  do
9     |    $M_{new}[i] \leftarrow \{0, 0\}$ ;
10    |   Swap pointer to  $M_{new}$  with second fragment (not  $M_c$ );
11    | end
12    | return  $M$ 
13 end

```

Алгоритм 11: *shift_map*(M, x_{i+1}, t_{i+1}) (fully lockfree)

Обновление карты

Обновление происходит аналогично предыдущим реализациям за исключением следующих моментов:

1. В самом начале операции происходит копирование указателей на фрагменты M_1, M_2 , чтобы гарантировать их сохранение во время выполнения операции (на случай если одновременно с этой операцией выполняется сдвиг, заменяющий один из фрагментов).
2. Т.к. операция сдвиг не может выполняться с другими операциями сдвига, в обновлении больше эта операция более не происходит. Вместо этого она либо выполняется в отдельном потоке с определённой частотой (гарантирующей, что отображаемая часть карты всегда содержится на одном или обоих фрагментах карты), либо например происходит в операции получения локальной карты проходимости пространства (если гарантируется, что эта операция не выполняется одновременно с такой же)
3. Операция применяется последовательно к обоим фрагментам карты

Таким образом вычислительная сложность остаётся $O(\min(N_P, N))$. Реальная константа будет незначительно больше, чем в предыдущей реализации из-за атомарного копирования указателей и применения на двух фрагментах (заметим что последнее может быть выполнено одновременно, но асимптотику операции в общем случае это не изменит).

Получение карты

Аналогично обновлению, операция получения почти не отличается от предыдущей реализации, за тем исключением, что первым действием операция копирует указатели, чтобы гарантировать сохранность данных во время работы. Вычислительная сложность, как и раньше $O(N)$.

4.4 Сравнение задержки обновления локальной карты проходимости пространства

Чтобы сравнить представленные модификации между собой мы подготовили их реализации на C++ и синтетические тесты для измерения времени их работы. В рамках этих тестов симулируются следующие сценарии:

- Сдвиг карты на 20 клеток.
- Добавление фрагмента на карту в одном потоке. Размер фрагмента фиксирован и равен 100. Положение фрагмента меняется и берётся циклично из списка: $-100, -50, -25, -1, 0, 1, 5, 10, 25, 50, 100$. Таким образом моделируются сценарии в которых только часть фрагмента попадает на локальную карту проходимости пространства.
- Добавление фрагмента на карту в нескольких потоках. Тот же сценарий, что и в предыдущем пункте, но запускается в нескольких потоках. Протестировано варианты с разным числом потоков: 1, 5, 9, 13.
- Получение локальной карты проходимости пространства.
- Стандартный цикл "публикации" локальной карты проходимости пространства: В 8 потоках производится сдвиг карты на 10 клеток и

добавления патча размера 100, после завершения работы всех потоков, вызывается операция получения карты, имитирующая передачу карты потребителям.

Все эксперименты повторялись для "видимых областей"разного размера N : 10, 16, 32, 64, 128, 256, 516, 1024, 2048, 4000. Для модификаций, мы брали значение $\hat{N} = 2N$ (в случае с последней модификации с полностью неблокирующими операциями полный размер карты с учётом неотображаемых областей составил $4N$)

4.4.1 Сдвиг карты

Результаты замеров представлены на рис. 4.7. Можно видеть линейный рост исходной реализации, как и было показано при обсуждении реализации. Остальные реализации имеют константное (в среднем), время выполнения, поскольку в общем случае требуют только обновления координаты x . Тем не менее константы отличаются между собой. Так самое быстрое время выполнения у полностью неблокирующей реализации, в которой не используются мьютексы для создания критических областей. Далее идёт версия карты с запасом, которая использует стандартный `std::mutex`. Наконец, худшее время выполнения среди модификаций имеет версия с неблокирующим обновлением, поскольку здесь используется блокировка `std::shared_mutex`. Отметим также что на малых размерах наивная (оригинальная) реализация показывает себя эффективнее модификаций.

4.4.2 Добавление фрагмента в одном потоке

Результаты замеров представлены на рис. 4.8. Как и в прошлом эксперименте оригинальная реализация имеет линейную сложность. Такую же сложность имеет версия карты с запасом. Такое поведения объясняется необходимостью приводить все клетки массива к одному моменту времени. Этим же объясняется большее время, которое выполняется сдвиг карты с запасом в

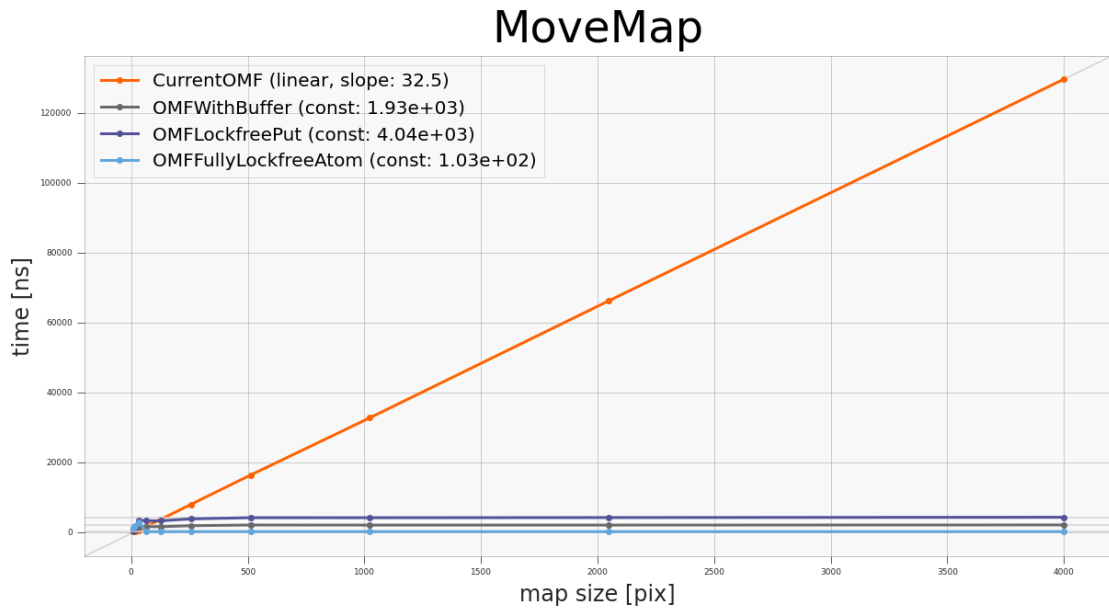


Рисунок 4.7 — График зависимости времени сдвига карты от размера карты.

сравнении с оригинальной версией: ей требуется обновить больше число клеток (вдвое больше, если быть точным, что хорошо отражают вычисленные экспериментально коэффициенты наклона). В то же время оставшиеся модификации с увеличением карты выходят на константу, т.к. не обновляют всю карту целиком, а только те клетки, что пересекаются с фрагментом. При этом у полностью неблокирующей реализации производительность ниже, чем у версии с неблокирующим обновлением. Вероятно это объясняется необходимостью атомарно копировать указатели на фрагменты карты, обращаться к элементам через указатель и повторению операции на двух массивах (последние два фактора снижают локальность данных и приводит к большему количеству промахов кэша).

4.4.3 Добавление фрагмента в нескольких потоках

Результаты замеров представлены на рис. [4.9](#) [4.10](#) [4.11](#) [4.12](#). Здесь применимы рассуждения из предыдущего пункта. Можно отметить, что с ростом числа потоков уменьшается размер карты, при котором неблокирующие модификации становятся вычислительно эффективнее оригинальной реализации.

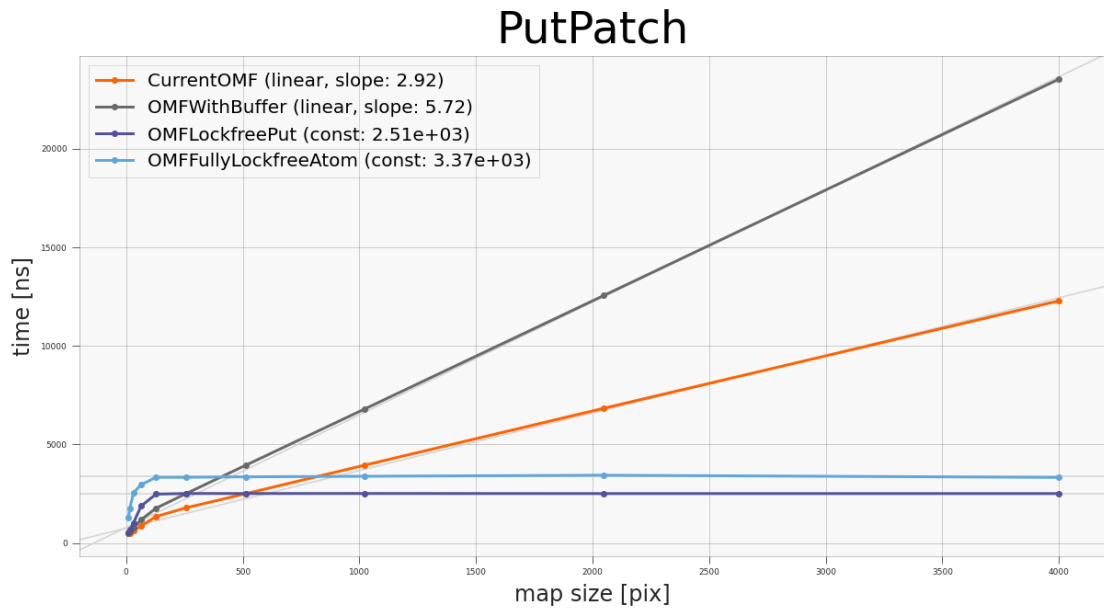


Рисунок 4.8 — График зависимости времени добавления фрагмента на карту от размера карты.

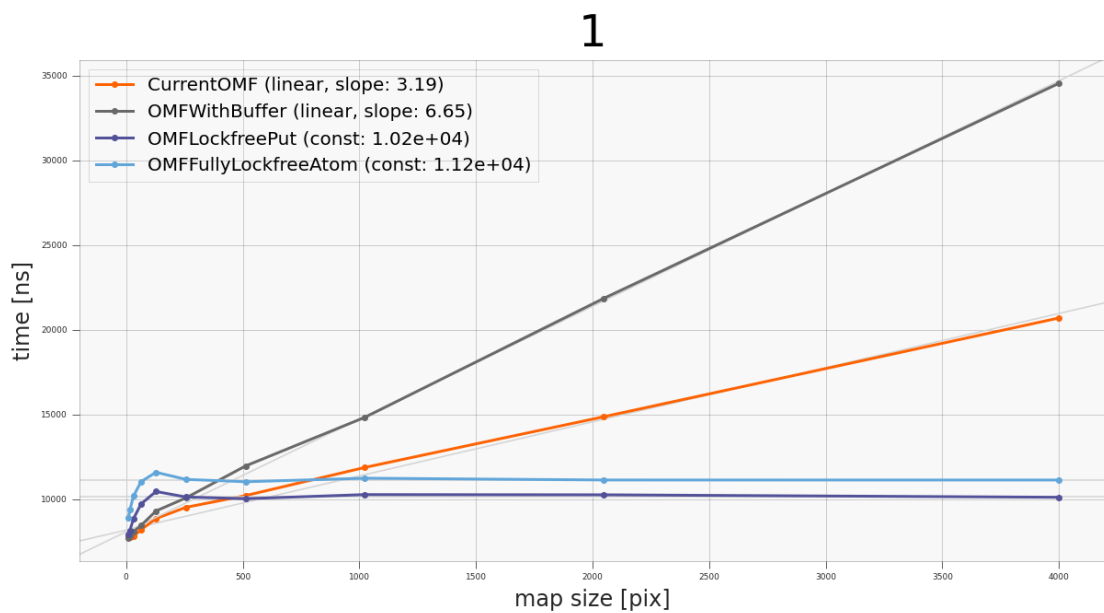


Рисунок 4.9 — График зависимости времени добавления фрагмента на карту в 1 потоке от размера карты.

4.4.4 Получение карты

Результаты замеров представлены на рис. 4.13. Здесь результаты заметно отличаются от предыдущих экспериментов. Время получения оригинальной

5

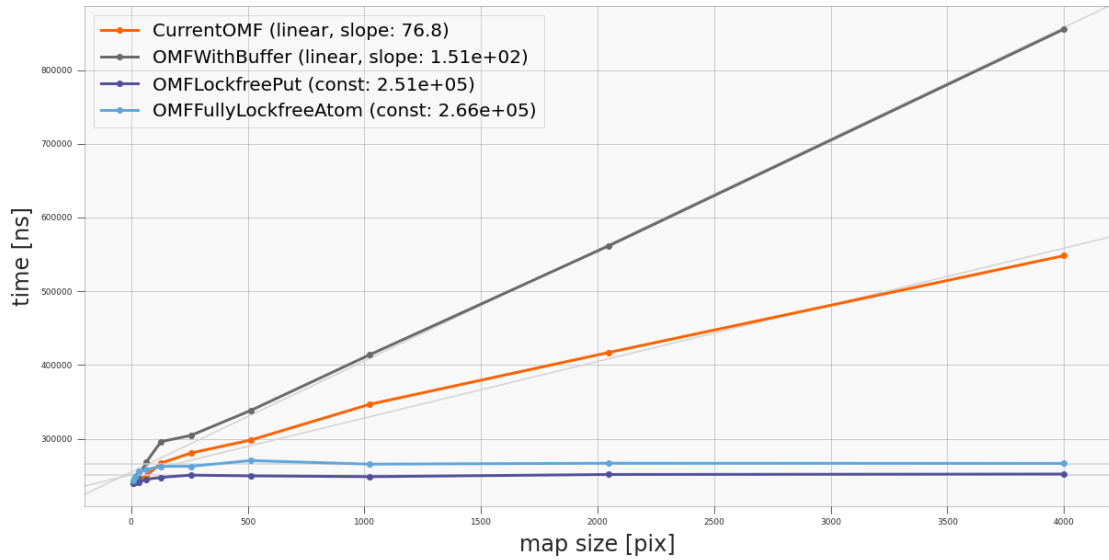


Рисунок 4.10 — График зависимости времени добавления патча на карту в 5 потоках от размера карты.

9

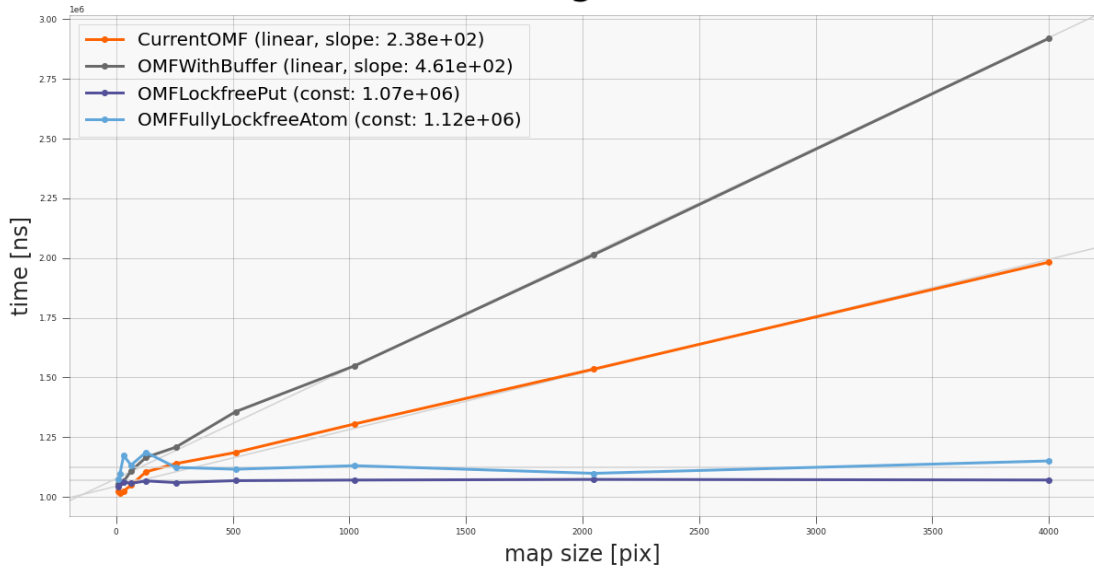


Рисунок 4.11 — График зависимости времени добавления патча на карту в 9 потоках от размера карты.

реализации и карты с запасом с увеличением карты возрастает настолько медленно, что кажется константным в сравнении с неблокирующими реализациями (тем не менее оно линейно) В последних же за счёт необходимости атомарно считывать значения клеток, а также приводить их к общему моменту времени, наблюдается линейный рост.

13

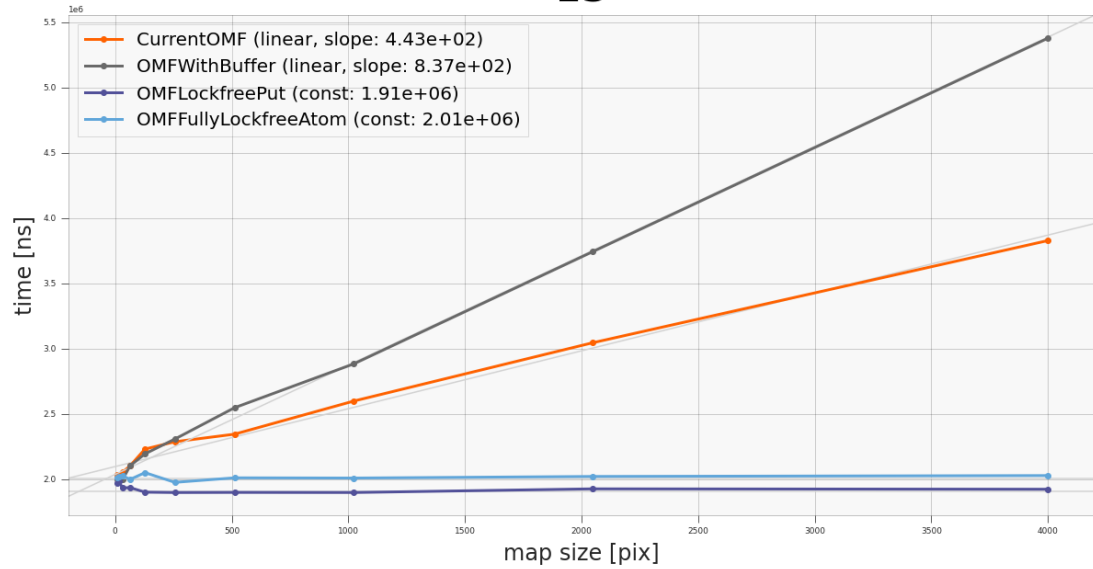


Рисунок 4.12 — График зависимости времени добавления патча на карту в 13 потоках от размера карты.

GetMap

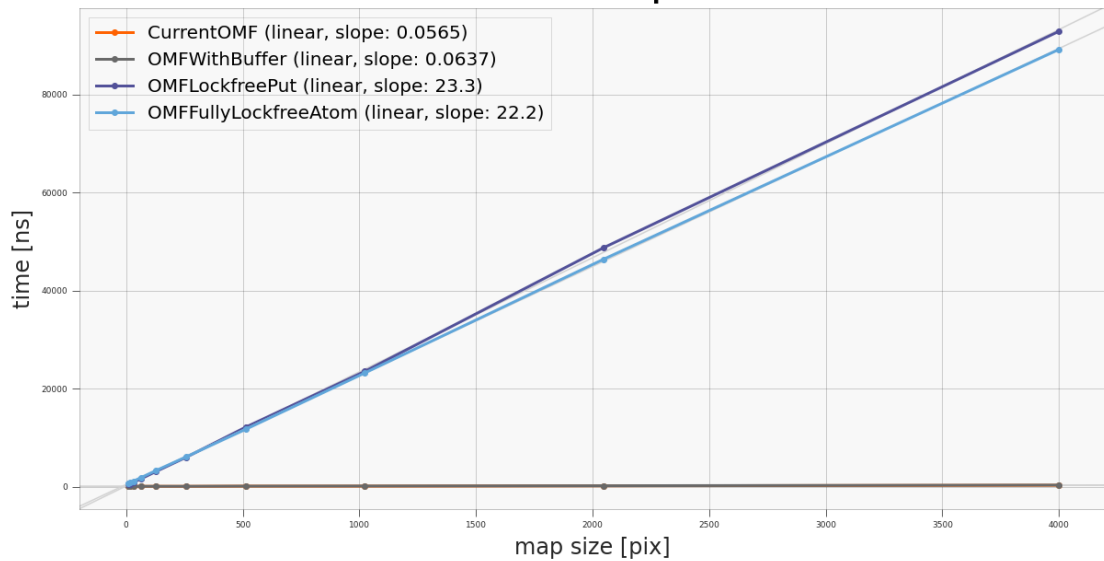


Рисунок 4.13 — График зависимости времени получения карты от её размера. График BaseOGM практически полностью совпадает с OMFWithBuffer и на таком масштабе они не отличимы.

4.4.5 Стандартный цикл "публикации" карты

Наиболее показательная для нас характеристика, показывающая какая реализация покажет себя лучше всего в реальном сценарии использования. Результаты приведены на рис. 4.14. Можно видеть что все представленные реализации имеют линейную вычислительную сложность относительно размера карты. Все представленные модификации превосходят по эффективности оригинальную реализацию, при чём каждая модификация показывает себя лучше предыдущей (по крайней мере по мере увеличения размера карты). Лучшую производительность показала полностью неблокирующая реализация: её экспериментально установленная константа линейной сложности в 13.89 меньше исходной реализации. Далее идёт модификация с неблокирующим обновлением содержимого карты (в 11.52 быстрее оригинальной исходной реализации). Наконец модификация карты с запасом, при всей своей тривиальности показала увеличение производительности в 6.45 раз.

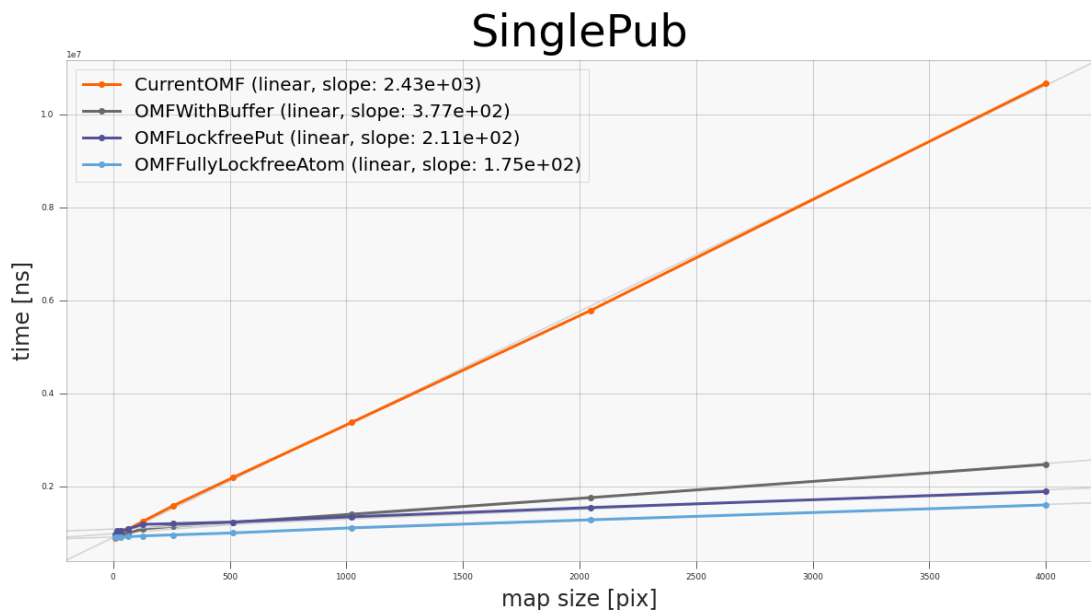


Рисунок 4.14 — График зависимости времени одной итерации "публикации" карты от её размера.

4.5 Реализация для 2D случая

Показав работоспособность неблокирующей модификации локальной карты проходимости пространства для одномерного случая, представим её расширение для двумерного случая. Как и ранее, мы разбиваем полную карту окружения на несколько, теперь уже 2D, массивов, состоящих из уже описанных ранее атомарных пар логарифма шансов и времени. Для движения в плоскости потребуется 9 массивов (рис. 4.15). Если видимая область карты пересекает границы отмеченные штрих-пунктирной линией создаются новые массивы и заменяют часть предыдущих таким образом, чтобы текущий массив, в котором находится видимая область карты, стал центральным. Важно отметить, что отступ этих линий от краёв массива должен быть подобран так, чтобы за одну итерацию сдвига, видимая область не могла преодолеть расстояние между линией и границей массива. В противном случае если параллельно будет происходить получение карты, то массив, в который "переходит" видимая область может быть ещё не загружена, что приведёт к ошибке выполнения.

Легко видеть, что такая реализация значительно увеличивает размер памяти, требуемой для хранения полученной структуры данных. Кроме того, как и в одномерном случае, операции получения и обновления карты должны применяться ко всем 9 массивам. Всё это существенно увеличивает вычислительную сложность. Требуется проведение дополнительных экспериментов для определения условий (средняя скорость движения БТС/средний размер добавляемых фрагментов/количество источников данных и частота их работы/масштаб локальной карты проходимости пространства), при которых такая реализация будет вычислительно эффективнее оригинальной версии.

4.6 Выводы

В данной главе было предложено три последовательных модификации локальной карты проходимости пространства и операций применяемых к ней, направленных на снижение вычислительной сложности производимых операций при условии наличия нескольких источников данных, работающих

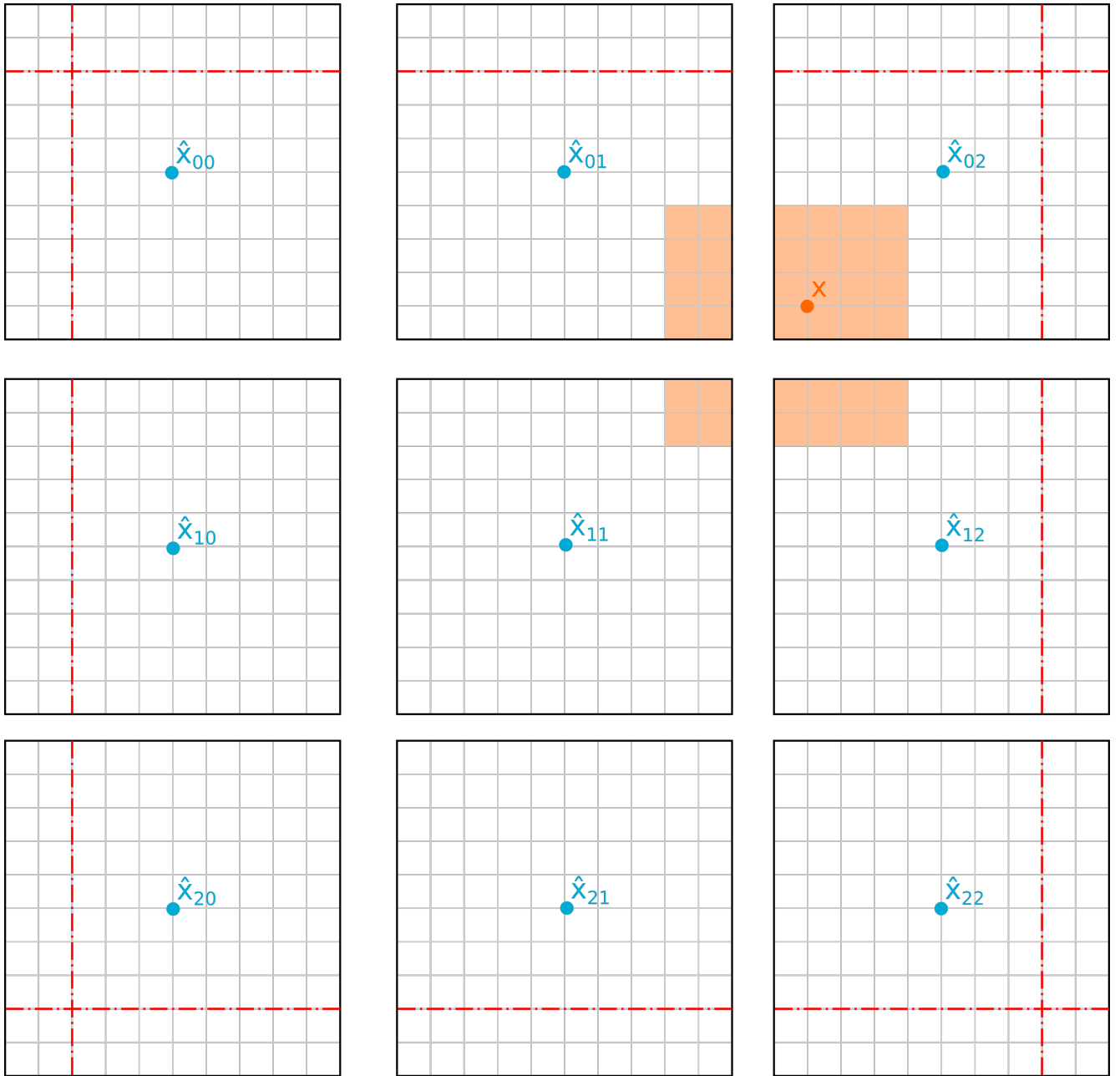


Рисунок 4.15 — Двумерная реализация локально карты проходимости пространства с неблокирующими операциями

параллельно и независимо друг от друга. Первая увеличивала размер массива, хранящего карту, с целью снизить стоимость сдвига карты. Вторая меняла формат элементов массива, которая позволила бы а) одновременное обновление карты несколькими источниками и б) избежать излишней синхронизации между всеми клетками карты, тем самым снизив вычислительную сложность обновления. Последняя модификация разбивала карты на регионы и позволяла динамически подгружать новые, не прерывая операции обновления/получения карты, тем самым делая структуру данных полностью неблокирующей.

Были подготовлены реализации оригинальной версии и предложенных модификаций для одномерного случая. Для этих реализаций мы произвели измерения времени выполнения разных операций над картой, при разных её размерах. Исследуемые операции включали: сдвиг, получение карты, добавления одного фрагмента на карту, одновременное добавление нескольких фрагментов на карту, стандартную итерацию «публикации» карты – несколько сдвигов карты с добавлением фрагментов, после которых выполнялась операция получения.

Результаты экспериментов подтвердили теоретическую вычислительную сложность алгоритмов:

1. Для сдвига исходная реализация имеет вычислительную сложность $\Theta(N)$, все предлагаемые модификации имеют (амортизационную) сложность $\Theta(1)$. При этом экспериментально вычисленная константа наименьшая у последней модификации с полностью неблокирующими операциями – 103 нс, далее идёт карта с запасом – 1930 нс, и карта с неблокирующей операцией обновления – 4040 нс.
2. Операция обновления содержимого имеет линейную сложность для исходной реализации и карты с запасом. При этом сложность для карты с запасом больше ввиду большего количества обновляемых клеток. У карты с неблокирующей операцией обновления и карты с полностью неблокирующими операциями константная вычислительная сложность: 2510 и 3370 соответственно.
3. Для получения локальной карты проходимости пространства все алгоритмы имеют линейную сложность $\Theta(N)$, при этом у модификаций с неблокирующими операциями вычислительная сложность на несколько порядков выше: 23.3 для карты с неблокирующим обновлением, 22.2 для карты со всеми неблокирующими операциями, при 0.06 у исходной реализации и карты с запасом.
4. Для обновления карты несколькими источниками данных (одновременно, для тех реализаций где это возможно), результаты аналогичны эксперименту с единичным обновлением карты. При увеличении числа источников данных наблюдается уменьшение «оптимального» размера карты, при котором модификации с неблокирующими операциями становятся вычислительно эффективнее исходной реализации.

5. В эксперименте с симуляцией одной итерации публикации карты все алгоритмы показали линейную сложность $O(N)$. При этом исходная реализация имеет линейный рост с коэффициентом 2430 нс/пикс, карта с запасом – 377 нс/пикс (в 6.45 раз быстрее исходной реализации), карта с неблокирующей операцией обновления – 211 нс/пикс (в 11.52 раз быстрее исходной реализации), карта с полностью не блокирующими операциями 175 нс/пикс (в 13.86 раз быстрее исходной реализации)

Было предложено расширение для двумерного сценария карты с полностью неблокирующими операциями, показавшей лучшую производительность в последнем эксперименте. В дальнейших работах мы планируем подготовить реализацию данного расширения и провести его тестирования на реальных данных.

4.7 Заключение к главе 4

Заключение

Основные результаты работы заключаются в следующем.

1. На основе анализа ...
2. Численные исследования показали, что ...
3. Математическое моделирование показало ...
4. Для выполнения поставленных задач был создан ...

И какая-нибудь заключающая фраза.

Автор выражает благодарность своему научному руководителю И. П. Николаеву, а также научному консультанту О.С. Шипитько за помощь в постановке задач и обсуждении результатов

В заключение благодарю коллектив компании «ЭвоКарго». Отдельно хочу поблагодарить К. В. Садовскова, А.П. Рудоманенко, С. А. Скрипичина, П. А. Комиссарову за дискуссии, обсуждение результатов и помощь в проведении экспериментов. Благодарю авторов шаблона *Russian-Phd-LaTeX-Dissertation-Template* за помощь в оформлении работы.

Список сокращений и условных обозначений

БТС	Беспилотное транспортное средство
ГПУ	Графическое процессорное устройство
ОЗУ	Оперативное запоминающее устройство
ПК	Персональный компьютер
ПО	Программное обеспечение
ЦПУ	Центральное процессорное устройство
CUDA	Compute Unified Device Architecture
FN	False Negative
RGB	Red, Green, Blue
ROS	Robot Operating System
TP	True Positive
UML	Unified Modeling Language

Список литературы

1. Taxonomy and definitions for terms related to on-road motor vehicle automated driving systems [Текст] / S. O.-R. A. V. S. Committee [и др.] // SAE Standard J. — 2014. — Т. 3016. — С. 1.
2. Scene representations for autonomous driving: an approach based on polygonal primitives [Текст] / M. Oliveira [и др.] // Robot 2015: Second Iberian Robotics Conference: Advances in Robotics, Volume 1. — Springer. 2015. — С. 503—515.
3. *Feng, T.* A survey of world models for autonomous driving [Текст] / T. Feng, W. Wang, Y. Yang // arXiv preprint arXiv:2501.11260. — 2025.
4. *Paskin, M.* Robotic mapping with polygonal random fields [Текст] / M. Paskin, S. Thrun // arXiv preprint arXiv:1207.1399. — 2012.
5. Generating evidential BEV maps in continuous driving space [Текст] / Y. Yuan [и др.] // ISPRS Journal of Photogrammetry and Remote Sensing. — 2023. — Т. 204. — С. 27—41.
6. *Elfes, A.* Occupancy grids: a probabilistic framework for mobile robot perception and navigation [Текст] : дис. ... канд. / Elfes Alberto. — Ph. D. Thesis. Department of electrical, computer engineering. Carnegie ..., 1989.
7. *Moravec, H. P.* Sensor Fusion in Certainty Grids for Mobile Robots [Текст] / H. P. Moravec // AI Magazine. — 1988. — Т. 9, № 2. — С. 61. — URL: <https://ojs.aaai.org/aimagazine/index.php/aimagazine/article/view/676>.
8. *Moravec, H.* High resolution maps from wide angle sonar [Текст] / H. Moravec, A. Elfes // Proceedings. 1985 IEEE international conference on robotics and automation. Т. 2. — IEEE. 1985. — С. 116—121.
9. An overview of autonomous vehicles sensors and their vulnerability to weather conditions [Текст] / J. Vargas [и др.] // Sensors. — 2021. — Т. 21, № 16. — С. 5397.
10. *Сысоева, С.* Актуальные технологии и применения датчиков автомобильных систем активной безопасности. Часть 6. Радары [Текст] / С. Сысоева // Компоненты и технологии. — 2007. — № 68. — С. 67—76.

11. A single LiDAR-based feature fusion indoor localization algorithm [Текст] / Y.-T. Wang [et al.] // *Sensors*. — 2018. — Vol. 18, no. 4. — P. 1294.
12. *Luo, Z.* A probability occupancy grid based approach for real-time LiDAR ground segmentation [Текст] / Z. Luo, M. V. Mohrenschildt, S. Habibi // *IEEE Transactions on Intelligent Transportation Systems*. — 2019. — Т. 21, № 3. — С. 998—1010.
13. Data driven prediction architecture for autonomous driving and its application on apollo platform [Текст] / K. Xu [и др.] // *2020 IEEE Intelligent Vehicles Symposium (IV)*. — IEEE. 2020. — С. 175—181.
14. *Rosenband, D. L.* Inside Waymo's self-driving car: My favorite transistors [Текст] / D. L. Rosenband // *2017 Symposium on VLSI Circuits*. — IEEE. 2017. — С. C20—C22.
15. Experience of the ARGO autonomous vehicle [Текст] / M. Bertozzi [и др.] // *Enhanced and Synthetic Vision 1998*. Т. 3364. — SPIE. 1998. — С. 218—229.
16. Radar/lidar sensor fusion for car-following on highways [Текст] / D. Göhring [и др.] // *The 5th International Conference on Automation, Robotics and Applications*. — IEEE. 2011. — С. 407—412.
17. Lidar-based glass detection for improved occupancy grid mapping [Текст] / H. Tibebu [и др.] // *Sensors*. — 2021. — Т. 21, № 7. — С. 2263.
18. Sensor and Sensor Fusion Technology in Autonomous Vehicles: A Review [Текст] / D. J. Yeong [и др.] // *Sensors*. — 2021. — Т. 21, № 6. — URL: <https://www.mdpi.com/1424-8220/21/6/2140>.
19. Obstacle detection quality as a problem-oriented approach to stereo vision algorithms estimation in road situation analysis [Текст] / A. Smagina [и др.] // *Journal of Physics: Conference Series*. Т. 1096. — IOP Publishing. 2018. — С. 012035.
20. Deep learning on monocular object pose detection and tracking: A comprehensive overview [Текст] / Z. Fan [и др.] // *ACM Computing Surveys*. — 2022. — Т. 55, № 4. — С. 1—40.
21. Manipulator grabbing position detection with information fusion of color image and depth image using deep learning [Текст] / D. Jiang [и др.] // *Journal of Ambient Intelligence and Humanized Computing*. — 2021. — Т. 12, № 12. — С. 10809—10822.

22. *Sasiadek, J. Z.* Sensor fusion [Текст] / J. Z. Sasiadek // Annual Reviews in Control. — 2002. — Т. 26, № 2. — С. 203—228.
23. *Duong, T.* Autonomous navigation in unknown environments with sparse bayesian kernel-based occupancy mapping [Текст] / T. Duong, M. Yip, N. Atanasov // IEEE Transactions on Robotics. — 2022. — Т. 38, № 6. — С. 3694—3712.
24. A Fusion of Dynamic Occupancy Grid Mapping and Multi-object Tracking Based on Lidar and Camera Sensors [Текст] / Y. Wang [и др.] // 2020 3rd International Conference on Unmanned Systems (ICUS). — 2020. — С. 107—112.
25. BEVFusion: Multi-Task Multi-Sensor Fusion with Unified Bird’s-Eye View Representation [Текст] / Z. Liu [и др.]. — 2024. — arXiv: [2205.13542](https://arxiv.org/abs/2205.13542) [cs.CV]. — URL: <https://arxiv.org/abs/2205.13542>.
26. Object Detection in Autonomous Driving Scenarios Based on an Improved Faster-RCNN [Текст] / Y. Zhou [и др.] // Applied Sciences. — 2021. — Т. 11, № 24. — URL: <https://www.mdpi.com/2076-3417/11/24/11630>.
27. *Bernardin, K.* Evaluating multiple object tracking performance: the clear mot metrics [Текст] / K. Bernardin, R. Stiefelhagen // EURASIP Journal on Image and Video Processing. — 2008. — Т. 2008, № 1. — С. 246309.
28. Performance measures and a data set for multi-target, multi-camera tracking [Текст] / E. Ristani [и др.] // European conference on computer vision. — Springer. 2016. — С. 17—35.
29. Hota: A higher order metric for evaluating multi-object tracking [Текст] / J. Luiten [и др.] // International journal of computer vision. — 2021. — Т. 129, № 2. — С. 548—578.
30. Yolop: You only look once for panoptic driving perception [Текст] / D. Wu [и др.] // Machine Intelligence Research. — 2022. — Т. 19, № 6. — С. 550—562.
31. Optimal Vehicle Pose Estimation Network Based on Time Series and Spatial Tightness with 3D LiDARs [Текст] / H. Wang [и др.] // Remote Sensing. — 2021. — ОКТ. — Т. 13. — С. 4123.
32. *Arnon, D. S.* A linear time algorithm for the minimum area rectangle enclosing a convex polygon [Текст] / D. S. Arnon, J. P. Giesermann. — 1983.

33. *Kuhn, H. W.* The Hungarian method for the assignment problem [Текст] / H. W. Kuhn // Naval research logistics quarterly. — 1955. — Т. 2, № 1/2. — С. 83—97.
34. *Российской Федерации, М. регионального развития.* Автомобильные дороги [Текст] / М. регионального развития Российской Федерации. — Москва, Российская Федерация : Министерство регионального развития Российской Федерации, 2013. — (Свод правил ; SP 34.13330.2012).
35. *Межгосударственный Совет по стандартизации, м. и. с.* Система проектной документации для строительства. Правила выполнения рабочей документации автомобильных дорог [Текст] : ГОСТ / м. и. с. Межгосударственный Совет по стандартизации. — СПДС, 2014. — Введен в действие с 01.01.2015.
36. *NVIDIA.* CUDA, release: 10.2.89 [Текст] / NVIDIA, P. Vingelmann, F. H. Fitzek. — 2020. — URL: <https://developer.nvidia.com/cuda-toolkit>.
37. Fast euclidean cluster extraction using gpus [Текст] / A. Nguyen [и др.] // Journal of Robotics and Mechatronics. — 2020. — Т. 32, № 3. — С. 548—560.
38. *Fischler, M. A.* Random sample consensus: a paradigm for model fitting with applications to image analysis and automated cartography [Текст] / M. A. Fischler, R. C. Bolles // Commun. ACM. — New York, NY, USA, 1981. — Июнь. — Т. 24, № 6. — С. 381—395. — URL: <https://doi.org/10.1145/358669.358692>.
39. *Rasmussen, C. E.* Gaussian processes in machine learning [Текст] / C. E. Rasmussen // Summer school on machine learning. — Springer, 2003. — С. 63—71.
40. *Stein, M. L.* Interpolation of spatial data: some theory for kriging [Текст] / M. L. Stein. — Springer Science & Business Media, 1999.
41. *Jones, A.* The matérn class of covariance functions [Текст] / A. Jones. — 07.2021. — URL: <https://andrewcharlesjones.github.io/journal/maternal-kernels.html> (дата обр. 15.04.2025).
42. *Moller, K.* Overcoming Blind Spots: Occlusion Considerations for Improved Autonomous Driving Safety [Текст] / K. Moller, R. Trauth, J. Betz // 2024 IEEE Intelligent Vehicles Symposium (IV). — 2024. — С. 819—826.

43. Reconstructed three-dimensional electron momentum density in lithium: A Compton scattering study [Текст] / Y. Tanaka [и др.] // Phys. Rev. B. — 2001. — ЯНВ. — Т. 63, вып. 4. — С. 045120. — URL: <https://link.aps.org/doi/10.1103/PhysRevB.63.045120>.
44. *Leach, G.* Improving worst-case optimal Delaunay triangulation algorithms [Текст] / G. Leach // 4th Canadian Conference on Computational Geometry. Т. 2. — Citeseer. 1992. — С. 15.
45. *Rennich, S.* Cuda C/C++ streams and concurrency [Текст] / S. Rennich. — URL: <https://developer.download.nvidia.com/CUDA/training/StreamsAndConcurrencyWebinar.pdf> (дата обр. 17.08.2025).
46. 08.2025. — URL: <https://docs.nvidia.com/nsight-systems/index.html> (дата обр. 17.08.2025).
47. Computing two-dimensional Delaunay triangulation using graphics hardware [Текст] / G. Rong [и др.] // Proceedings of the 2008 symposium on Interactive 3D graphics and games. — 2008. — С. 89—97.
48. *Chekanov, M.* L-shape fitting algorithm for 3D object detection in bird's-eye-view in an autonomous driving system [Текст] / M. Chekanov, O. Shipitko. — 2024. — URL: <http://dx.doi.org/10.1117/12.3023477>.
49. *М. О. Чеканов С. В. Кудряшова, О. С. Ш.* Трехмерная детекция объектов на основе L- shape модели в автономных системах движения [Текст] / О. С. Ш. М. О. Чеканов С. В. Кудряшова // Сенсорные системы. — 2025. — Т. 39, № 1. — С. 66—78.
50. ROS: an open-source Robot Operating System [Текст] / M. Quigley [и др.] // ICRA workshop on open source software. Т. 3. — Kobe. 2009. — С. 5.
51. *Bradski, G.* The OpenCV Library [Текст] / G. Bradski // Dr. Dobb's Journal of Software Tools. — 2000.
52. *Rusu, R. B.* 3D is here: Point Cloud Library (PCL) [Текст] / R. B. Rusu, S. Cousins // IEEE International Conference on Robotics and Automation (ICRA). — Shanghai, China : IEEE, 05.2011.
53. *Woodall, W.* pcl-conversions [Текст] / W. Woodall. — URL: https://wiki.ros.org/pcl_conversions (дата обр. 17.08.2025).

54. *Martin, R. C.* The dependency inversion principle [Текст] / R. C. Martin // C Report. — 1996. — Т. 8, № 6. — С. 61—66.
55. *Marder-Eppstein, E.* tf2-ros [Текст] / E. Marder-Eppstein, W. Meeussen. — URL: https://wiki.ros.org/tf2_ros (дата обр. 17.08.2025).
56. Head First Design Patterns: A Brain-Friendly Guide [Текст] / E. Freeman [и др.]. — "O'Reilly Media, Inc.", 2004.

Список рисунков

1.1	Пример сопоставления вычисленных и эталонных идентификаторов объектов для IDF1 [27]	22
2.1	Наивный подход к проецированию ограничивающей рамки в 2D на вид сверху.	25
2.2	Неверная проекция грузовика. Зона слева от грузовика воспринимается, как часть объекта, и мешает объехать его.	26
2.3	Проекция двумерной ограничивающей рамки на вид сверху с учетом сегментации проезжей части.	27
2.4	Проекция двумерной ограничивающей рамки на вид сверху с учетом сегментации проезжей части.	31
2.5	Пример неправильно построенной ограничивающей рамки.	33
2.6	Примеры работы сравниваемых алгоритмов построения ограничивающих рамок.	35
2.7	График зависимости среднего коэффициента Жаккара от ограничения дистанции до детектируемых препятствий.	37
2.8	Примеры оформления поперечного профиля из ГОСТ 21.701-2013 [35]	38
2.9	Демонстрация работы алгоритма 1 на данных, полученных с БТС	41
2.10	Сравнение регрессии на основе гауссовских процессов с использованием в качестве ядра радиальной базисной функции и ядра Матерна [41]	42
2.11	Пример работы алгоритма разделения облака точек земли и препятствий.	45
2.12	Демонстрация влияния окклюзии на безопасность движения БТС [42].	46
2.13	Пример семантической сегментации проезжей части моделью YOLO на изображении с камеры БТС	47
2.14	Иллюстрация алгоритма проецирования в сценарии, когда проезжая часть лежит в известной плоскости. Для облегчения восприятия проецируемая сетка представлена одним рядом	49
2.15	Пример заслонённой области свободной для проезда	50

2.16	Иллюстрация алгоритма проецирования в сценарии, когда поверхность проезжей части удовлетворяет уравнению $z = z(x,y)$. Для облегчения восприятия проецируемая сетка представлена одним рядом	51
2.17	Линейная интерполяция z в треугольнике.	52
2.18	Линейная интерполяция z в треугольнике.	53
2.19	Профилирование в Nsight Systems изменившейся части программы, реализующей алгоритм 2, при добавлении дополнительных точек для аппроксимации.	56
2.20	График полного времени работы алгоритма 2 с дополнительными тестовыми точками при последовательном и параллельном вычислении.	57
2.21	Пример «выреза» слишком большого размера, классификация которого как «занято» была бы некорректна.	58
2.22	Демонстрация работы алгоритма проецирования сегментации проезжей части на вид сверху.	59
3.1	Диаграмма компонентов программного комплекса Occupancy Map Fusion на языке UML(Unified Modeling Language).	63
3.2	Диаграмма деятельности программного комплекса Occupancy Map Fusion.	64
3.3	Иллюстрация поля зрения, занимаемого препятствием.	67
4.1	Расположение локальной карты проходимости и её фрагмента от одного из источников данных относительно робота и глобальной системы координат	80
4.2	Демонстрация формирования очереди фрагментов. Горизонтальные отрезки обозначают время формирования фрагмента каждым источником: начало отрезка соответствует времени регистрации данных $t_{s,k}$, конец отрезка соответствует времени завершения формирования фрагмента и момент добавления его в очередь фрагментов. Внизу иллюстрации продемонстрирован итоговый порядок фрагментов в очереди: фрагменты извлекаются из очереди в порядке справа налево	81

4.3	Иллюстрация алгоритма обновления локальной карты проходимости пространства фрагментом от источника данных s . 4.3a: время регистрации фрагмента новее последнего времени обновления карты. 4.3б: Сценарий, когда время регистрации фрагмента старше последнего времени обновления карты	83
4.4	Оригинальная реализация локальной карты проходимости пространства	84
4.5	Карта с подготовленными окрестностями	87
4.6	Карта с неблокирующей операцией сдвига	93
4.7	График зависимости времени сдвига карты от размера карты.	97
4.8	График зависимости времени добавления фрагмента на карту от размера карты.	98
4.9	График зависимости времени добавления фрагмента на карту в 1 потоке от размера карты.	98
4.10	График зависимости времени добавления патча на карту в 5 потоках от размера карты.	99
4.11	График зависимости времени добавления патча на карту в 9 потоках от размера карты.	99
4.12	График зависимости времени добавления патча на карту в 13 потоках от размера карты.	100
4.13	График зависимости времени получения карты от её размера. График BaseOGM практически полностью совпадает с OMFWithBuffer и на таком масштабе они не отличимы.	100
4.14	График зависимости времени одной итерации "публикации" карты от её размера.	101
4.15	Двумерная реализация локально карты проходимости пространства с неблокирующими операциями	103

Список таблиц

1	Сравнение характеристика датчиков, используемых в колёсной робототехники	17
2	Статистики полученных коэффициента Жаккара для сравниваемых алгоритмов	36